Developing BPL Processes

Version 2026.1
2026-04-20

InterSystems Corporation One Congress Street Boston MA 02114 www.intersystems.com

Developing BPL Processes
PDF generated on 2026-04-20
InterSystems IRIS Version 2026.1

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)
Tel: +1-617-621-0700
Tel: +44 (0) 844 854 2917
Email: support@InterSystems.com

Table of Contents

1

Introduction to BPL Processes

BPL business processes are a form of business logic you can use within interoperability productions. A BPL business process is written in the Business Process Language (BPL) and is derived from Ens.BusinessProcessBPL. It is identical in every way to a class derived from Ens.BusinessProcess, except that it supports BPL.

Note that there is overlap among the options available in business processes, data transformations, and business rules. For a comparison, see Comparison of Business Logic Tools.

1.1 BPL Features

BPL is a language used to describe executable business processes within a standard XML document. BPL syntax is based on several of the proposed XML standards for defining business process logic, including the Business Process Execution Language for Web Services (BPEL4WS or BPEL) and the Business Process Management Language (BPML or BPMI).

BPL is a superset of other proposed XML-based standards, in that it provides additional elements whose purpose is to help you build integration solutions. These additional elements include support for the following:

- Execution flow control elements such as <branch>, <if>, <switch>, <foreach>, <while>, and <until>. For information about how to use these and other BPL syntax elements, see Business Process and Data Transformation Language Reference.
- Generation of executable code from business process logic.
- Embedding SQL and custom-written code into the business process logic.

  You can use ObjectScript or Python as the scripting language.
- The BPL Editor, a full-featured, visual modeling tool for graphically viewing and editing business process logic. This tool includes complete round-trip engineering between the visual and BPL representations of the business process. Changes to one representation are automatically reflected in the other.
- Automatic support for both asynchronous and synchronous messaging between business processes and other members of an integration solution. BPL streamlines this difficult and error-prone programming task.
- Persistent state. BPL permits a long-running business process to automatically suspend execution—and efficiently save its execution state to a built-in, persistent cache—whenever it is inactive; for example, when it is waiting for an asynchronous response. InterSystems IRIS automatically manages all state preservation and the ability to smoothly resume processing.
- Rich and varied data transformation services, including SQL queries embedded within the business process.

You can create a BPL business process using the Management Portal or your IDE. The recommended way is to use the BPL Editor in the Management Portal.

1.2 A BPL Business Process Can Be Reusable

A *business process component* or *BPL component* is a BPL business process that a programmer wishes to identify as a modular, reusable sequence of steps in the BPL language. A BPL component is analogous to a function, macro, or subroutine in other programming languages.

Only another BPL business process can call a BPL component. It does this using the BPL <call> element. The BPL business process component performs tasks, then returns control to the BPL business process that called it.

The production architecture already allows one BPL business process to call another BPL business process. The optional *component* designation simply provides convenience. It allows you to classify certain BPL business processes as simpler, lower-level components that:

- Are not intended to run as stand-alone business processes (although nothing in the architecture prevents this)
- May be reusable (in the sense of a function, macro, or subroutine in the BPL language)

Business processes that are not components are assumed to have more complex, special-purpose designs, and to operate at a higher conceptual level than components. It is expected that BPL non-components call BPL components to accomplish tasks.

Important: There is no requirement that you use the component designation for any BPL business process. It is available as a convenience for any BPL programmer who prefers it.

You make a business process into a component by setting an attribute of the top-level <process> container for the BPL business process. The attribute is called *component* and you can set it to 1 (true) or 0 (false). For syntax details, see Business Process and Data Transformation Language Reference.

To set the value of the *component* attribute, you can do either of the following:

- In the **General** tab of the BPL Editor, select **Is component** to include this process in the Component Library.
- Edit the BPL <process> element within the XData BPL block in the class code in an IDE.

To set up a <call> to a component from a BPL business process, see Adding a Call Activity.

1.3 See Also

- Comparison of Business Logic Tools
- Introduction to the BPL Editor
- Creating BPL Business Processes
- Editing a BPL Diagram

2

Introduction to the BPL Editor

This page introduces the BPL Editor (the **Business Process Designer** page), which enables you to create BPL business processes for interoperability productions.

For information on the legacy UI, see Introduction to the Legacy BPL Editor.

2.1 Accessing the Editor

To access this page:

1. Log in to the Management Portal.
2. Click **Interoperability** > **Build** > **Business Processes**.
3. Click **Open in the New UI**.

You can also access the BPL Editor from the Production Configuration page.

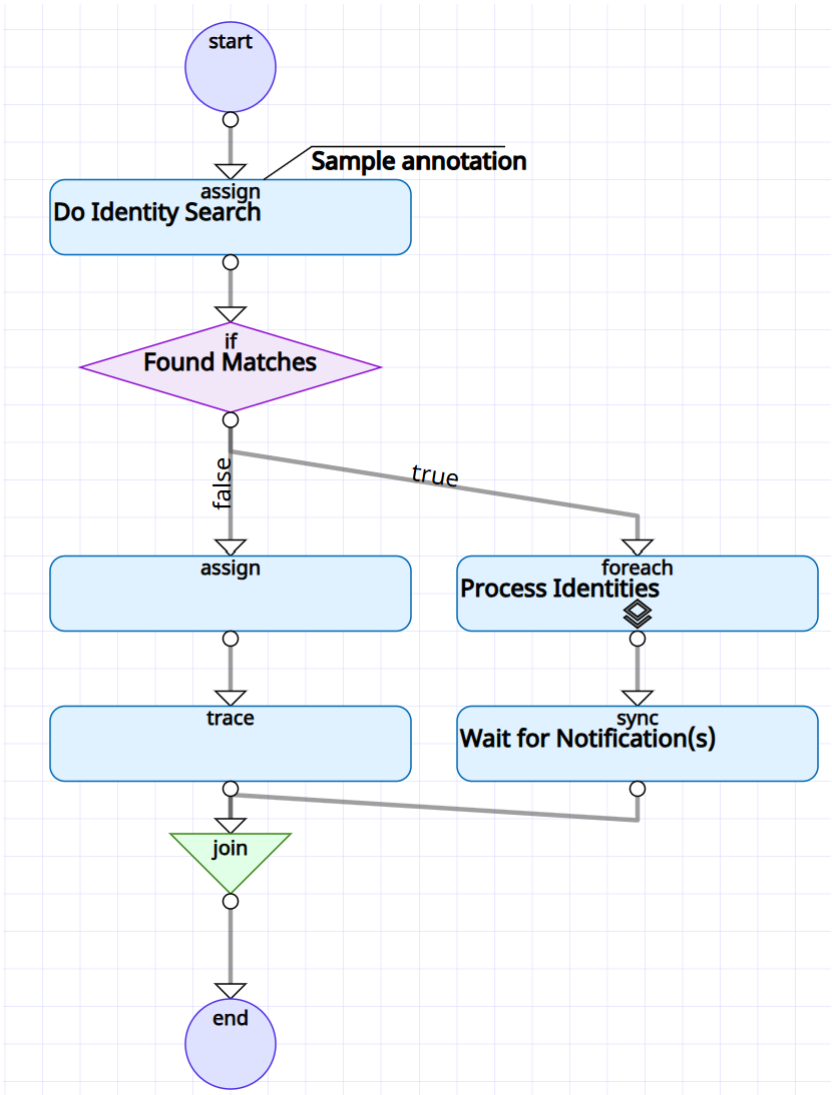
2.2 Areas of the Page

When you display the BPL Editor, it shows the last BPL you opened in this namespace, if any. This page has the following areas:

- The top of the page displays options you can use to create and open BPL processes, compile the currently displayed BPL, add activities to the diagram, undo and redo changes, and so on.
- By default, the left area displays the BPL diagram. You can also display the Tree View.
- The right area displays the Inspector, which contains several sections applicable to the currently displayed BPL.

2.3 BPL Diagram

A BPL diagram is the editable, graphical representation of the logic defined by the business process. The following is an example.



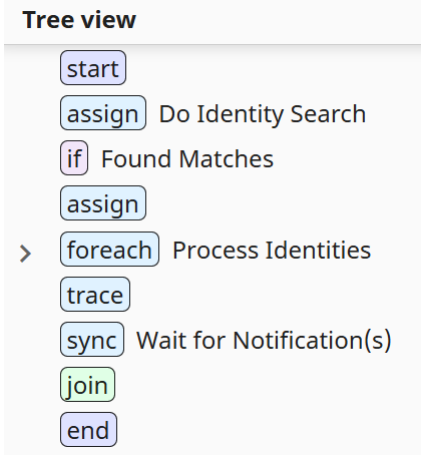
A BPL business process consists of a connected set of activities, shown as different shapes in the diagram. Activities can use values received by the BPL and make decisions based on them, they call other business components, they can manipulate data, and they can call custom code. The process of creating a BPL business process consists primarily of defining the activities it includes and connecting those activities.

Several kinds of BPL activities are displayed as their own subdiagrams. The BPL Editor indicates this visually; for example, the **foreach** element in the previous diagram shows a symbol in the bottom of its shape. To see the subdiagram, you need to drill down. Alternatively, you can use the Tree View, which displays the entire BPL at once.

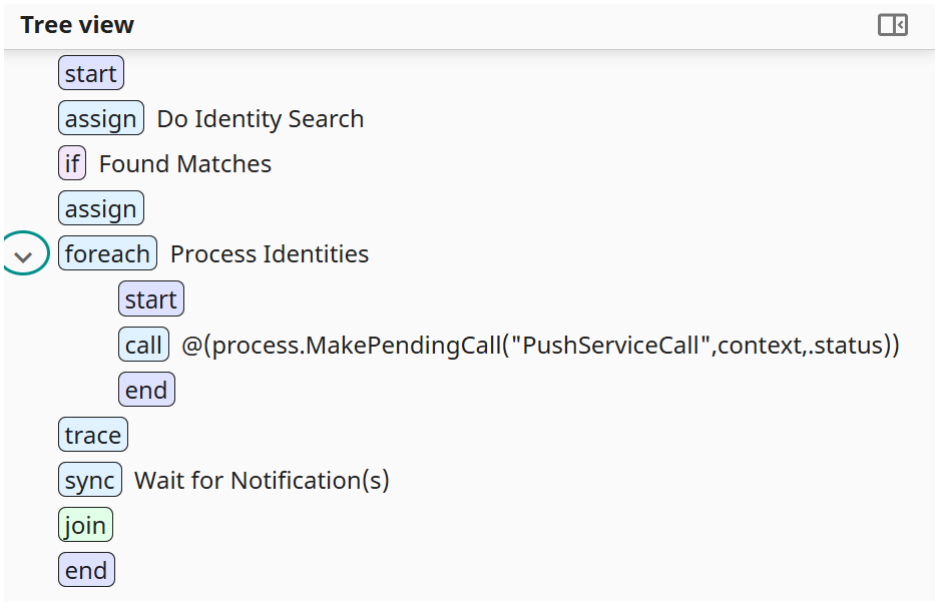
2.4 Tree View

When you display the BPL Editor, normally you see only the BPL diagram.

If you click the Show Tree button, the BPL Editor also displays the diagram as a tree. For example:



You can expand this tree to see all the logic in the BPL. For example, we can expand the **foreach** element, as follows:



To hide the Tree View, click its Hide button.

You can also resize the Tree View by dragging the divider on its right.

2.5 Inspector

By default, the right area of the page displays the Inspector, which consists of the following expandable sections:

- **General**—contains settings for the overall definition of the BPL business process. See Setting General Properties of the BPL Business Process.
- **Context**—enables you to define the context object for this BPL business process.
- **Activity**—contains settings for the selected item in the BPL diagram; see Adding Activities to a BPL Diagram.
- **Preferences**—contains settings pertaining to the appearance of the BPL diagram. See Setting BPL Diagram Preferences.

To hide the Inspector, click its Hide button. To display the Inspector, click the Show button.

You can also resize the Inspector by dragging the divider on its left.

2.6 See Also

- Creating BPL Business Processes
- Editing a BPL Diagram

3

Creating BPL Business Processes

This page describes at a high level how to create and edit BPL business processes for interoperability productions.

For these tasks, you use the BPL Editor.

For information on using the legacy UI for these tasks, see Creating BPL Business Processes(Legacy UI).

3.1 Creating a BPL Business Process

To create a BPL business process, do the following in the BPL Editor:

1. Click **New**.

   This displays a dialog box.

2. Specify some or all of the following information:

   - **Package** (required)—Enter a package name or start typing and select a package in the current namespace.

     Do not use a reserved package name; see Reserved Package Names.

   - **Name** (required)—Enter a name for your BPL business process class.
   - **Description**—Enter a description for your BPL business process class; this becomes the class description.

3. Click **OK**.

The start and end points of the BPL diagram display in the BPL Editor, ready for you to add activities to your BPL business process.

3.2 Opening a BPL Business Process

To open a BPL business process, do the following in the BPL Editor:

1. Click **Open**.

   If you are currently viewing a BPL and you have made changes but have not yet saved them, InterSystems IRIS prompts you to confirm that you want to proceed (which will discard those changes).

2. Click the package that contains the BPL.

Then click the subpackage as needed.

3. Click the BPL class.
4. Click **OK**.

3.3 Setting General Properties of the BPL Business Process

Each BPL business process has a small set of properties that apply to the process as a whole. To set these properties, do the following in the BPL Editor

1. Display the Inspector.
2. Expand the **General** group.
3. Modify the following settings:
   - **Layout**—Select either **Automatic** or **Manual** for the size of the diagram. If you select **Manual** you can enter a **Width** and **Height**.
   - **Annotation**—Enter text to include in the class description.
   - **Language**—Can be **Python** or **ObjectScript**.
   - **Includes**—An optional comma-delimited list of include file names, so that you can use macros in your <code> segments.
   - **Version**—Enter an optional version number of the BPL diagram.
   - **Component**—If this check box is selected, include this process in the component library where it can be called by other processes.

   See <process> for details on these properties.

3.4 Defining the context Object

Each BPL business process has a context object that provides information available for use in the BPL logic. To define this object, do the following in the BPL Editor:

1. Display the Inspector.
2. Expand the **Context** group.
3. Modify the following settings:
   - **Request Class**—Choose the class of the incoming request for this process.
   - **Response Class**—Choose the class of the response returned by this process.
   - **Context Superclass**—Use this option to provide custom context properties, in a different way than adding to the **Context properties** list, described next. To use **Context Superclass**, create a custom subclass of Ens.BP.Context. In this subclass, define class properties to use as context properties. Use the name of this class as the value of

Context Superclass in the business process. Then when you create <assign> actions, for example, you can choose these custom properties in addition to the standard properties of the *context* object.

4. To add a property, click **Add Entry**.

   Then enter values in the following fields:

   - **Name**—Must be a valid identifier.
   - **Collection type**—Choose one of the following: **Single Value**, **List Collection**, or **Array Collection**
   - **Type**—Type of this property including parameters.

     Enter a data type class name in the **Type** field or click the magnifying glass to browse for a class you want to use as a data type.
   - **Default** (ignored for collections)—Enter an initial expression for a single value data type.
   - **Annotation**—Enter an optional annotation.
   - **Instantiate**—Select this check box for object-valued properties if you want the object to be instantiated when it is created.

5. Click **Save** to save your changes or click **Cancel** to discard them.

To set property parameters such as MINVAL, MAXVAL, MINLEN, MAXLEN, or others, add data type parameters to a *context* property when you first add the property, or at any subsequent time, by inserting a comma-separated list of parameters enclosed in parentheses after the data type class name. That is, rather than simply entering %String or %Integer, you can enter data types such as:

```
%String(MAXLEN=256)
%Integer(MINVAL=0,MAXVAL=100)
%String(VALUELIST=",Buy,Sell,Hold")
```

Once you have defined properties of the *context* object, you can refer to them anywhere in BPL using ordinary dot syntax and the property name, as in: `context.MyData`

For reference details, see the following resources:

- Typically you choose the property type from types described in Data Types. These include %String, %Integer, %Boolean, and so on.
- System data types have optional parameters. For details, see Parameters. These include the *MINLEN* and *MAXLEN* parameters that set the minimum and maximum allowed lengths of a %String property. The default maximum %String length is 50 characters; you can reset this by setting the *MAXLEN* for that %String property to another value.

By default, the *ruleContext* passed to the rule is the business process execution context. If you specify a different object as a context, there are some restrictions on this object: It must have a property called %Process of type Ens.BusinessProcess; this is used to pass the business process calling context to the rules engine. You do not need to set the value of this property, but it must be present. Also, the object must match what is expected by the rule itself. No checking is done to ensure this; it is up to the developer to set this up correctly.

3.5 Adding an Activity

Each BPL business process consists of a set of connected activities. In general, to add an activity to a BPL business process, do the following in the BPL Editor:

1. Click **Add** in the ribbon bar and select an option.

This adds a new shape to the BPL diagram.

2. Edit the activity details in the **Activity** tab on the right.
3. Use drag and drop options to modify how this activity is connected to others.

For details, see Adding Activities to BPL and Editing a BPL Diagram.

3.6 Undoing a Change

To undo the previous change in the BPL Editor, click the Undo button.

To redo the previous change in the BPL Editor, click the Redo button.

3.7 Saving a BPL

To save a BPL while in the BPL Editor, do one of the following:

- Click **Save**.
- Click **Save As**. Then specify a new package, class name, and description and click **OK**.
- Click **Compile**. This option saves the BPL and then compiles it.

3.8 Compiling a BPL

To compile a BPL while in the BPL Editor, click **Compile**. This option saves the BPL and then compiles it.

3.9 See Also

- Editing a BPL Diagram
- Listing and Managing Business Processes

4

Editing a BPL Diagram

Each BPL business process consists of a set of connected activities, represented by shapes in the BPL diagram. This page describes how to make changes in a BPL diagram, within the BPL Editor.

For information on using the legacy UI for these tasks, see Editing a BPL Diagram (Legacy UI).

4.1 Basics

- To select an activity, click it. When you do so, its attributes display in the **Activity** tab, where you can edit their values.
- To select multiple activities, hold down the **Ctrl** key while clicking.
- To clear the selection of a selected activity, click it again.
- You can connect activities via drag and drop actions.
- The toolbar provides options for adding activities; cutting, copying, pasting, deleting activities; and undoing changes.
- The BPL Editor automatically validate activities as you add them to the diagram. If it detects an element with a logical error, it displays a red warning on the **Activity** tab for the element along with the reason for the error.
- Several kinds of BPL activities are displayed as their own subdiagrams. To see them in detail, it is necessary to drill down into them.

4.2 Color Indicators

The BPL Editor provides the following color indicators for shapes in the diagram:

- Typically the interior color of a BPL diagram shape is white, with a blue outline. If you select the shape, its interior color changes to yellow and the outline becoes bolder.
- If the shape is in error, its outline is red.
- If the shape is disabled, its interior color is gray, with a gray outline. When you select a disabled shape it shows a dotted outline.

Also, when a shape represents a complex activity such as <if> or <switch> that has multiple branches, joins, or other types of related shapes elsewhere in the BPL diagram, clicking on one of these shapes highlights the related shapes in green with a purple outline.

4.3 Specifying Diagram Preferences

The **Preferences** tab contains the following settings that apply to the appearance of the BPL diagram:

- **Gridlines**—Select one of the following choices for the appearance of the grid lines on the diagram: **None**, **Light**, **Medium**, or **Dark**.
- **Show annotations**—Reveal or hide the text notes that explain each shape. When you reveal annotations, they appear to the upper right of each shape that has an <annotation> element in the BPL document.
- **Auto arrange**—Cause each new shapes in the diagram to automatically conform to a structured arrangement without needing to select the Arrange button after adding each shape.

  Changing the position of shapes does not change the underlying BPL code.

4.4 Adding an Activity

To add an activity to a BPL diagram, do the following:

1. Select an option from the **Add** list.

   This immediately places a new, unconnected activity to the diagram.
2. While the new activity is still selected, specify values as needed in the **Activity** group in the Inspector:
   - **Name**—Enter a name for the caption inside the shape.
   - **Disabled**—Optionally select this check box to disable the activity; clear it to enable. The default is enabled.
   - **Annotation**—Optionally enter text to appear as comments next to the shape in the diagram.

   Other details depend on the type of activity. See the BPL Reference.
3. Connect this activity to other activities as needed.

Or if this activity should be inserted between two existing activities, do the following:

1. Select the connector that connects those two activities.
2. Select an option from the **Add** list.

   This immediately inserts the new activity between the two other activities, connected to both of them.
3. While the new activity is still selected, specify values as needed on the **Activity** tab, as described above.

4.4.1 Adding a Call Activity

A common task in a BPL business process is to add a Call activity. The following information is necessary to properly create a new <call> to one of the available business processes or business operations in the production:

- Input
- Output
- Name
- Target

- Request

4.5 Editing Properties of an Activity

To edit properties of an activity, do the following:

1. Click the activity.
2. Display the Inspector, if it is not currently shown.
3. Specify values as needed in the **Activity** group in the Inspector:
   - **Name**—Enter a name for the caption inside the shape.
   - **Disabled**—Optionally select this check box to disable the activity; clear it to enable. The default is enabled.
   - **Annotation**—Optionally enter text to appear as comments next to the shape in the diagram.

   Other details depend on the type of activity. See the BPL Reference.

4.6 Removing an Activity

To remove an activity from a BPL diagram, do the following:

1. Select the activity.
2. Click the Remove button in the toolbar.

4.7 Adding a Connection

Each activity is displayed with one triangular *input* point and one *output* circle. You use these when connecting activities.

To add a connection from one activity to another, do the following:

1. Click the output circle of one activity.
2. Drag the cursor to input triangle of the other activity and then release.

   Equivalently, you can click the input triangle of one activity and drag to the output circle of the other activity.
3. Optionally select the connector, and enter a name for it on the **Activity** tab.

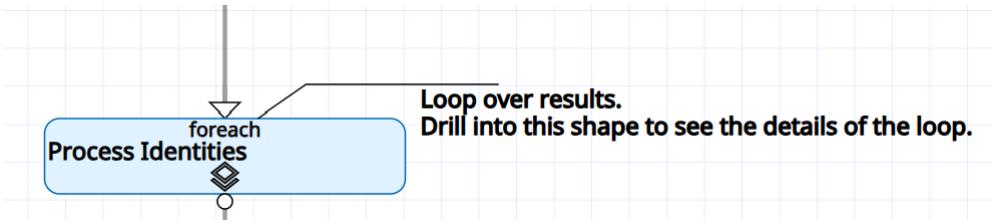
The tool does not allow you to make an illegal connection.

Once two shapes are connected, the connection is preserved no matter where you drag the respective shapes. You can drag shapes to any layout position you wish, within the same diagram.

4.8 Drilling Down and Back

Several kinds of BPL activities are displayed as their own subdiagrams. To see them in detail, it is necessary to drill down into them. Examples include <foreach> and <sequence>. The main BPL diagram shows only a stub, with an indicator that you need to click for more details.

For example, the following shows a <foreach> loop, as you would see it within the main BPL diagram:



The symbol at the bottom of the shape is a reminder that there are details not shown here.

To drill down, click the symbol.

The BPL Editor then displays the full details for that activity, from start to end.

To return to the higher logical level, lick the Drill Up button in the toolbar.

4.9 Adjusting the Layout

After you add shapes or create new connections, you can tidy the diagram by clicking the Arrange button on the tool bar.

If you want your diagrams to always use this type of structured layout, select the **Auto arrange** check box in the **Preferences** group of the Inspector.

By default, when you open a BPL diagram for the first time, the auto arrange feature is enabled. This choice may or may not be appropriate for a particular drawing. You can disable automatic arrangement to ensure that your diagram always displays with exactly the layout you want by clearing the **Auto arrange** check box in the **Preferences** group in the Inspector. This way, when the diagram is displayed, it does not take on any layout characteristics except what you have specified.

4.10 See Also

- Creating BPL Business Processes
- Listing and Managing the Business Processes

5

Available Variables in BPL (Execution Context)

A BPL business process can refer to the *execution context* variables, introduced here. These are available even if the BPL business process is suspended and later resumed. Some of these variables are available to every activity within a BPL business process. Others are generally available, but go in and out of scope, depending on the type of activity that the business process is executing at the time.

Important: These variables are available to only to *BPL* business processes; that is, to business process classes that inherit from Ens.BusinessProcessBPL. They are not available to custom business processes, which must handle similar issues using custom code.

5.1 The context Object

Any activity within a BPL business process can refer to the *context* object.

This variable is a general-purpose container for any data that needs to be persisted during the life cycle of the business process. You define each data item as a property of the *context* object when creating the BPL business process. See Defining the context Object.

To refer to a property of the *context* object, use dot syntax and the property name, as in: `context.MyData`

5.2 The request Object

Any activity within a BPL business process can refer to the *request* object.

This variable contains the properties that were in the original request message object—the incoming message that first caused this business process to be instantiated. This is known as the *primary request*.

To refer to a property of the *request* object, use dot syntax and the property name, as in: `request.OriginalThought`

5.3 The response Object

Any activity within a BPL business process can refer to (or modify) the *response* object.

This variable contains the properties that are required to build the final response message object to be returned by this business process instance. The business process returns this final response either when it reaches the end of its life cycle, or when it encounters a <reply> activity.

To refer to a property of the *response* object, use dot syntax and the property name, as in: `response.BottomLine`

5.4 The callrequest Object

The *callrequest* object contains any properties that are required to build the request message object to be sent by a <call>.

A <call> activity sends a request message and, optionally, receives a response. A BPL <call> element must include a <request> activity to put values into the properties on the request message object. In order to accomplish this, the <request> provides a sequence of <assign> activities that place values into properties on the *callrequest* object. Typically, some of these values are derived from properties on the original *request* object, but you are free to assign any value.

As soon as the <assign> activities inside the <request> are completed, the message is sent, and the associated *callrequest* object goes out of scope. *callrequest* has no meaning outside its associated <request> activity; it is already out of scope when the associated <call> begins processing its next activity, the optional <response>.

Within the scope of the relevant <request> element, you can refer to the properties of *callrequest* using dot syntax, as in: `callrequest.UserData`

5.5 The callresponse Object

Upon completion of a <call> activity, the *callresponse* object contains the properties of the response message object that was returned to the <call>. If the <call> was designed with no response, there is no *callresponse*. Similarly, if you use <sync> to wait for a response, but the response does not return within the timeout period specified by the <sync> element, there is no *callresponse*.

Every <call> that expects a response must provide a <response> activity within the <call>. The purpose of the <response> activity is to retrieve the response values and make them available to the business process as a whole. The *callresponse* object is available anywhere inside the <response> activity. However, as soon as the <response> activity completes, the associated *callresponse* object goes out of scope. Therefore, if you want to use the values in *callresponse* elsewhere in the business process, you must <assign> these values to properties on the *context* or *response* objects, and you must do so before the end of the <response> activity in which they were received.

To refer to a property of *callresponse*, use dot syntax, as in: `callresponse.UserAnswer`

5.6 The syncresponses Collection

The *syncresponses* variable is a collection, keyed by the names of the <call> activities being synchronized by a <sync>.

When a <sync> activity begins, *syncresponses* is cleared in preparation for new responses. As the <call> activities return, responses go into the collection. When the <sync> activity completes, *syncresponses* may contain all, some, or none of the desired responses (see *synctimedout*). *syncresponses* is available anywhere inside the <sequence> that contains the relevant <call> and <sync> activities, but goes out of scope outside that <sequence>.

To refer to the response value from one of the synchronized calls, use the syntax: `syncresponses.GetAt("name")`

Where the relevant <call> was defined as: `<call name="name">`

5.7 The synctimedout Value

The *synctimedout* variable is an integer value that may be 0, 1, or 2. The value indicates the outcome of a <sync> activity after several calls. You can test the value of *synctimedout* after the <sync> and before the end of the <sequence> that contains the calls and <sync>. *synctimedout* has one of three values:

- If 0, no call timed out. All the calls had time to complete. This is also the value if the <sync> activity had no *timeout* set.
- If 1, at least one call timed out. This means not all <call> activities completed before the timeout.
- If 2, at least one call was interrupted before it could complete.

The *synctimedout* variable is available to a BPL business process anywhere inside the <sequence> that contains the relevant <call> and <sync> activities, but goes out of scope outside that <sequence>. Generally you will test *synctimedout* for status and then retrieve the responses from completed calls out of the *syncresponses* collection. You can refer to *synctimedout* with the same syntax as for any integer variable name, that is: `synctimedout`

5.8 The status Value

The *status* variable is a value of type %Status that indicates success or failure.

Note: Error handling for a BPL business process happens automatically without your ever needing to test or set the *status* value in the BPL source code. The *status* value is documented here in case you need to trigger a BPL business process to exit under certain special conditions.

When a BPL business process starts up, *status* is automatically assigned a value indicating success. To test that *status* has a success value, you can use the macro `$$$ISOK(status)` in ObjectScript. If the test returns a True value, *status* has a success value.

As the BPL business process runs, if at any time *status* acquires a failure value, InterSystems IRIS immediately terminates the business process and writes the corresponding text message to the Event Log. This happens regardless of how *status* acquired the failure value. Thus, the best way to cause a BPL business process to exit immediately, but gracefully is to set *status* to a failure value.

status can acquire a failure value in any of the following ways:

- *status* automatically receives the returned %Status value from any <call> that the business process makes to another business host. If the value of this %Status indicates failure, *status* automatically receives the failure value. This is the most common way in which *status* is set, and it happens automatically, without any special statements in the BPL code.

- An <assign> activity can set *status* to a failure value. The usual convention for doing this is to use an <if> element to test the result of some prior activity, and then within the <true> or <false> element use <assign> to set *status* to a failure value when failure conditions exist.
- Statements within a <code> activity can set *status* to a failure value. The BPL business process does not perceive the change in the value of *status* until the <code> activity has fully completed. Therefore, if you want a failure *status* to cause an immediate exit from a <code> activity, you must place a quit command in the <code> activity immediately after setting a failure value for *status*.

To test that *status* has a failure value, use the macro `$$$ISERR(status)` in ObjectScript. If the test returns a True value, *status* has a failure value. You will be able to perform this test only within the body of a <code> activity before it returns to the main BPL business process, since the business process will automatically quit with an error as soon as it detects that *status* has acquired a failure value following any <call>, <assign>, or <code> activity.

status is available to a BPL business process anywhere inside the <process>. You can refer to *status* with the same syntax as for any variable of the %Status type, that is: `status`

CAUTION: Like all other execution context variable names, *status* is a reserved word in BPL. Do not use it except as described in this topic.

5.9 The process Object

Any activity within a BPL business process can refer to the *process* object.

This variable represents the current instance of the BPL business process object. The *process* object is provided so that you can invoke any business process method, such as **SendRequestSync()** or **SendRequestAsync()**, from any context within the flow of the BPL business process, for example from within the text block of a <code> activity.

The *process* object is typically needed only within the <code> activity. To refer to a method of the *process* object, use dot syntax and the method name, as in: `process.SendRequestSync()` or `process.ClearAllPendingResponses()`

6

Available BPL Elements

This page introduces the elements you can use in a BPL business process. The elements are grouped here by category.

6.1 Control Flow

BPL includes a number of control flow elements that you can use to control the order of execution. First, here are the elements involved in branching:

- <branch> conditionally causes an immediate change in the flow of execution.
- <if> evaluates a condition and perform ones action if true, another if false.
- <label> provides a destination for a conditional branch operation.
- <switch> evaluates a set of conditions to determine which of several actions to perform.

Additional elements are used for looping:

- <break> breaks out of a loop and exits the loop activity.
- <continue> jumps to the next iteration within a loop, without exiting the loop.
- <foreach> defines a sequence of activities to be executed iteratively.
- <until> defines a sequence of activities to be repeatedly executed *until* a condition is true.
- <while> defines a sequence of activities to be repeatedly executed *as long as* a condition is true.

Two elements can be used for grouping activities:

- <flow> performs activities in a non-determinate order.
- <sequence> organizes one or more calls to other business operations and business processes. Structures parts of the BPL diagram.

Note: In addition to these options you can initiate a immediate, but graceful exit as follows: set the *status* variable to a failure value using an <assign> or <code> statement.

6.2 Messaging

BPL includes elements that make synchronous and asynchronous requests to business operations, and to other business processes.

- <call> sends a request and (optionally) receives a response from a business operation or business process. The call may be synchronous or asynchronous.
- <request> prepares the request for a call to another business operation or business process.
- <response> receives the response returned from a call to another business operation or business process.
- <sync> waits for a response from one or more asynchronous calls to other business operations and business processes.
- <reply> returns a primary response from the business process before execution of the process is fully complete.

6.3 Scheduling

The <delay> element can be used to delay execution of a business process for a specified duration or until a future time.

6.4 Rules and Decisions

The <rule> element executes a business rule. This element specifies the business rule name, plus parameters to hold the result of the decision and (optionally) the reason for that result.

The parameters for the <rule> element can include any property in the *context* variable. Therefore, for a business process that invokes a rule, the typical design is to ensure that the business process accomplishes the following:

1. Provides <property> and <context> elements so that the *context* object contains properties with appropriate names and types.

   For example, if the rule determines eligibility for a state education loan, you might add properties such as Age, State, and Income.

2. Gathers values for the properties in whatever way you wish, for example by sending requests to business operations or business processes, and as responses return, assigning values to the properties in *context*.
3. Provides a <rule> element that invokes a business rule that returns an answer based on these input values.

For details on *context*, see Available Variables in BPL.

For information on creating business rules, see Developing Business Rules.

6.5 Data Manipulation

BPL includes several elements that allow you to move data from one location to another. For example, a typical business process makes a series of calls to business operations or other business processes. To set up these calls, as well as to process the data they return, the business process shuffles data between the various BPL variables—*context*, *request*, *response*, and others. This shuffling and other data manipulation tasks are accomplished using the elements described below.

- <assign> assigns a value to a property.
- <sql> executes an embedded SQL SELECT statement.
- <transform> transforms one object into another using a data transformation.
- <xpath> evaluates an XPath expression on a target XML document.
- <xslt> executes an XSLT transformation to modify a data stream.

6.6 User-written Code

For cases where BPL is not expressive enough, InterSystems IRIS® provides mechanisms to embed user-written code within the automatically generated business process code.

- <code> allows you to specify the required code within a CDATA block.
- <empty> performs no action; acts as a placeholder until code can be written.

6.7 Logging

BPL includes elements you can use to log informational and error messages.

- <alert> writes a text message to an external alert mechanism.
- <milestone> stores a message to acknowledge a step achieved by a business process.
- <trace> write a text message to a console window and to the Event Log.

6.8 Error Handling

BPL includes elements that you can use to throw and catch faults, and perform compensation for errors or faults. These elements are closely interrelated. For details, see Handling Errors in BPL. The list of elements is as follows:

- <catch> catches a fault produced by a <throw> element.
- <catchall> catches a fault or system error that does not match any <catch>.
- <compensate> invoke a <compensationhandler> from <catch> or <catchall>.
- <compensationhandler> performs a sequence of activities to undo a previous action.
- <compensationhandlers> contains one or more <compensationhandler> elements.
- <faulthandlers> provides zero or more <catch> and one <catchall> element.
- <scope> wraps a set of activities with its fault and compensation handlers.
- <throw> throws a specific, named fault.

7

BPL Syntax Rules

This topic describes the syntax rules for referring to properties and for creating expressions within various activities in a business process in an interoperability production.

7.1 References to Message Properties

In activities within a BPL process, it may be necessary to refer to properties of the message. The rules for referring to a property are different depending on the kind of messages you are working with.

- For messages other than virtual documents, use syntax like the following:

  ```
  message.propertyname
  ```

  Or:

  ```
  message.propertyname.subpropertyname
  ```

  Where *propertyname* is a property in the message, and *subpropertyname* is a property of that property.
- For virtual documents other than XML virtual documents, use the syntax described in Syntax Guide for Virtual Property Paths.
- For XML virtual documents, see Routing XML Virtual Documents in Productions.

7.2 Literal Values

When you assign a value to a property, you often specify a literal value. Literal values are also sometimes suitable in other places, such as the value in a **trace** action.

A literal value is either of the following:

- A numeric literal is just a number. For example: 42.3
- A string literal is a set of characters *enclosed by double quotes*. For example: `"ABD"`

  Note: This string cannot include XML reserved characters. For details, see XML Reserved Characters.

  For virtual documents, this string cannot include separator characters used by that virtual document format. See Separator Characters in Virtual Documents and When XML Reserved Characters Are Also Separators.

7.2.1 XML Reserved Characters

Because BPL processes are saved as XML documents, you must use XML entities in the place of XML reserved characters:

To include this character...	Use this XML entity...
>	>
<	<
&	&
'	'
"	"

For example, to assign the value `Joe's "Good Time" Bar & Grill` to a property, set **Value** equal to the following:

```
"Joe&apos;s &quot;Good Time&quot; Bar &amp; Grill"
```

This restriction does not apply inside <code> and <sql> activities, because InterSystems IRIS® automatically wraps a CData block around the text that you enter into the editor. (In the XML standard, a CData block encloses text that should not be parsed as XML. Thus you can include reserved characters in that block.)

7.2.2 Separator Characters in Virtual Documents

In most of the virtual document formats, specific characters are used as separators between segments, between fields, between subfields, and so on. If you need to include any of these characters as literal text when you are setting a value in the message, you must instead use the applicable escape sequence, if any, for that document format.

For information on these separators, see:

- EDIFACT Separators
- X12 Separators

7.2.3 When XML Reserved Characters Are Also Separators

- If the character (for example, &) is a separator and you want to include it as a literal character, use the escape sequence that applies to the virtual document format.
- In all other cases, use the XML entity as shown previously in XML Reserved Characters.

7.2.4 Numeric Character Codes

You can include decimal or hexadecimal representations of characters within literal strings.

The string `&#n;` represents a Unicode character when *n* is a decimal Unicode character number. One example is `&#233;` for the Latin e character with acute accent mark (é).

Alternatively, the string `&#xh;` represents a Unicode character when *h* is a hexadecimal Unicode character number. One example is ¿ for the inverted question mark (¿).

7.3 Valid Expressions

When you assign a value to a property, you can specify an expression, in the language that you selected for the BPL process. You also use expressions in other places, such as the condition for an <if> activity, the value in a <trace> activity, statements in a <code> activity, and so on.

The following are all valid expressions:

- Literal values, as described in the previous section.
- Function calls (InterSystems IRIS provides a set of utility functions for use in business rules and data transformations. For details, see Utility Functions for Use in Productions.)
- References to properties, as described in References to Properties.
- Any expression that combines these, using the syntax of the scripting language you chose for BPL process. Note the following:
  - In ObjectScript, the concatenation operator is the _ (underscore) character, as in:

    ```
    value='"prefix"_source.{MSH:ReceivingApplication}_"suffix"'
    ```
  - To learn about useful ObjectScript string functions, such as $CHAR and $PIECE, see ObjectScript Reference.
  - For a general introduction, see Using ObjectScript.

7.4 Indirection

InterSystems IRIS supports indirection in values for the following BPL element-and-attribute combinations only:

- <call name=
- <call target=
- <sync calls=
- <transform class=

The *at sign* symbol, @, is the indirection operator.

For example, the <call> element supports indirection in the values of the *name* or *target* attributes. The *name* identifies the call and may be referenced in a later <sync> element. The *target* is the configured name of the business operation or business process to which the request is being sent. Either of these strings can be a literal value:

```
<call name="Call" target="MyApp.MyOperation" async="1">
```

Or the @ indirection operator can be used to access the value of a context variable that contains the appropriate string:

```
<call name="@context.nextCallName" target="@context.nextBusinessHost" async="1">
```

For information on @ indirection syntax, see <call>, <sync>, and <transform>.

Important: BPL and DTL are similar in many ways, but DTL does *not* support indirection.

8

Handling Errors in BPL

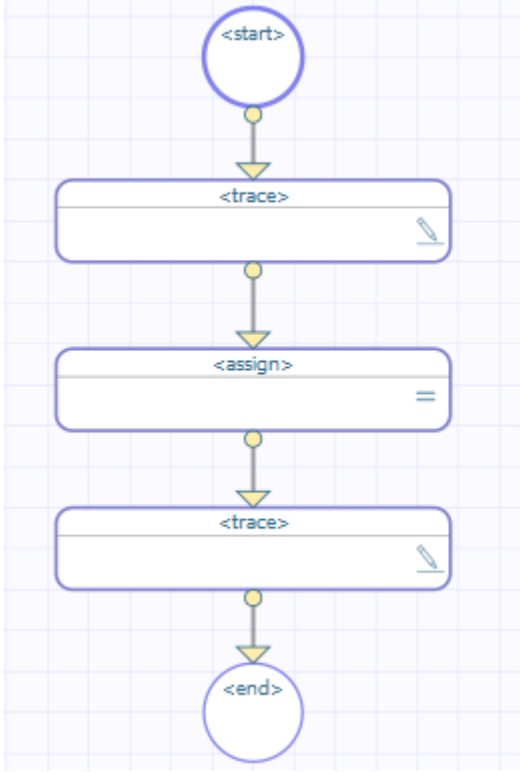
This topic explains how BPL business processes support error handling. BPL provides fault handlers that allow your business process to throw and catch errors, and compensation handlers that allow your business process to specify how it recovers from errors by undoing the actions that led to the error condition.

Important: When you use this error handling system with <call> statements that communicate with other business hosts, make sure that the target business hosts return an error status in the case of an error. If the target component returns success even in the case of an error, the BPL process will not trigger <catchall> logic.

The BPL elements involved in error handling are <scope>, <throw>, <catch>, <catchall>, <compensate>, <compensation-handlers>, <compensationhandler>, and <faulthandlers>. This topic introduces these elements and explains how they work together to support the different error handling scenarios.

8.1 System Error with No Fault Handling

The following is an example of a BPL business process that produces an error condition and provides no error handling:



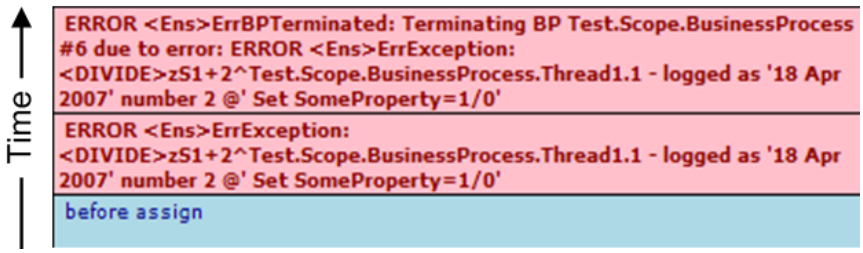
This BPL business process does the following:

1. The first <trace> element generates the message `before assign`.
2. The <assign> element tries to set SomeProperty equal to the expression 1/0. This attempt produces a divide-by-zero system error.
3. The business process ends and sends a message to the Event Log.

   The second <trace> element is never used.

8.1.1 Event Log Entries

The corresponding Event Log entries look like this.



For background information, see Event Log.

8.1.2 XData for This BPL

This BPL is defined by the following XData block:

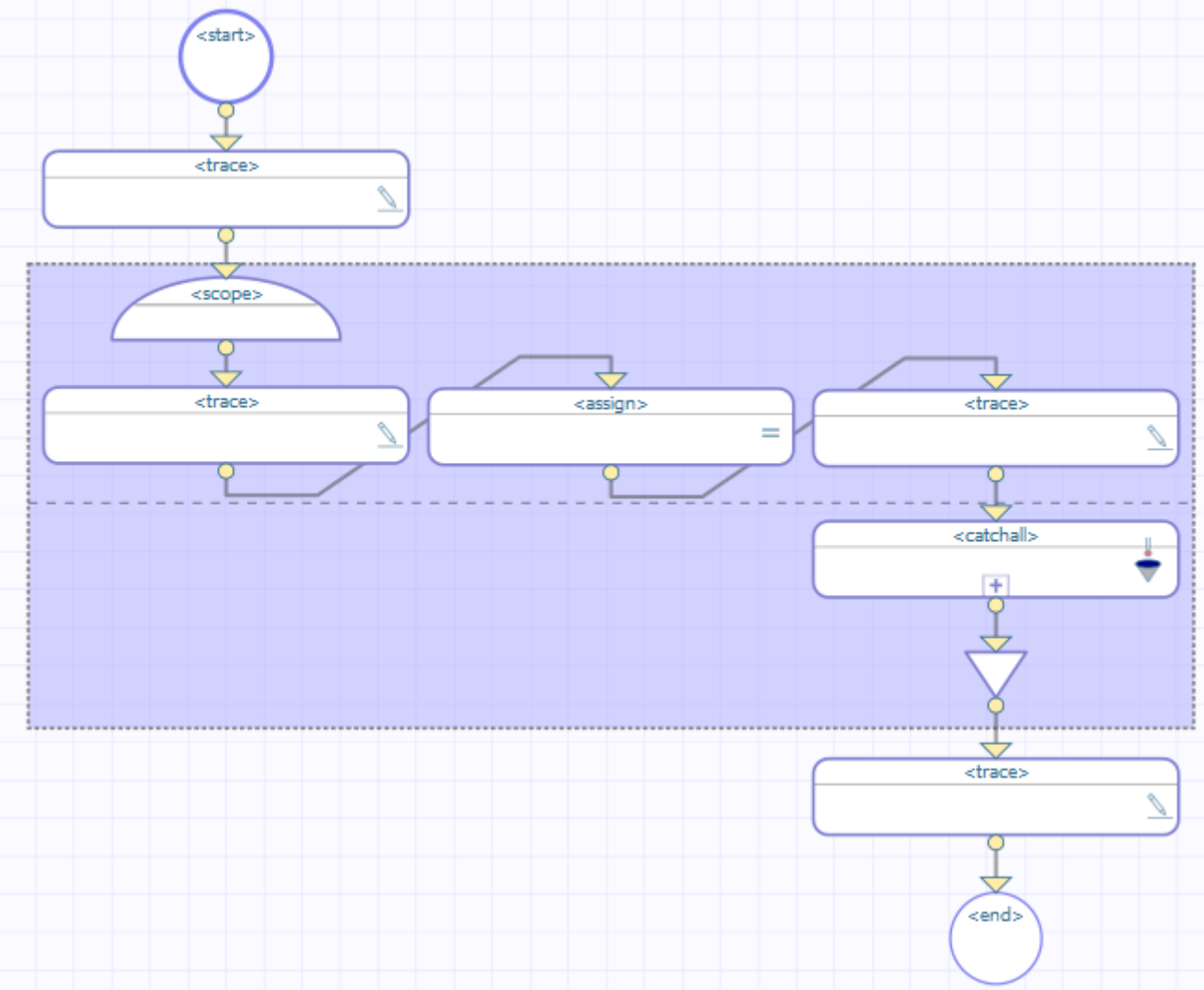
Class Member

```
XData BPL
{
<process language='objectscript'
         request='Test.Scope.Request'
         response='Test.Scope.Response' >
  <sequence>
    <trace value='"before assign"'/>
    <assign property="SomeProperty" value="1/0"/>
    <trace value='"after assign"'/>
  </sequence>
</process>
}
```

8.2 System Error with Catchall

To enable error handling, BPL defines an element called <scope>. A scope is a wrapper for a set of activities. This scope may contain one or more activities, one or more fault handlers, and zero or more compensation handlers. The fault handlers are intended to catch any errors that activities within the <scope> produce. The fault handlers may invoke compensation handlers to compensate for those errors.

The following example provides a <scope> with a <faulthandlers> block that includes a <catchall>. Because the <scope> includes a <faulthandlers> element, the rectangle includes a horizontal dashed line across the middle; the area below this line displays the contents of the <faulthandlers>.

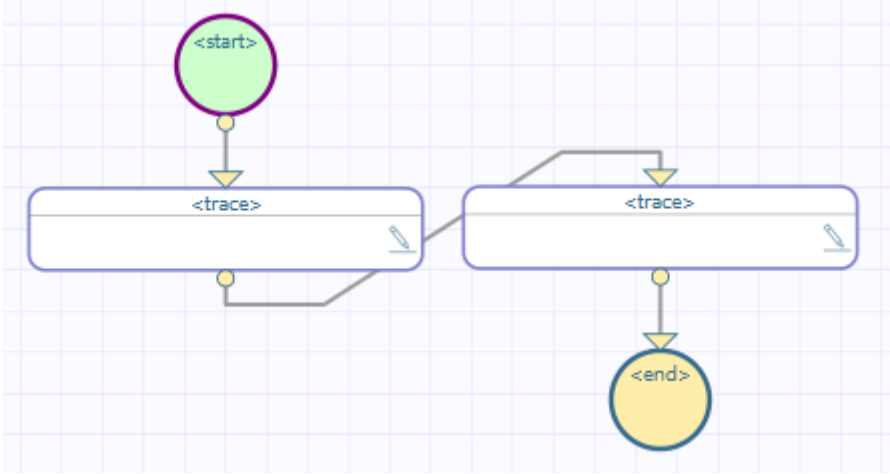


This BPL business process does the following:

1. The first <trace> element generates the message `before scope`.
2. The <scope> element starts the scope.
3. The second <trace> element generates the message `before assign`.
4. The <assign> element tries to evaluate the expression 1/0. This attempt produces a divide-by-zero system error.
5. Control now goes to the <faulthandlers> defined within the <scope>. The <scope> rectangle includes a horizontal dashed line across the middle; the area below this dashed line displays the contents of the <faulthandlers> element. In this case, there is no <catch>, but there is a <catchall> element, so control goes there.

   Note that InterSystems IRIS® skips the <trace> element message immediately after the <assign> element.

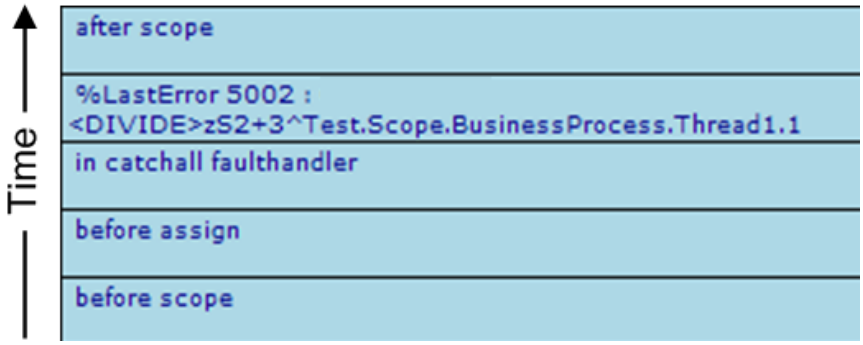
   If we drill down into <catchall>, we see this:



6. Within <catchall>, a <trace> element generates the message `in catchall faulthandler`.
7. Within <catchall>, another <trace> element generates a message that explores the nature of the error using $System.Status methods and the special variables %Context and %LastError. See the details in Event Log Entries.
8. The <scope> ends.
9. The last <trace> element generates the message `after scope`.

8.2.1 Event Log Entries

The corresponding Event Log entries look like this.



If an unexpected system error occurs, and a <faulthandlers> block is present inside a <scope>, the BPL business process does *not* automatically place entries in the Event Log as shown in the System Error with No Fault Handling example. Rather, the <faulthandlers> block determines what the business process will do. In the current example, it outputs a <trace> message that contains information about the error. The Event Log entry showing the actual error is produced by the following statement within the <catchall> block:

XML

```
<trace value=
  '"%LastError "_
  $System.Status.GetErrorCodes(..%Context.%LastError)_
  " : "_
  $System.Status.GetOneStatusText(..%Context.%LastError)'
  />
```

The BPL context variable %LastError always contains a %Status value. If the error was an unexpected system error such as <UNDEF> this %Status value is created from the error “ObjectScript error” which has code 5002, and the text of the

$ZERROR special variable. To get the corresponding error code and text out of %LastError, use the $System.Status methods **GetErrorCodes** and **GetOneStatusText**, then concatenate them into a <trace> string, as shown above.

8.2.2 XData for This BPL

This BPL is defined by the following XData block:

Class Member

```
XData BPL
{
<process language='objectscript'
         request='Test.Scope.Request'
         response='Test.Scope.Response' >
  <sequence>
    <trace value='"before scope"'/>
    <scope>
      <trace value='"before assign"'/>
      <assign property="SomeProperty" value="1/0"/>
      <trace value='"after assign"'/>
      <faulthandlers>
        <catchall>
          <trace value='"in catchall faulthandler"'/>
          <trace value=
            '"%LastError "_
            $System.Status.GetErrorCodes(..%Context.%LastError)_
            " : "_
            $System.Status.GetOneStatusText(..%Context.%LastError)'
            />
        </catchall>
      </faulthandlers>
    </scope>
    <trace value='"after scope"'/>
  </sequence>
</process>
}
```

8.3 Thrown Fault with Catchall

When a <throw> statement executes, its *fault* value is an expression that evaluates to a string. Faults are not objects, as in other object-oriented languages such as Java; they are string values. When you specify a fault string it *needs* the extra set of quotes to contain it, as shown below:

XML

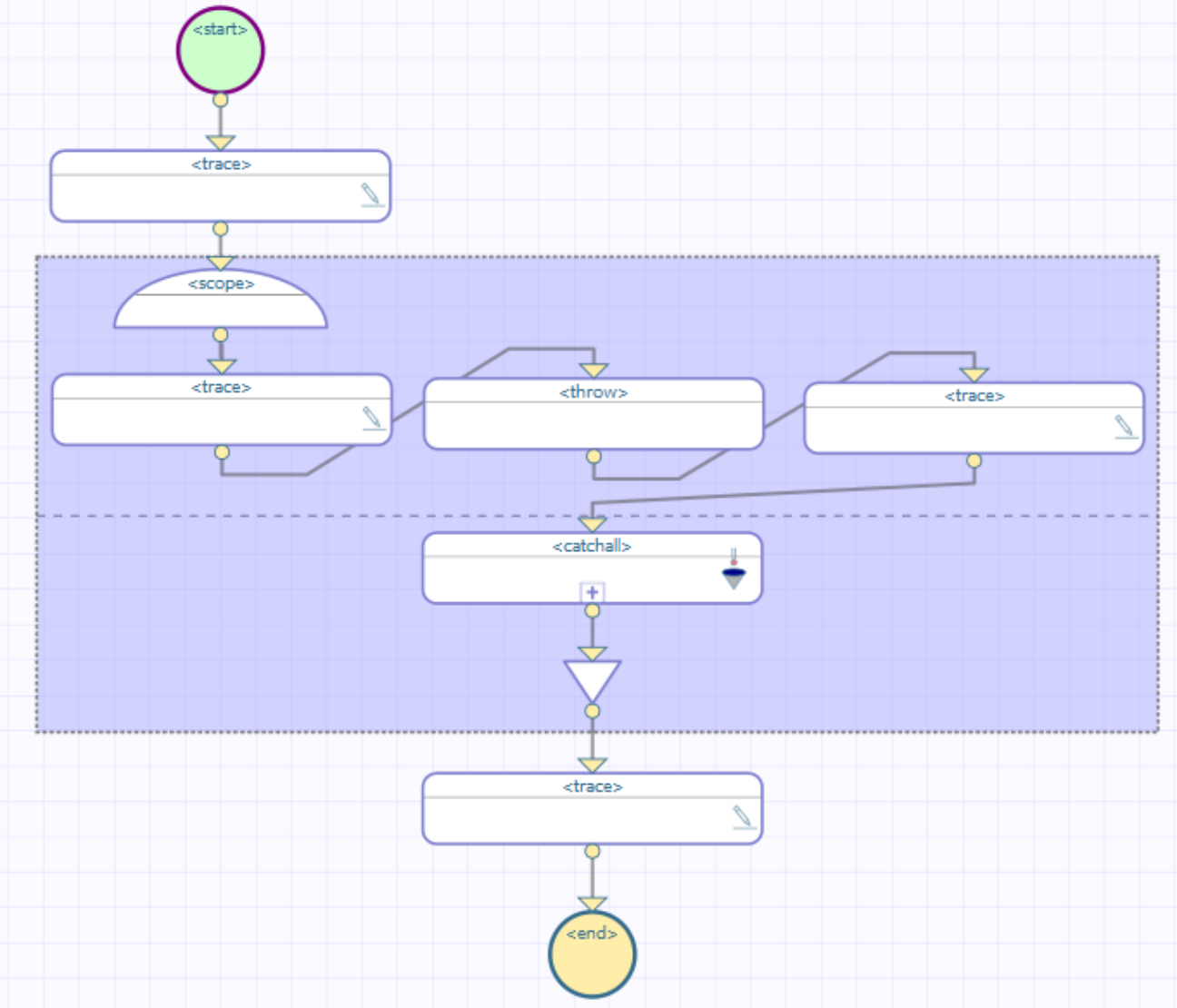
```
<throw fault='"thrown"'/>
```

When a <throw> statement executes, control immediately goes to the <faulthandlers> block inside the same <scope>, skipping all intervening statements after the <throw>. Inside the <faulthandlers> block, the program attempts to find a <catch> block whose *value* attribute matches the *fault* string expression in the <throw> statement. This comparison is case-sensitive.

If there is a <catch> block that matches the fault, the program executes the code within this <catch> block and then exits the <scope>. The program resumes execution at the next statement following the closing </scope> element.

If a fault is thrown, and the corresponding <faulthandlers> block contains *no* <catch> block that matches the fault string, control goes from the <throw> statement to the <catchall> block inside <faulthandlers>. After executing the contents of the <catchall> block, the program exits the <scope>. The program resumes execution at the next statement following the closing </scope> element. It is good programming practice to ensure that there is always a <catchall> block inside every <faulthandlers> block, to ensure that the program catches any unanticipated errors.

Suppose you have the following BPL. For reasons of space, the <start> and <end> elements are not shown.

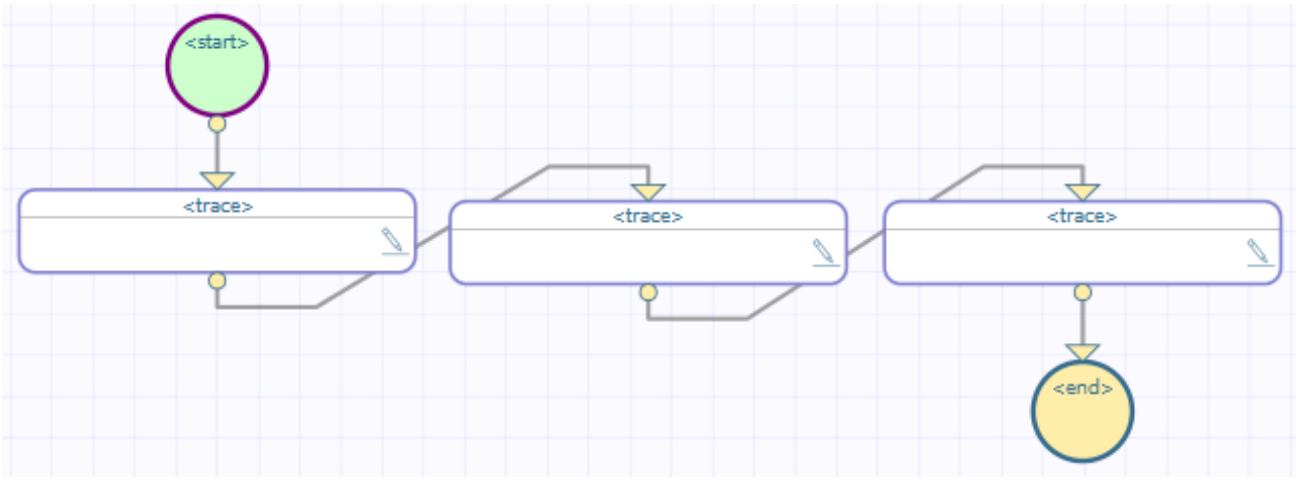


This BPL business process does the following:

1. The first <trace> element generates the message `before scope`.
2. The <scope> element starts the scope.
3. The second <trace> element generates the message `before assign`.
4. The <throw> element throws a specific, named fault (`"MyFault"`).
5. Control now goes to the <faulthandlers> defined within the <scope>. The <scope> rectangle includes a horizontal dashed line across the middle; the area below this dashed line displays the contents of the <faulthandlers> element. In this case, there is no <catch> but there is a <catchall> element, so control goes there.

   Note that InterSystems IRIS skips the third <trace> element.

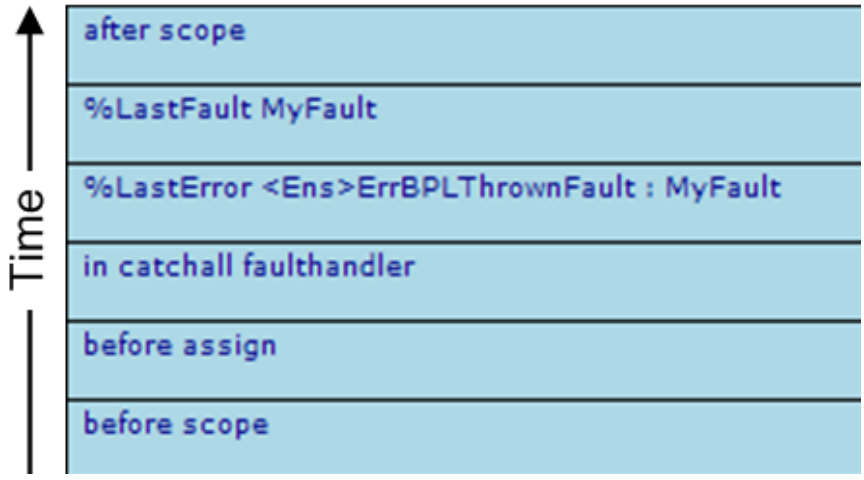
   If we drill down into <catchall>, we see this:



6. Within <catchall>, the first <trace> element generates the message `in catchall faulthandler`.
7. Within <catchall>, the second <trace> element generates the message that provides information on the fault using $System.Status methods and the special variables %Context and %LastError. The %LastError value as the result of a thrown fault is different from its value as the result of a system error:
   - **GetErrorCodes** returns `<Ens>ErrBPLThrownFault`
   - **GetOneStatusText** returns text derived from the *fault* expression in the <throw> statement
8. Within <catchall>, the third <trace> element generates a message that provides information on the fault using the BPL context variable %LastFault. It contains the text derived from the *fault* expression from the <throw> statement.
9. The <scope> ends.
10. The last <trace> element generates the message `after scope`.

8.3.1 Event Log Entries

The corresponding Event Log entries look like this:



8.3.2 XData for This BPL

This BPL is defined by the following XData block:

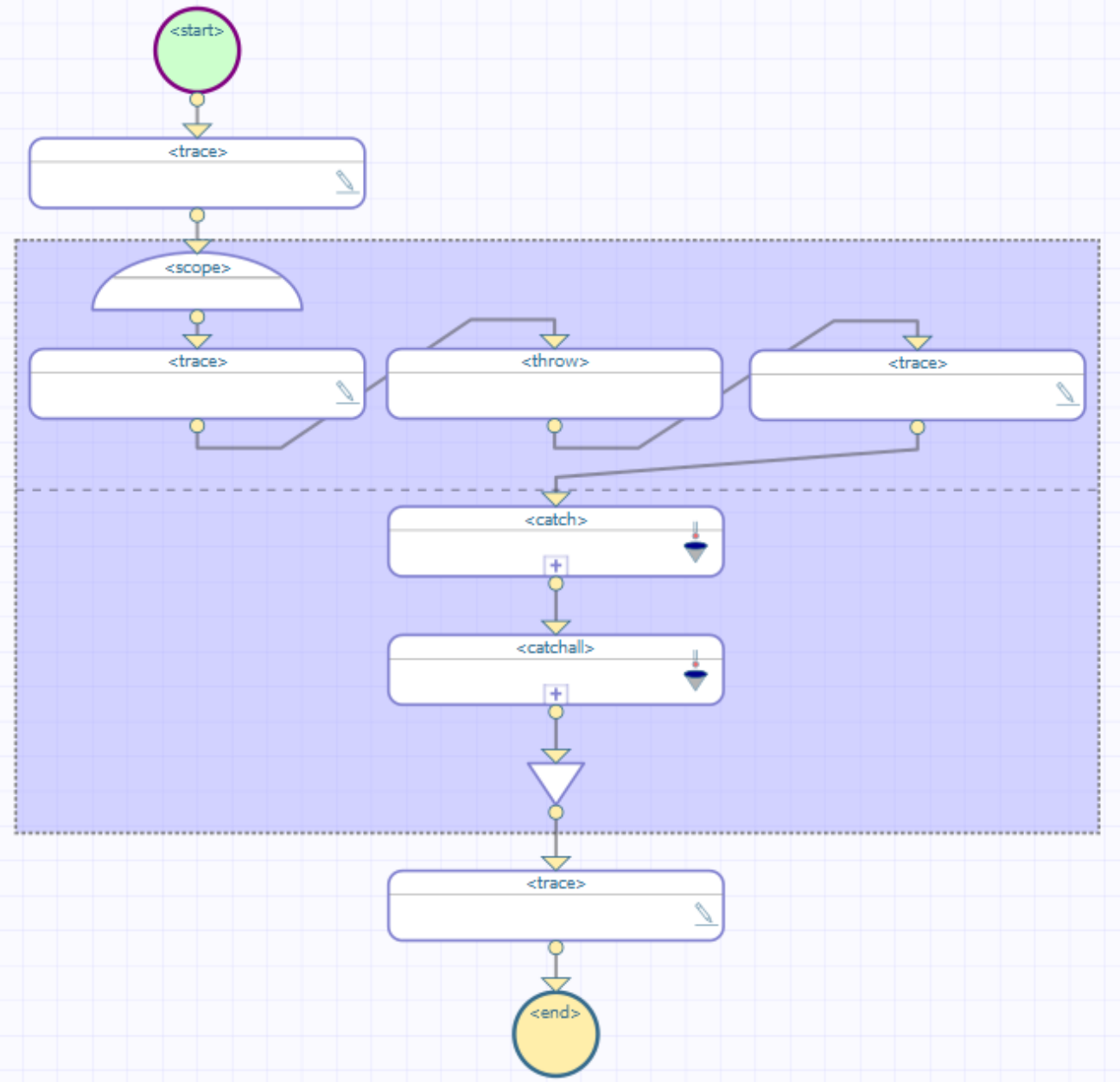
Class Member

```
XData BPL
{
<process language='objectscript'
         request='Test.Scope.Request'
         response='Test.Scope.Response' >
  <sequence>
    <trace value='"before scope"'/>
    <scope>
      <trace value='"before assign"'/>
      <throw fault='"MyFault"'/>
      <trace value='"after assign"'/>
      <faulthandlers>
        <catchall>
          <trace value='"in catchall faulthandler"'/>
          <trace value=
            '"%LastError "_
            $System.Status.GetErrorCodes(..%Context.%LastError)_
            " : "_
            $System.Status.GetOneStatusText(..%Context.%LastError)'
            />
          <trace value='"%LastFault "_..%Context.%LastFault'/>
        </catchall>
      </faulthandlers>
    </scope>
    <trace value='"after scope"'/>
  </sequence>
</process>
}
```

8.4 Thrown Fault with Catch

A thrown fault may reach a <catchall>, as in the previous example, or it may have a specific <catch>.

Suppose you have the following BPL:

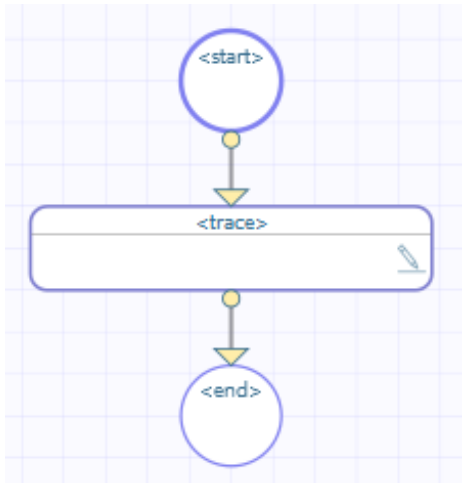


This BPL business process does the following:

1. The first <trace> element generates the message `before scope`.
2. The <scope> element starts the scope.
3. The second <trace> element generates the message `before throw`.
4. The <throw> element throws a specific, named fault (`"MyFault"`).
5. Control now goes to the <faulthandlers> defined within the <scope>. The <scope> rectangle includes a horizontal dashed line across the middle; the area below this dashed line displays the contents of the <faulthandlers> element. In this case, a <catch> element exists whose *fault* value is `"MyFault"`, so control goes there. The <catchall> element is ignored.

   Note that InterSystems IRIS skips the <trace> element message after the <throw> element.

   If we drill down into <catch>, we see this:

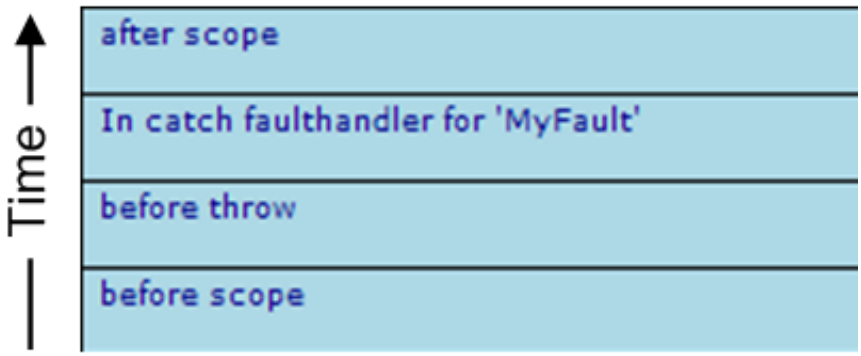


Note: If a <catchall> is provided, it must be the last statement in the <faulthandlers> block. All <catch> blocks must appear before <catchall>.

6. Within <catch>, the <trace> element generates the message `in catch faulthandler for 'MyFault'`.
7. The <scope> ends.
8. The last <trace> element generates the message `after scope`.

8.4.1 Event Log Entries

The corresponding Event Log entries look like this:



8.4.2 XData for This BPL

This BPL is defined by the following XData block:

Class Member

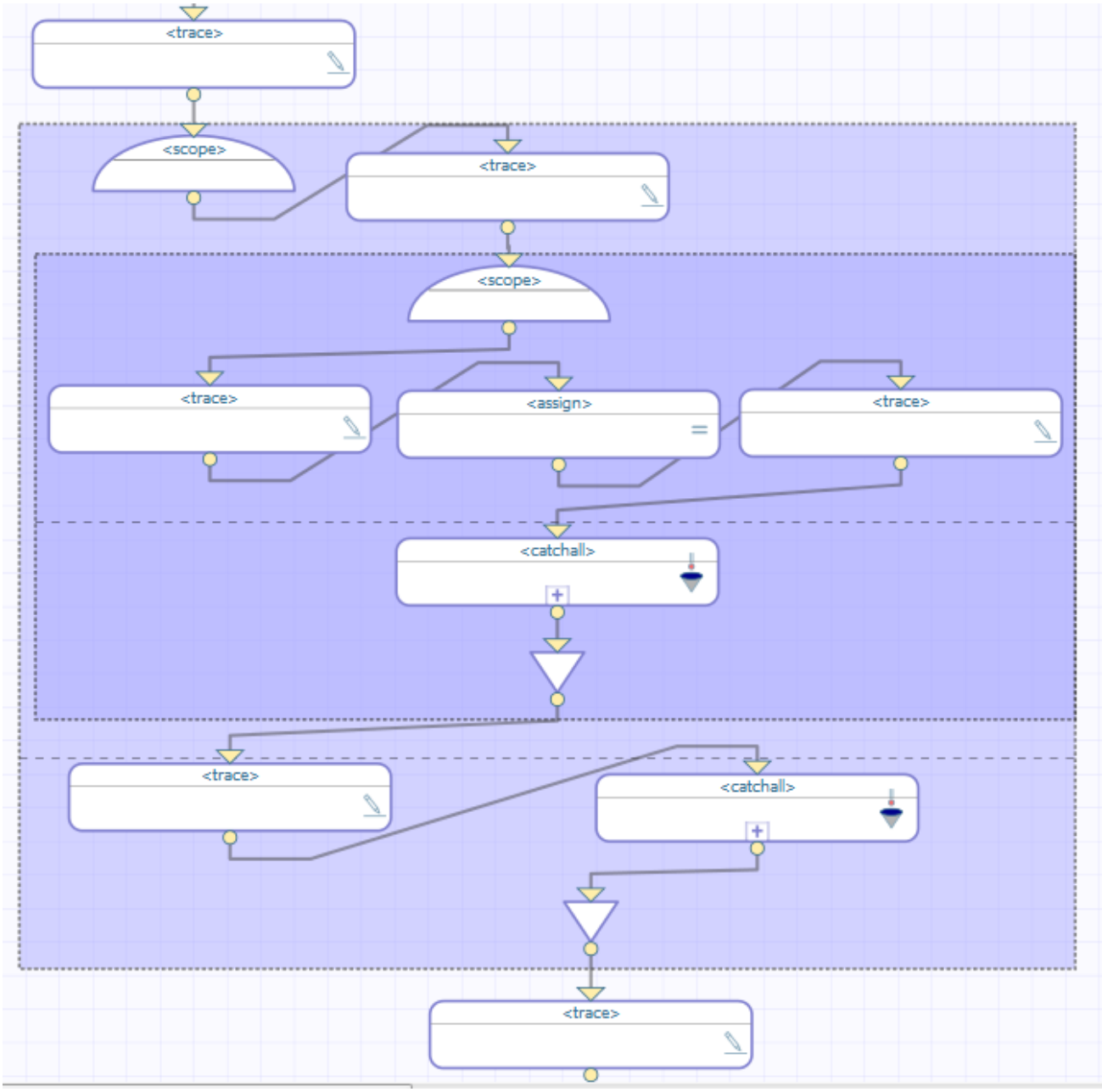
```
XData BPL
{
<process language='objectscript'
         request='Test.Scope.Request'
         response='Test.Scope.Response' >
  <sequence>
    <trace value='"before scope"'/>
    <scope>
      <trace value='"before throw"'/>
      <throw fault='"MyFault"'/>
      <trace value='"after throw"'/>
      <faulthandlers>
        <catch fault='"MyFault"'>
```

```
          <trace value='"In catch faulthandler for &apos;MyFault&apos;"'/>
        </catch>
        <catchall>
          <trace value='"in catchall faulthandler"'/>
          <trace value=
            '"%LastError "_
            $System.Status.GetErrorCodes(..%Context.%LastError)_
            " : "_
            $System.Status.GetOneStatusText(..%Context.%LastError)'
            />
          <trace value='"%LastFault "_..%Context.%LastFault'/>
        </catchall>
      </faulthandlers>
    </scope>
    <trace value='"after scope"'/>
  </sequence>
</process>
}
```

8.5 Nested Scopes, Inner Fault Handler Has Catchall

It is possible to nest <scope> elements. An error or fault that occurs within the inner scope may be caught within the inner scope, or the inner scope may ignore the error and allow it to be caught by the <faulthandlers> block in the outer scope. The next several topics illustrate how BPL handles errors and faults that occur within an inner scope, when two or more scopes are nested.

Suppose you have the following BPL (shown here without the <start> and <end> elements):

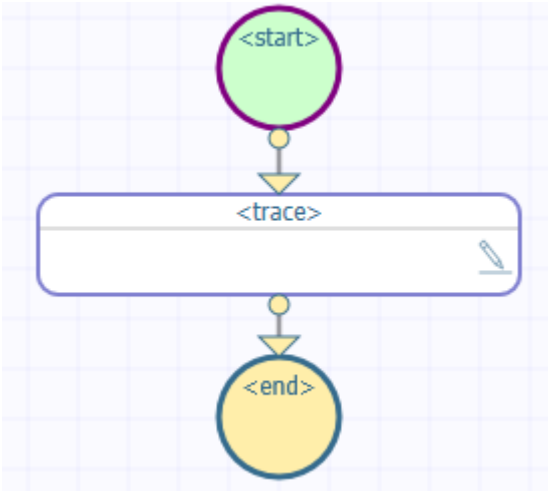


This BPL business process does the following:

1. The first <trace> element generates the message `before outer scope`.
2. The first <scope> element starts the outer scope.
3. The second <trace> element generates the message `in outer scope, before inner scope`.
4. The second <scope> element starts the inner scope.
5. The next <trace> element generates the message `in inner scope, before assign`.
6. The <assign> element tries to evaluate the expression 1/0. This attempt produces a divide-by-zero system error.
7. Control now goes to the <faulthandlers> defined within the inner <scope>. This <scope> rectangle includes a horizontal dashed line across the middle; the area below this dashed line displays the contents of the <faulthandlers> element. In this case, there is no <catch> but there is a <catchall>, so control goes there.

Note that InterSystems IRIS skips the <trace> element immediately after the <assign> element.

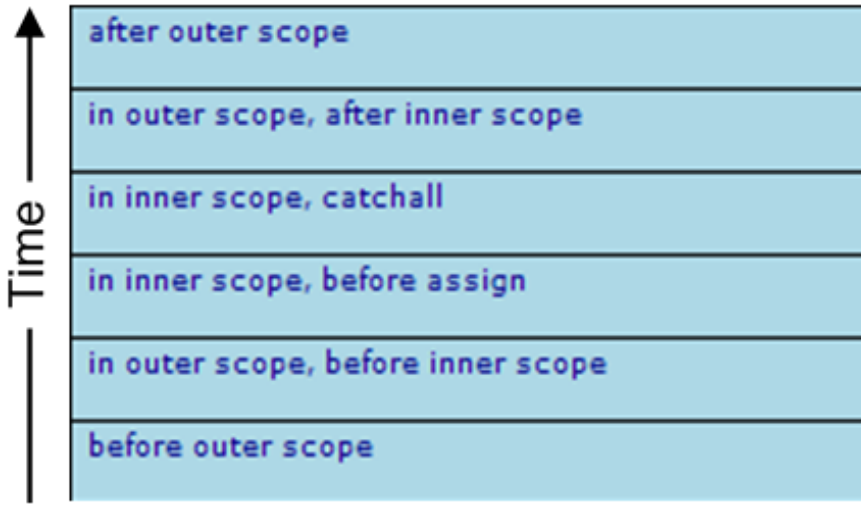
If we drill into this <catchall>, we see this:



8. Within this <catchall>, the <trace> element generates the message `in inner scope, catchall`.
9. The inner <scope> ends.
10. The next <trace> element generates the message `in outer scope, after inner scope`.
11. The outer <scope> rectangle includes a horizontal dashed line across the middle; the area below this dashed line displays the contents of the <faulthandlers> element that contains a <catchall>. Because there is no fault, this <catchall> is ignored.
12. The outer <scope> ends.
13. The last <trace> element generates the message `after outer scope`.

8.5.1 Event Log Entries

The corresponding Event Log entries look like this:



8.5.2 XData for This BPL

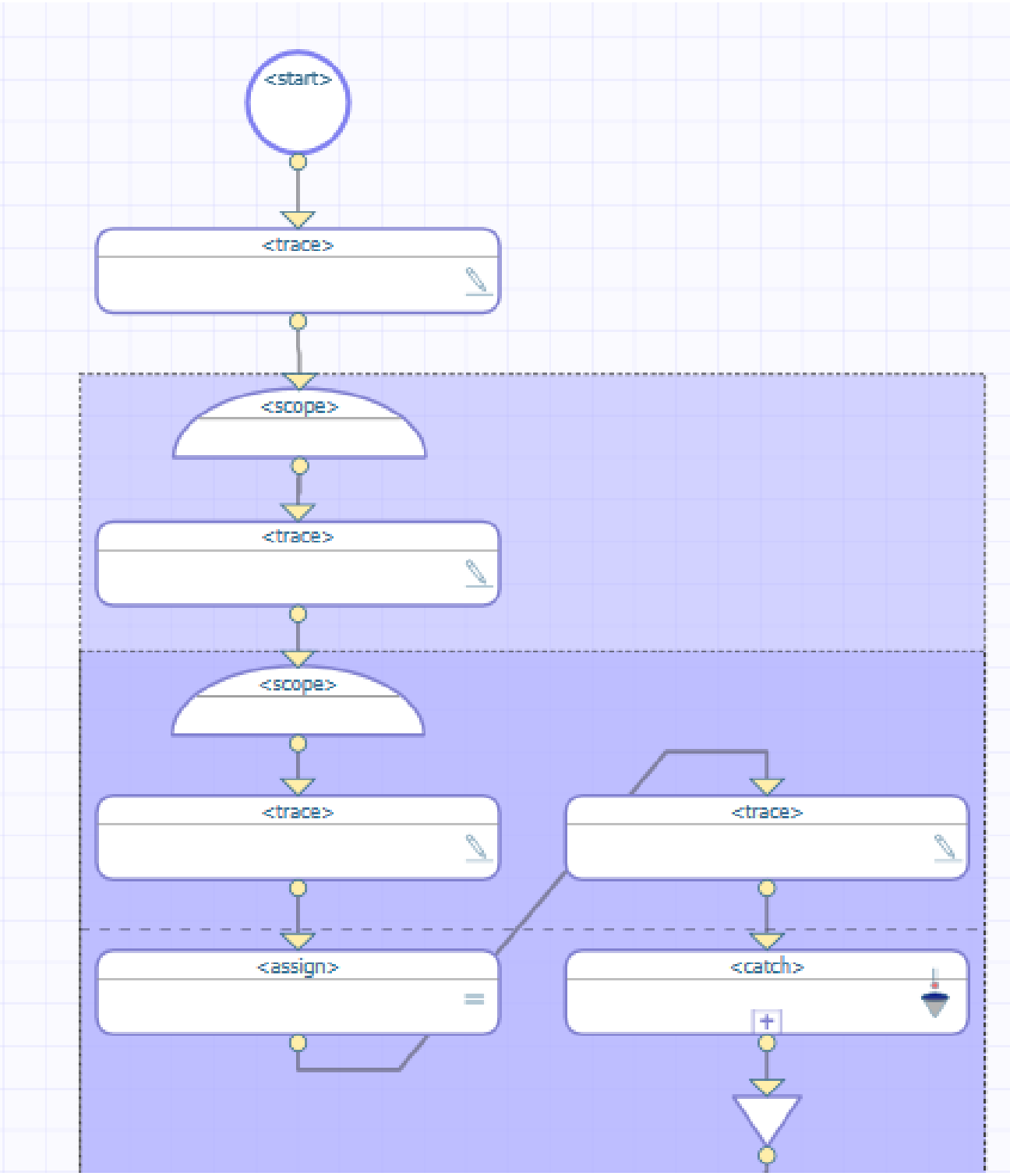
This BPL is defined by the following XData block:

Class Member

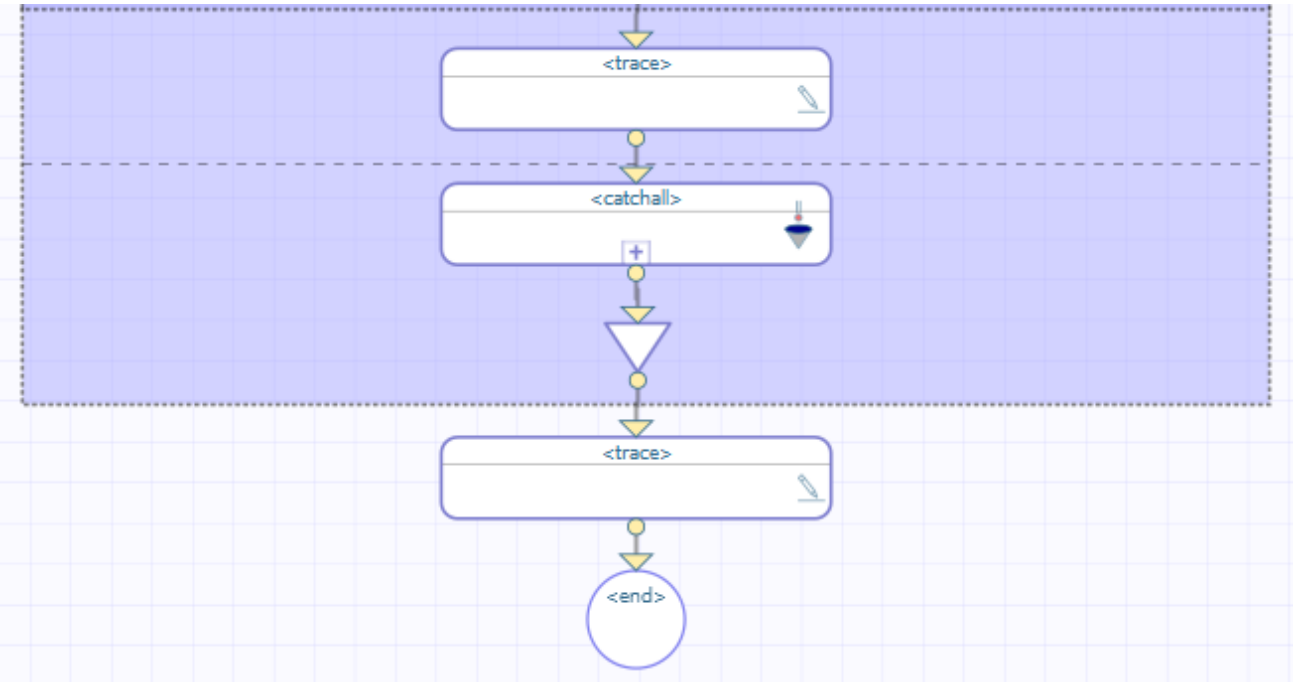
```
XData BPL
{
<process language='objectscript'
         request='Test.Scope.Request'
         response='Test.Scope.Response' >
  <sequence>
    <trace value='"before outer scope"'/>
    <scope>
      <trace value='"in outer scope, before inner scope"'/>
      <scope>
        <trace value='"in inner scope, before assign"'/>
        <assign property="SomeProperty" value="1/0"/>
        <trace value='"in inner scope, after assign"'/>
        <faulthandlers>
          <catchall>
            <trace value='"in inner scope, catchall"'/>
          </catchall>
        </faulthandlers>
      </scope>
      <trace value='"in outer scope, after inner scope"'/>
      <faulthandlers>
        <catchall>
          <trace value='"in outer scope, catchall"'/>
        </catchall>
      </faulthandlers>
    </scope>
    <trace value='"after outer scope"'/>
  </sequence>
</process>
}
```

8.6 Nested Scopes, Outer Fault Handler Has Catchall

Suppose you have the following BPL (partially shown):



The rest of this BPL is as follows:

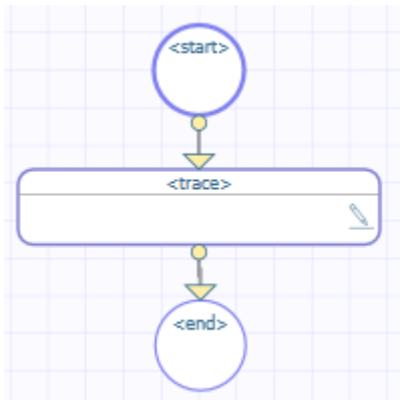


This BPL business process does the following:

1. The first <trace> element generates the message `before outer scope`.
2. The first <scope> element starts the outer scope.
3. The next <trace> element generates the message `in outer scope, before inner scope`.
4. The second <scope> element starts the inner scope.
5. The next <trace> element generates the message `in inner scope, before assign`.
6. The <assign> element tries to evaluate the expression 1/0. This attempt produces a divide-by-zero system error.
7. Control now goes to the <faulthandlers> defined within the inner <scope>. This <scope> rectangle includes a horizontal dashed line across the middle; the area below this dashed line displays the contents of the <faulthandlers> element. In this case, a <catch> exists, but its *fault* value does not match the thrown fault. There is no <catchall> in the inner scope.

   Note that InterSystems IRIS skips the <trace> element that is immediately after <assign>.
8. Control now goes to the <faulthandlers> block in the outer <scope>. No <catch> matches the fault, but there *is* a <catchall> block. Control goes to this <catchall>.

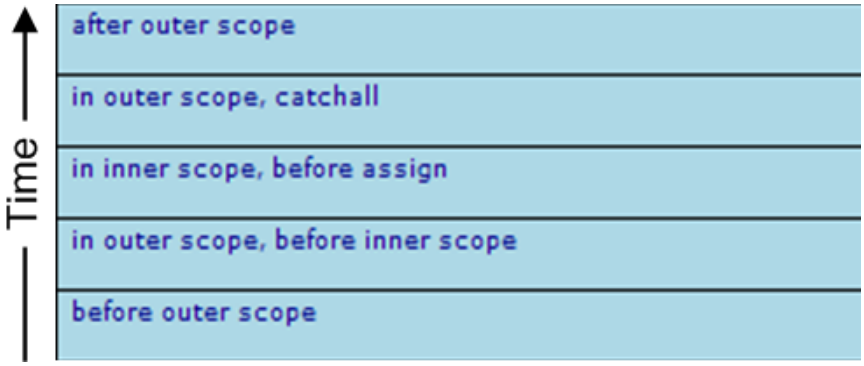
   If we drill into this <catchall>, we see this:



9. Within this <catchall>, the <trace> element generates the message `in outer scope, catchall`.
10. The outer <scope> ends.
11. The last <trace> element generates the message `after outer scope`.

8.6.1 Event Log Entries

The corresponding Event Log entries look like this:



8.6.2 XData for This BPL

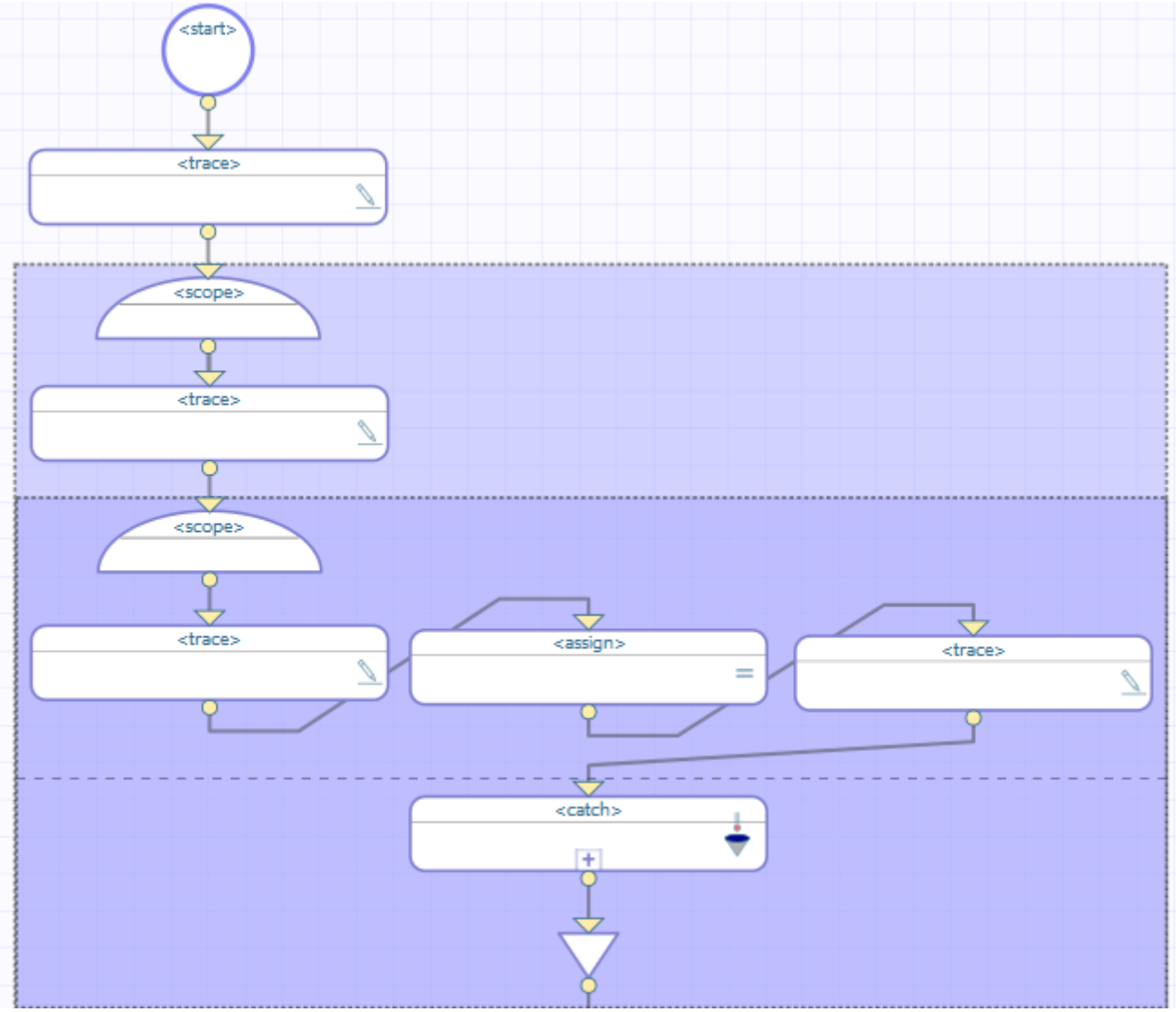
This BPL is defined by the following XData block:

Class Member

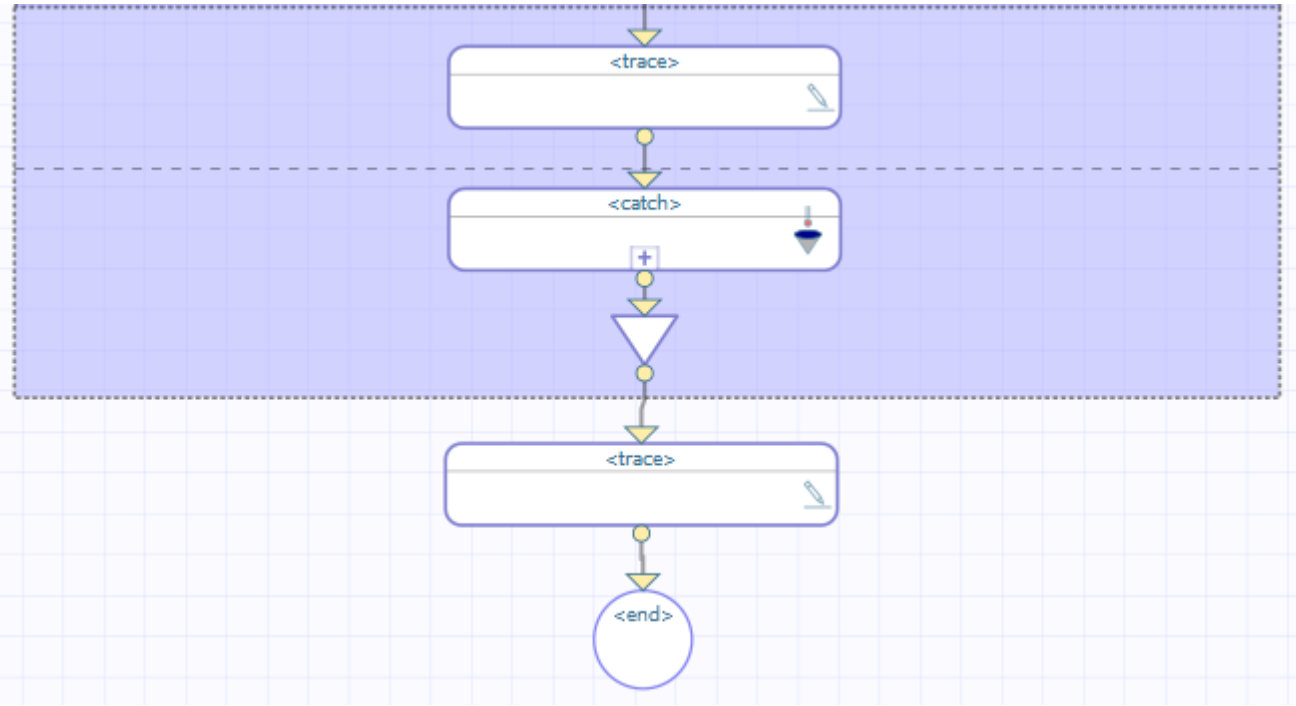
```
XData BPL
{
<process language='objectscript'
         request='Test.Scope.Request'
         response='Test.Scope.Response' >
  <sequence>
    <trace value='"before outer scope"'/>
    <scope>
      <trace value='"in outer scope, before inner scope"'/>
      <scope>
        <trace value='"in inner scope, before assign"'/>
        <assign property="SomeProperty" value="1/0"/>
        <trace value='"in inner scope, after assign"'/>
        <faulthandlers>
          <catch fault='"MismatchedFault"'>
            <trace value=
              '"In catch faulthandler for &apos;MismatchedFault&apos;"'/>
          </catch>
        </faulthandlers>
      </scope>
      <trace value='"in outer scope, after inner scope"'/>
      <faulthandlers>
        <catchall>
          <trace value='"in outer scope, catchall"'/>
        </catchall>
      </faulthandlers>
    </scope>
    <trace value='"after outer scope"'/>
  </sequence>
</process>
}
```

8.7 Nested Scopes, No Match in Either Scope

Suppose you have the following BPL (partially shown):



The rest of this BPL is as follows:

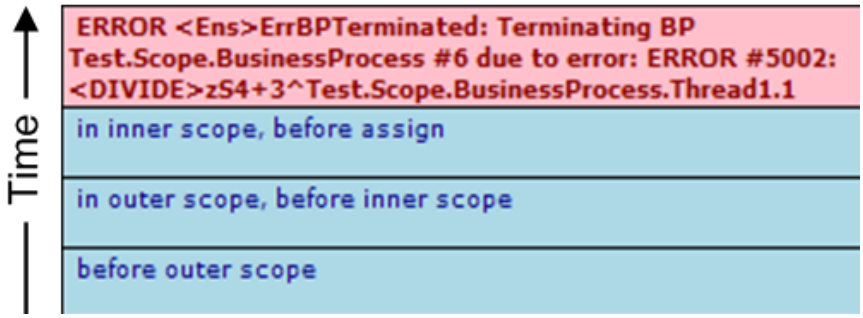


This BPL business process does the following:

1. The first <trace> element generates the message `before outer scope`.
2. The first <scope> element starts the outer scope.
3. The next <trace> element generates the message `in outer scope, before inner scope`.
4. The second <scope> element starts the inner scope.
5. The next <trace> element generates the message `in inner scope, before assign`.
6. The <assign> element tries to evaluate the expression 1/0. This attempt produces a divide-by-zero system error.
7. Control now goes to the <faulthandlers> block in the inner <scope>. The <scope> rectangle includes a horizontal dashed line across the middle; the area below this dashed line displays the contents of the <faulthandlers> element. In this case, a <catch> exists, but its *fault* value does not match the thrown fault. There is no <catchall> in the inner scope.
8. Control now goes to the <faulthandlers> block in the outer <scope>. No <catch> matches the fault, and there is no <catchall> block.
9. The BPL immediately stops, sending a message to the Event Log.

8.7.1 Event Log Entries

The corresponding Event Log entries look like this.



There is an important difference between this Event Log and the one in the System Error with No Fault Handling example. The two examples have this in common: Each fails to provide adequate fault handling for the case when the divide-by-zero error occurs.

The difference is that the System Error with No Fault Handling example has no <scope> and no <faulthandlers> block. Under these circumstances, InterSystems IRIS automatically outputs the system error to the Event Log, as shown in the first example.

The current example is different because each <scope> *does* include a <faulthandlers> block. Under these circumstances, InterSystems IRIS does not automatically output the system error to the Event Log, as it did in the System Error with No Fault Handling example. It is up to the BPL business process developer to decide to output <trace> messages to the Event Log in case of an unexpected error. In the current example, no <faulthandlers> block catches the fault, so the only information that is traced regarding the system error is contained in the automatic message about business process termination (item 4 above).

The system error message does appear in the ObjectScript shell:

```
ERROR #5002: ObjectScript error: <DIVIDE>zS4+3^Test.Scope.BusinessProcess.Thread1.1
```

8.7.2 XData for This BPL

This BPL is defined by the following XData block:

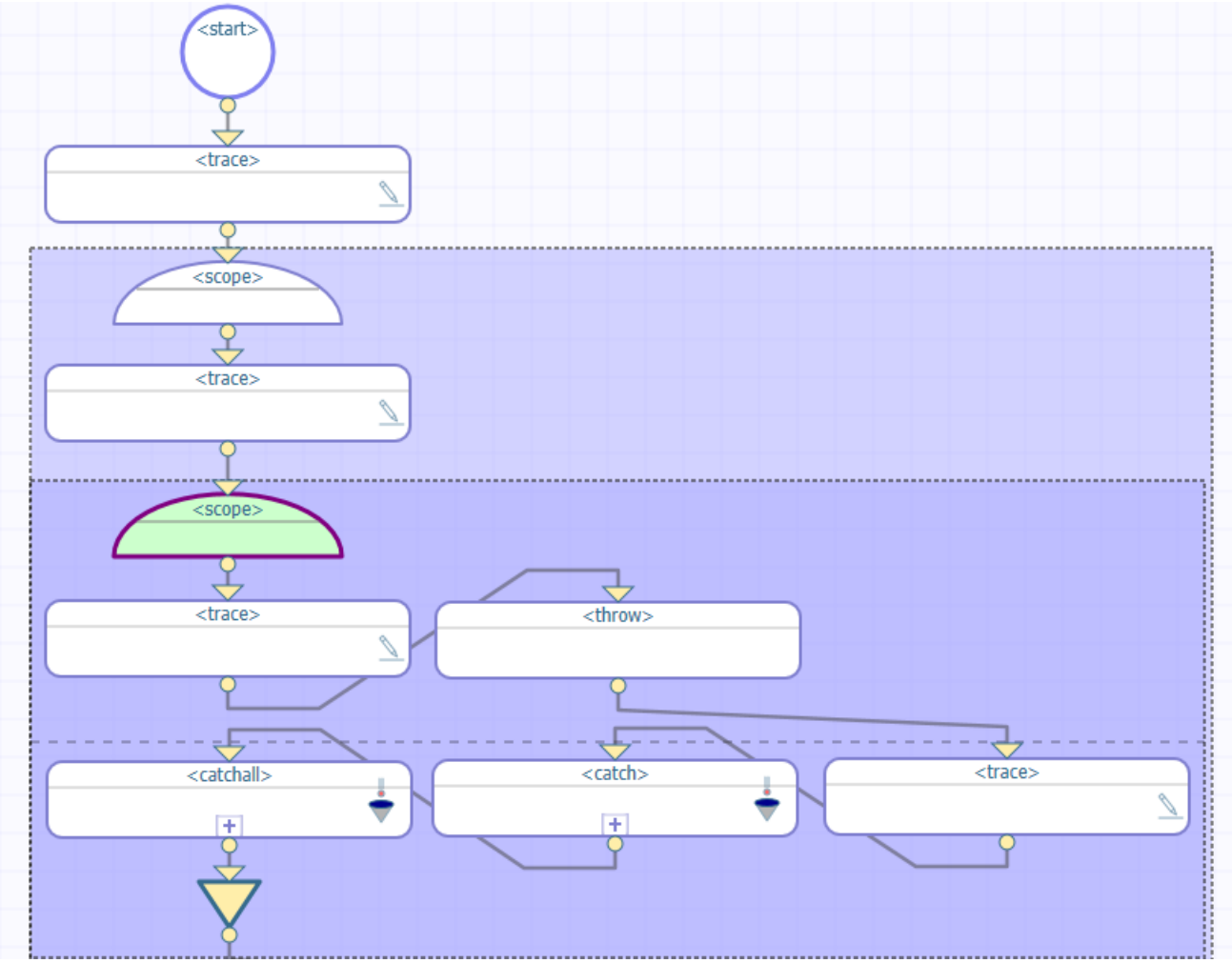
Class Member

```
XData BPL
{
<process language='objectscript'
         request='Test.Scope.Request'
         response='Test.Scope.Response' >
  <sequence>
    <trace value='"before outer scope"'/>
    <scope>
      <trace value='"in outer scope, before inner scope"'/>
      <scope>
        <trace value='"in inner scope, before assign"'/>
        <assign property="SomeProperty" value="1/0"/>
        <trace value='"in inner scope,after assign"'/>
        <faulthandlers>
          <catch fault='"MismatchedFault"'>
            <trace value=
              '"In catch faulthandler for &apos;MismatchedFault&apos;"'/>
          </catch>
        </faulthandlers>
      </scope>
      <trace value='"in outer scope, after inner scope"'/>
      <faulthandlers>
        <catch fault='"MismatchedFault"'>
          <trace value=
            '"In catch faulthandler for &apos;MismatchedFault&apos;"'/>
        </catch>
      </faulthandlers>
    </scope>
    <trace value='"after outer scope"'/>
```

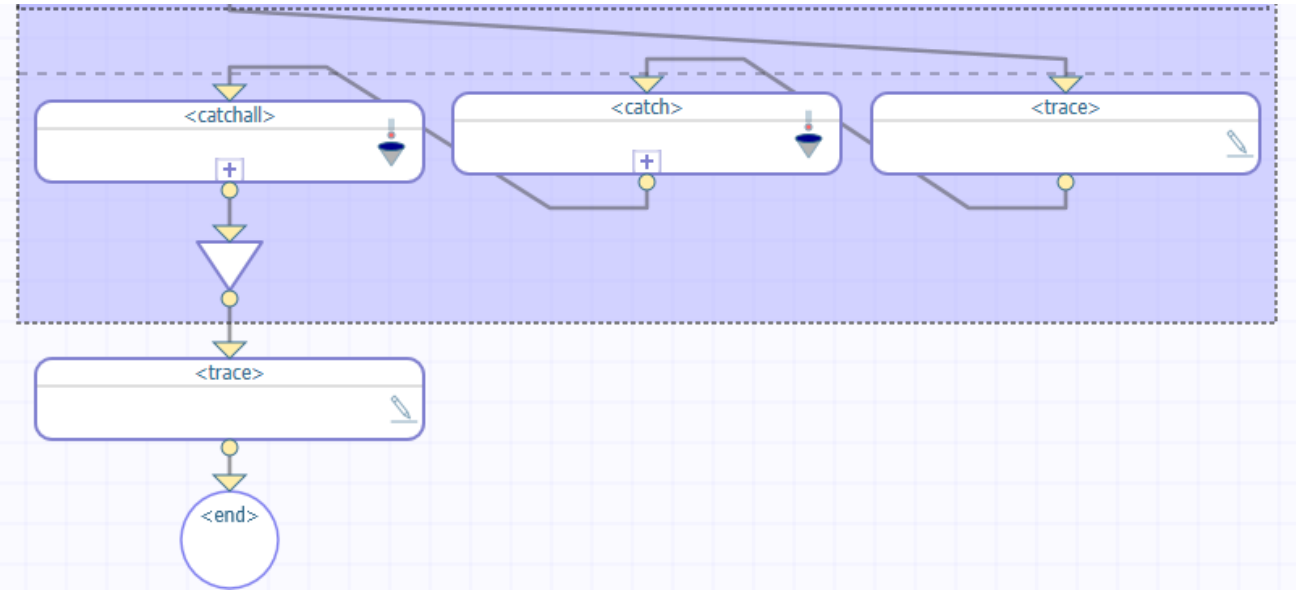
```
  </sequence>
</process>
}
```

8.8 Nested Scopes, Outer Fault Handler Has Catch

Suppose you have the following BPL (partially shown):



The rest of this BPL is as follows:

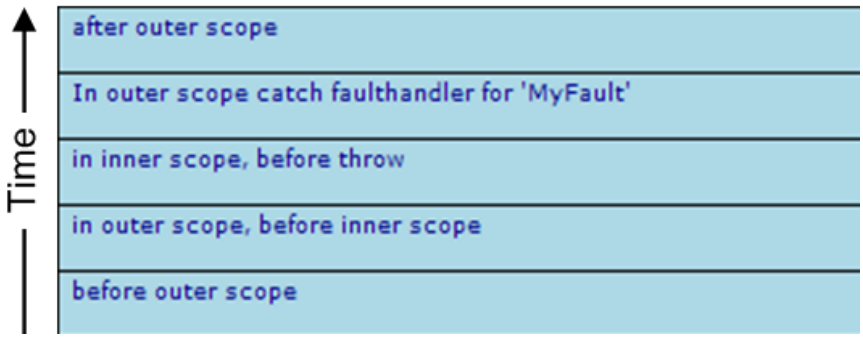


This BPL business process does the following:

1. The first <trace> element generates the message `before outer scope`.
2. The first <scope> element starts the outer scope.
3. The next <trace> element generates the message `in outer scope, before inner scope`.
4. The second <scope> element starts the inner scope.
5. The next <trace> element generates the message `in inner scope, before throw`.
6. The <throw> element throws a specific, named fault (`"MyFault"`).
7. Control now goes to the <faulthandlers> defined within the inner <scope>. A <catch> exists, but its *fault* value is `"MismatchedFault"`. There is no <catchall> in the inner scope.
8. Control goes to the <faulthandlers> block in the outer <scope>. It contains a <catch> whose *fault* value is `"MyFault"`.
9. The next <trace> element generates the message `in outer scope catch faulthandler for 'MyFault'`.
10. The second <scope> ends.
11. The last <trace> element generates the message `after outer scope`.

8.8.1 Event Log Entries

The corresponding Event Log entries look like this:



8.8.2 XData for This BPL

This BPL is defined by the following XData block:

Class Member

```
XData BPL
{
<process language='objectscript'
         request='Test.Scope.Request'
         response='Test.Scope.Response' >
  <sequence>
    <trace value='"before outer scope"'/>
    <scope>
      <trace value='"in outer scope, before inner scope"'/>
      <scope>
        <trace value='"in inner scope, before throw"'/>
        <throw fault='"MyFault"'/>
        <trace value='"in inner scope, after throw"'/>
        <faulthandlers>
          <catch fault='"MismatchedFault"'>
            <trace value=
    '"In inner scope catch faulthandler for &apos;MismatchedFault&apos;"'/>
          </catch>
        </faulthandlers>
      </scope>
      <trace value='"in outer scope, after inner scope"'/>
      <faulthandlers>
        <catch fault='"MyFault"'>
          <trace value=
            '"In outer scope catch faulthandler for &apos;MyFault&apos;"'/>
        </catch>
      </faulthandlers>
    </scope>
    <trace value='"after outer scope"'/>
  </sequence>
</process>
}
```

8.9 Thrown Fault with Compensation Handler

In business process management, it is often necessary to reverse some segment of logic. This convention is known as “compensation.” The ruling principle is that if the business process does something, it must be able to undo it. That is, if a failure occurs, the business process must be able to compensate by undoing the action that failed. You need to be able to unroll all of the actions from that failure point back to the beginning, as if the problem action never occurred. BPL enables this with a mechanism called a compensation handler.

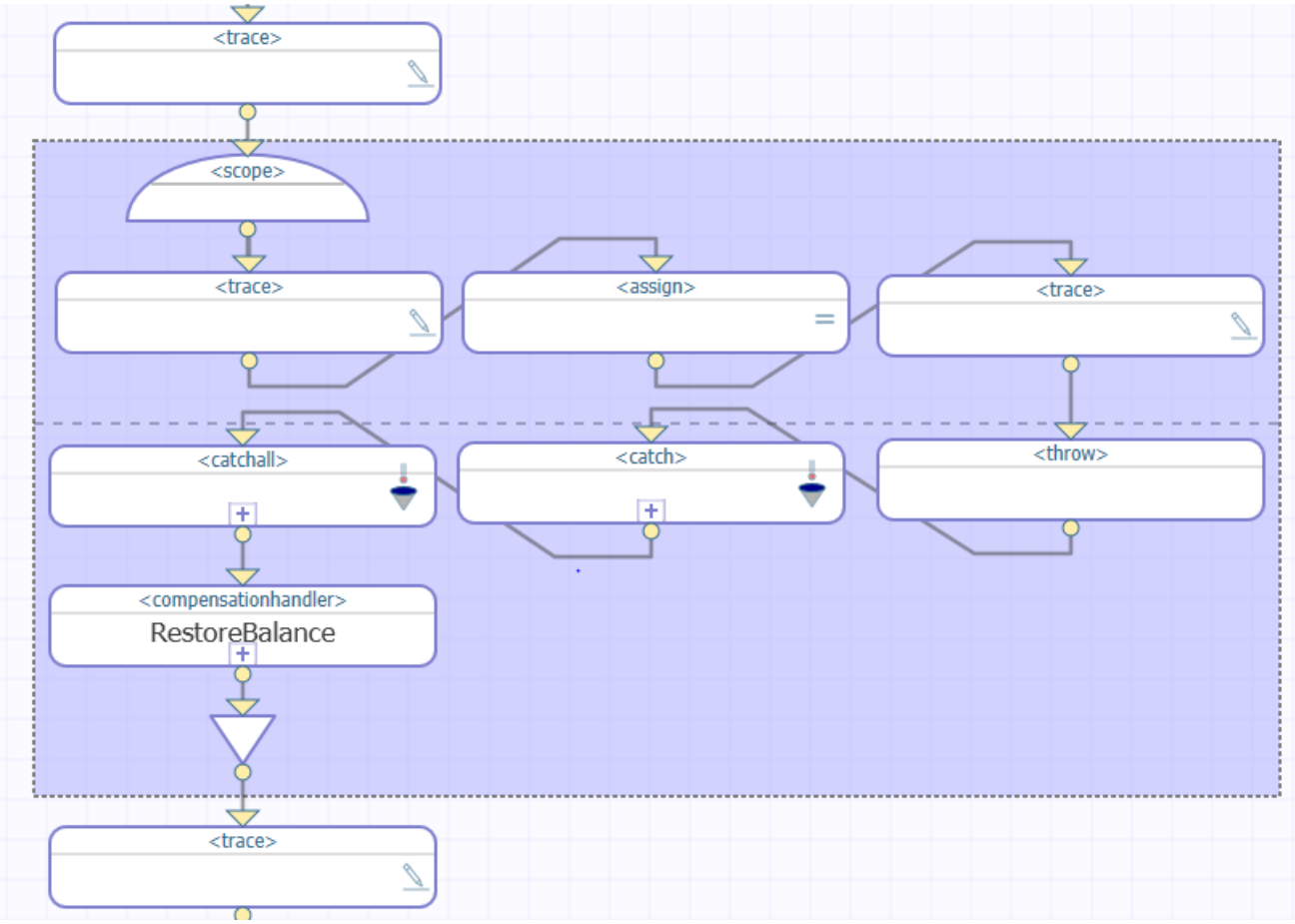
BPL <compensationhandler> blocks are somewhat like subroutines, but they do not provide a generalized subroutine mechanism. You can “call” them, but *only* from <faulthandler> blocks, and *only* within the same <scope> as the <compensationhandler> block. The <compensate> element invokes a <compensationhandler> block by specifying its name as a target. Extra quotes are *not* needed for this syntax:

XML

```
<compensate target="general"/>
```

Compensation handlers are only useful if you *can* undo the actions already performed. For example, if you transfer money into the wrong account, you can transfer it back again, but there are some actions that cannot be neatly undone. You must plan compensation handlers accordingly, and also organize them according to how far you want to roll things back.

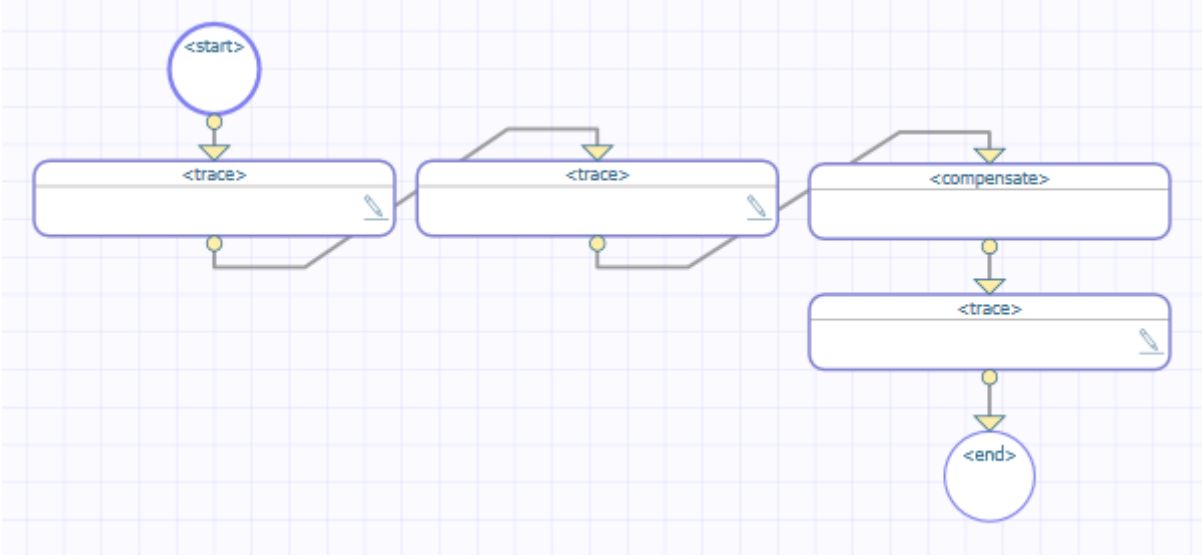
Suppose you have the following BPL:



This BPL business process does the following:

1. The Context tab (not shown) defines a property called MyBalance and sets its value to 100.
2. The first <trace> element generates the message `before scope balance is`, followed by the value of MyBalance.
3. The <scope> element starts the scope.
4. The next <trace> element generates the message `before debit`.
5. The <assign> element decrements MyBalance by 1.
6. The next <trace> element generates the message `after debit`.
7. The <throw> element throws a specific, named fault (`"BuyersRegret"`).
8. Control now goes to the <faulthandlers>. A <catch> exists whose *fault* value is `"BuyersRegret"`, so control goes there.

   If we drill down into this <catch> element, we see the following:

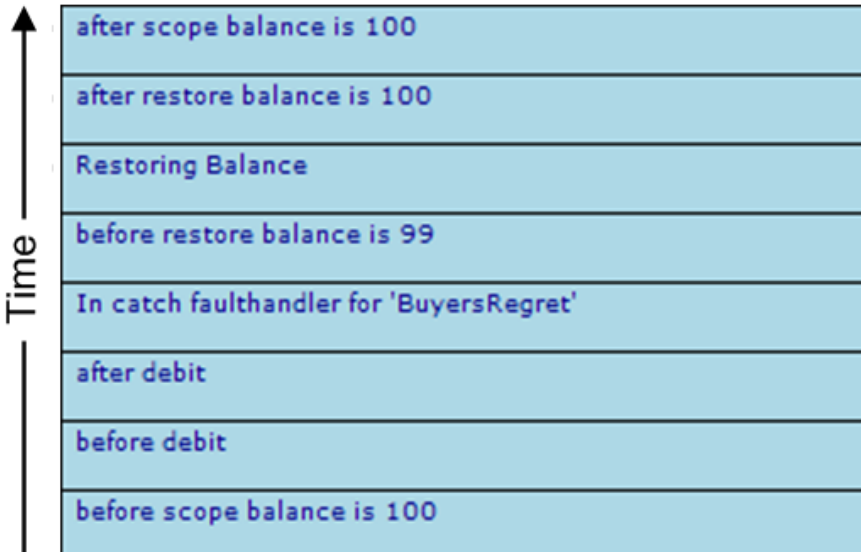


9. Within this <catch>, the first <trace> element generates the message `in catch faulthandler for 'BuyersRegret'`.
10. Within this <catch>, the second <trace> element generates the message `before restore balance is`, followed by the current value of MyBalance.
11. The <compensate> element is used. For this element, *target* is a <compensationhandler> whose *name* is RestoreBalance. Within this <compensationhandler> block:
    - A <trace> statement outputs the message “Restoring Balance”
    - An <assign> statement increments MyBalance by 1.

    Note: It is not possible to reverse the order of <compensationhandlers> and <faulthandlers>. If both blocks are provided, <compensationhandlers> must appear first and <faulthandlers> second.
12. The next <trace> element generates the message `after restore balance is`, followed by the current value of MyBalance.
13. The <scope> ends.
14. The last <trace> element generates the message `after scope balance is`, followed by the current value of MyBalance.

8.9.1 Event Log Entries

The corresponding Event Log entries look like this:



8.9.2 XData for This BPL

This BPL is defined by the following XData block:

Class Member

```
XData BPL
{
<process language='objectscript'
         request='Test.Scope.Request'
         response='Test.Scope.Response' >
  <context>
    <property name="MyBalance" type="%Library.Integer" initialexpression='100'/>
  </context>
  <sequence>
    <trace value='"before scope balance is "_context.MyBalance'/>
    <scope>
      <trace value='"before debit"'/>
      <assign property='context.MyBalance' value='context.MyBalance-1'/>
      <trace value='"after debit"'/>
      <throw fault='"BuyersRegret"'/>
      <compensationhandlers>
        <compensationhandler name="RestoreBalance">
          <trace value='"Restoring Balance"'/>
          <assign property='context.MyBalance' value='context.MyBalance+1'/>
        </compensationhandler>
      </compensationhandlers>
      <faulthandlers>
        <catch fault='"BuyersRegret"'>
          <trace value='"In catch faulthandler for &apos;BuyersRegret&apos;"'/>
          <trace value='"before restore balance is "_context.MyBalance'/>
          <compensate target="RestoreBalance"/>
          <trace value='"after restore balance is "_context.MyBalance'/>
        </catch>
        <catchall>
          <trace value='"in catchall faulthandler"'/>
          <trace value=
            '"%LastError "_
            $System.Status.GetErrorCodes(..%Context.%LastError)_
            " : "_
            $System.Status.GetOneStatusText(..%Context.%LastError)'
            />
          <trace value='"%LastFault "_..%Context.%LastFault'/>
        </catchall>
      </faulthandlers>
    </scope>
    <trace value='"after scope balance is "_context.MyBalance'/>
  </sequence>
</process>
}
```

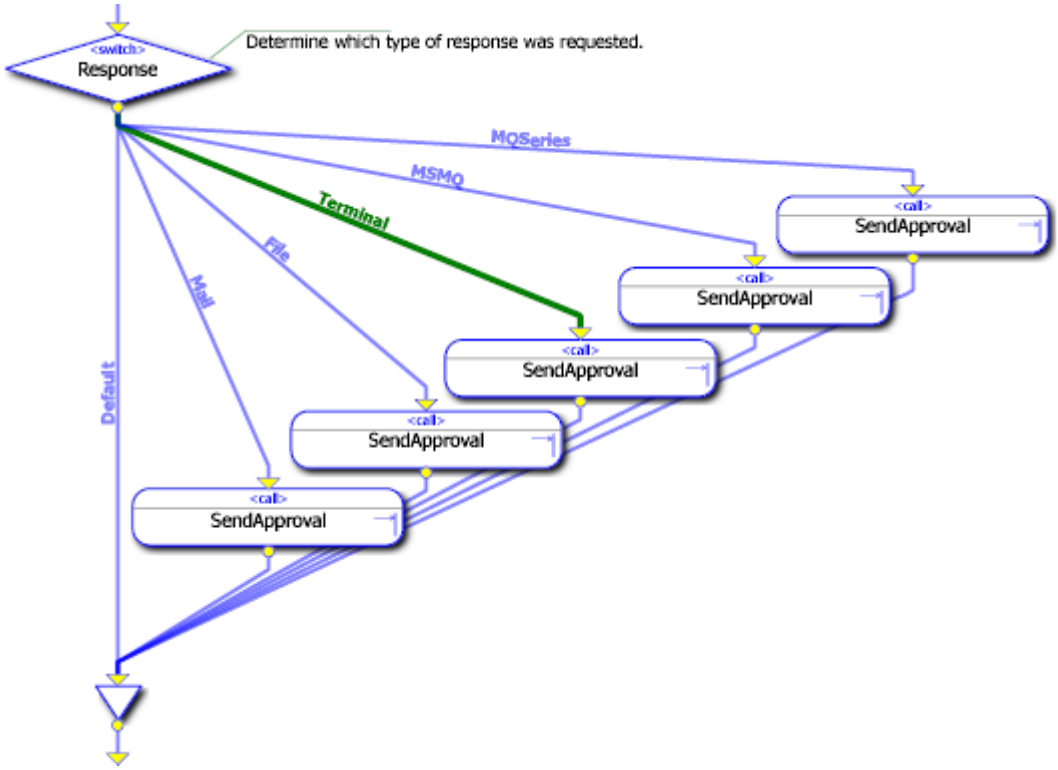
9

BPL Business Process Example

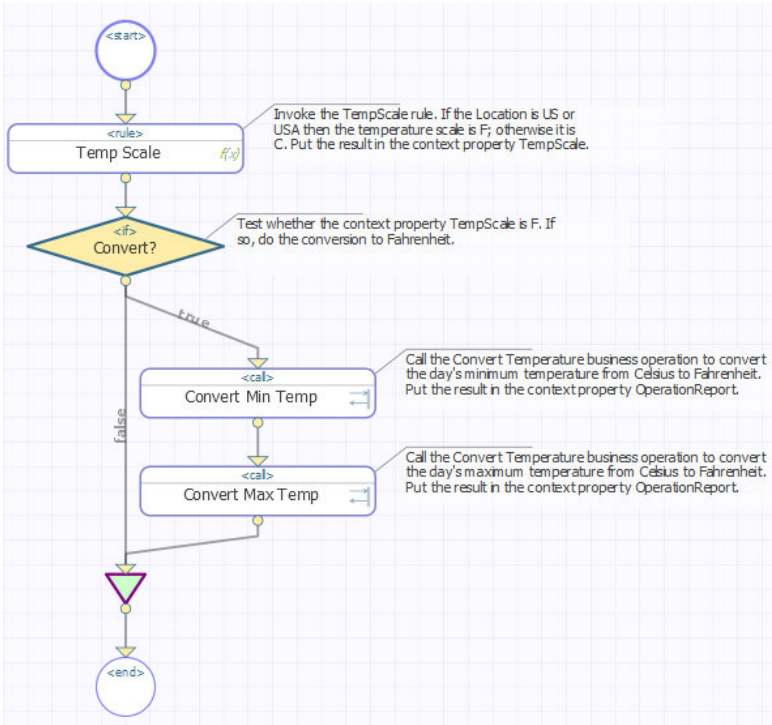
This page provides examples of BPL business processes.

9.1 Example with <switch>

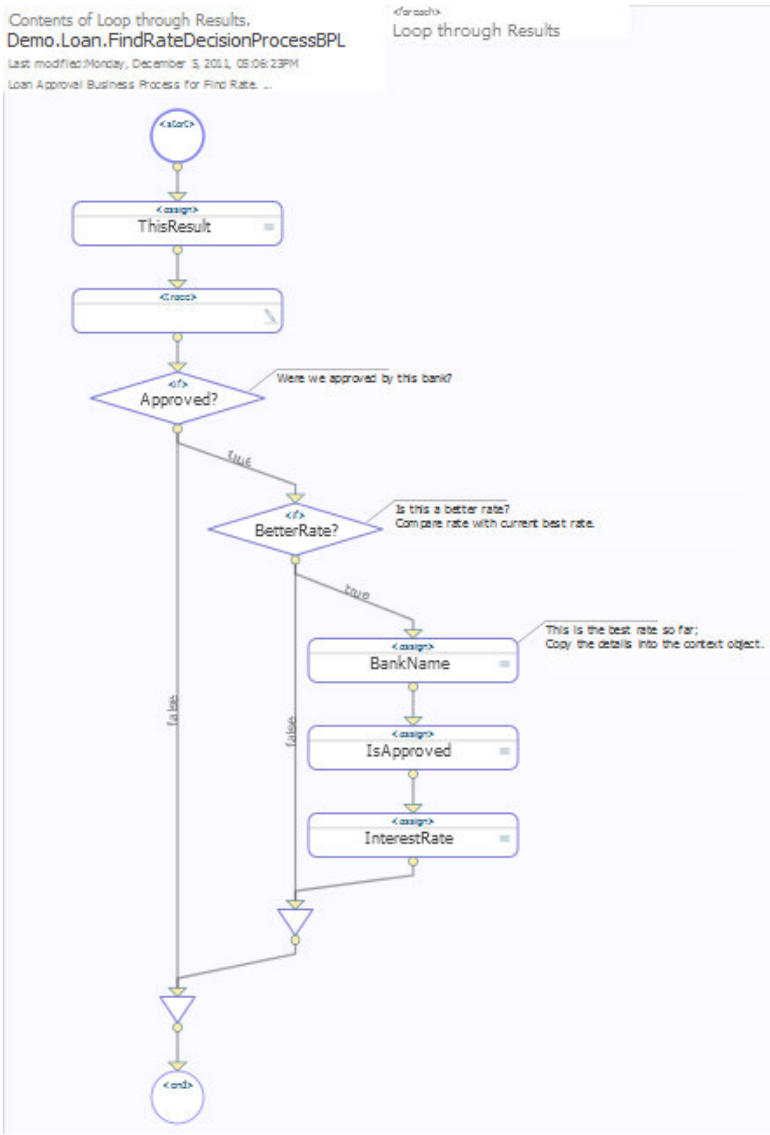
Within a <switch> activity, each possible path is automatically labeled with the corresponding <switch> value. All of the possible paths from a <switch> activity converge at a Join shape before a single arrow connects from the Join shape to the next activity in the BPL diagram.



9.2 Example 1 with <if>

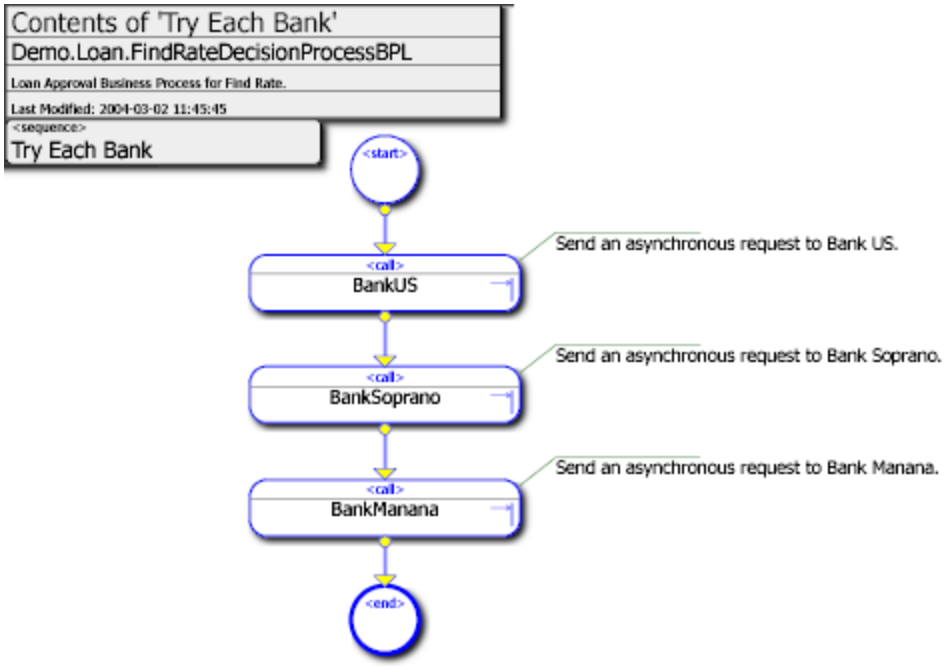


9.3 Example 2 with <if>



9.4 Example with <call>

In this example, three different banks can be consulted for prime rate and credit approval information.



The XML representation of this process is as follows:

Class Definition

```
/// Loan Approval Business Process for Bank Soprano.
/// Bank Soprano simulates a bank with great service but
/// somewhat high interest rates.
Class Demo.Loan.BankSoprano Extends Ens.BusinessProcessBPL
{

XData BPL
{
<process request="Demo.Loan.Msg.Application"
         response="Demo.Loan.Msg.Approval">

  <context>
    <property name="CreditRating" type="%Integer"/>
    <property name="PrimeRate" type="%Numeric"/>
  </context>

  <sequence>

  <trace value='"received application for "_request.Name'/>

  <assign name='Init Response'
          property="response.BankName"
          value='"BankSoprano"'>
    <annotation>
      <![CDATA[Initialize the response object.]]>
    </annotation>
  </assign>

  <call name="PrimeRate"
        target="Demo.Loan.WebOperations"
        async="1">
    <annotation>
      <![CDATA[Send an asynchronous request for the Prime Rate.]]>
    </annotation>
    <request type="Demo.Loan.Msg.PrimeRateRequest"/>
    <response type="Demo.Loan.Msg.PrimeRateResponse">
      <assign property="context.PrimeRate"
              value="callresponse.PrimeRate"/>
    </response>
  </call>

  <call name="CreditRating"
        target="Demo.Loan.WebOperations"
        async="1">
    <annotation>
      <![CDATA[Send an asynchronous request for the Credit Rating.]]>
```

```
    </annotation>
    <request type="Demo.Loan.Msg.CreditRatingRequest">
      <assign property="callrequest.TaxID" value='request.TaxID'/>
    </request>
    <response type="Demo.Loan.Msg.CreditRatingResponse">
      <assign property="context.CreditRating"
              value="callresponse.CreditRating"/>
    </response>
  </call>

  <sync name='Wait'
        calls="PrimeRate,CreditRating"
        type="all"
        timeout="10">
    <annotation>
      <![CDATA[Wait for the response from the async requests.
               Wait for up to 10 seconds.]]>
    </annotation>
  </sync>

  <switch name='Approved?'>

    <case name='No PrimeRate'
          condition='context.PrimeRate=""'>
      <assign name='Not Approved'
              property="response.IsApproved"
              value="0"/>
    </case>

    <case name='No Credit'
          condition='context.CreditRating=""'>
      <assign name='Not Approved'
              property="response.IsApproved"
              value="0"/>
    </case>

    <default name='Approved' >
      <assign name='Approved'
              property="response.IsApproved"
              value="1"/>
      <assign name='InterestRate'
          property="response.InterestRate"
          value="context.PrimeRate+10+(99*(1-(context.CreditRating/100)))">
        <annotation>
          <![CDATA[Copy InterestRate into response object.]]>
        </annotation>
      </assign>
    </default>

  </switch>

  <delay
    name='Delay'
    duration="2+($zcrc(request.Name,4)#5)">
    <annotation>
      <![CDATA[Wait for a random duration.]]>
    </annotation>
  </delay>

  <trace value='"application is "
         _$s(response.IsApproved:"approved for "_response.InterestRate_"%",
         1:"denied")'/>

  </sequence>
</process>
}

}
```

10

Listing and Managing Business Processes

The **Business Process List** page provides information on the business processes available for interoperability productions in the current namespace. From this list, you have options to create, edit, and see the activity of business processes.

10.1 Introduction

To access the **Business Process List** page in the Management Portal, click **Interoperability** > **List** > **Business Processes**.

The page lists the business processes available in the current namespace. This page lists two kinds of business processes:

- BPL business processes are displayed in blue. You can double-click one to open it in the BPL Editor.
- Business processes displayed in black are custom classes you must edit in an IDE.

10.2 Options on This Page

This page provides the following buttons:

- **New**—Click this to create a new BPL.
- **Open** (*BPL classes only*)—Select a BPL business process class and then click this button to edit that class in the BPL Editor.
- **Export**—Select a BPL business process class and then click this button to export that class to an XML file.
- **Import**—Click to import a business process that was exported to an XML file.
- **Delete**—Select a BPL business process class and then click this button to delete that class.
- **Instances**—Select a BPL business process class and then click this button to list any instances of that class in the running production. If a business process has completed its work, there is no entry for it on this page.

  See Monitoring Productions.
- **Rule Log**—Select a BPL business process class and then click this button to view the business rule log for rules invoked by this business process.

  See Monitoring Productions.

10.3 Related Options

You can also export and import business process classes as you do any other class in InterSystems IRIS. You can use the **Classes** page of the Management Portal, which is accessed by selecting **System Explorer** > **Classes**.

10.4 See Also

- Creating BPL Business Processes
- Monitoring Productions

BPL Reference

This reference provides detailed information about each BPL element.

Common BPL Attributes and Elements

Describes attributes and elements that are present in most BPL elements used in BPL business processes.

Common Attributes

Most BPL elements can contain the following attributes, which are listed here for brevity.

name

Usually optional. The name of this element. Specify a string of up to 255 characters.

disabled

Optional. You can temporarily disable the element by setting its *disabled* attribute to 1 (true). To re-enable the element, either remove the *disabled* attribute or set it to 0 (false).

xpos

Optional. Sets the *x* coordinate of the graphic that represents this element in BPL diagrams. Ignored by the BPL compiler. Specify a positive integer.

ypos

Optional. The *y* coordinate. Specify a positive integer.

xend

Optional. If the graphic that represents this element has two icons (start and end), then *xend* sets the *x* coordinate for the ending icon. Ignored by the BPL compiler. Specify a positive integer.

yend

Optional. The ending *y* coordinate. Specify a positive integer.

Common Element: <annotation>

Most BPL elements can contain the <annotation> element, which allows you to associate descriptive text with a shape in a BPL diagram. This element is as follows:

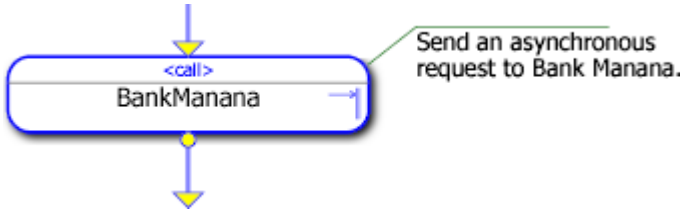
```
<annotation>
   <![CDATA[ Gets the current Account Balance for a customer.]]>
</annotation>
```

The text within the CDATA block appears as a commentary on the associated activity. The following example provides an <annotation> for a <call> activity:

XML

```
<call name="BankManana">
  <annotation>
    <![CDATA[Send an asynchronous
      request to Bank Manana.]]>
  </annotation>
</call>
```

The CDATA block enables you to include line breaks and special characters such as the apostrophe (') without needed to use XML escape sequences. Note the line break between `asynchronous` and `request` in the example above, which the diagram reproduces literally as follows:



The maximum length of the <annotation> string is 32,767 characters, including the CDATA escape characters.

BPL <alert>

Sends an alert message to a user device, as a step in a BPL business process.

Syntax

```
<alert value="The system needs service right away."/>
```

Attributes and Elements

value attribute

Required. The text for the alert message. Specify an expression or a literal string.

LanguageOverride attribute

Optional. Specifies the scripting language in which any expressions (within in this element) are written.

Can be `"python"`, `"objectscript"`, or `"basic"` (not documented). Default is the language specified in the containing <process> element.

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Description

The <alert> element sends an alert message to a user device.

The text of the message is always written to the Event Log as an entry of type Alert. However, the real purpose of the <alert> element is to contact the user through email or other notification device. The <alert> element does this by sending the text of the message to a configuration item called Ens.Alert, which has been set up with all the information necessary to contact user devices outside InterSystems IRIS.

Important: If no Ens.Alert item has been configured as a member of the production, the <alert> simply goes to the Event Log.

For details, see Defining Alert Processors.

BPL <assign>

Assigns a value to a property, as a step in a BPL business process.

Syntax

```
<assign property="propertyname" value="expression" />
```

Attributes and Elements

property attribute

Required. The target of this assignment. This must be a property in an execution context object, usually *context*, *request*, *response*, *callrequest*, or *callresponse*. For details, see the table in the Description section.

value attribute

Required. Value of the property. Specify a literal value or an expression that returns a valid value for the property.

LanguageOverride attribute

Optional. Specifies the scripting language in which any expressions (within in this element) are written.

Can be `"python"`, `"objectscript"`, or `"basic"` (not documented). Default is the language specified in the containing <process> element.

action attribute

Optional. If *property* is a collection (list or array), use *action* to specify the type of assignment to perform on the collection. If not specified, a `set` is performed. Specify a literal string, either `append`, `set`, `clear`, `insert`, or `remove` as described below.

key attribute

Optional, except in some cases when *property* is a collection (list or array). If so, you must use this key to specify the member of the collection that is the target of this assignment. Specify an expression that evaluates to a key.

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Description

This section describes the importance of the execution context to BPL business processes, and explains how to use the <assign> element to set values in the business process execution context.

A business process must have certain state information saved to disk and restored from disk, whenever it suspends or resumes execution. This feature is especially important for long-running business processes, which may take days or weeks to complete. To address this need, InterSystems IRIS provides every BPL business process with a group of objects and variables called the *execution context*. The variables in the execution context are automatically saved and restored each time the BPL business process suspends and resumes execution. The correct operation of a BPL business process depends on the appropriate use of these variables.

Every variable in the execution context has a specific name and purpose, and can have its value set using the <assign> element. The following table lists the variables in the execution context.

Variable	Purpose
callrequest	The *callrequest* object contains any properties that are required to build the request message object to be sent by a <call>. Within the corresponding <request> activity, use a sequence of <assign> elements to set the property values in *callrequest*.
callresponse	Upon completion of a <call> activity, the *callresponse* object contains the properties of the response message object that was returned to the <call>. Within the corresponding <response> activity, use a sequence of <assign> elements to copy the returned values from properties on *callresponse* into properties on *context* or *response*.
context	The *context* object is a general-purpose data container for the business process. *context* has no automatic definition. To define properties of this object, use the <context> element. That done, you may refer to these properties anywhere inside the <process> element using dot syntax, as in: `context.Balance`
request	The *request* object contains any properties of the original request message object that caused this business process to be instantiated. You may refer to *request* properties anywhere inside the <process> element using dot syntax, as in: `request.UserID`
response	The *response* object contains any properties that are required to build the final response message object to be returned by the business process. You may refer to *response* properties anywhere inside the <process> element using dot syntax, as in: `response.IsApproved`. Use the <assign> element to assign values to these properties.
status	*status* is a value of type %Status that indicates success or failure. As the BPL business process runs, if at any time *status* acquires a failure value, InterSystems IRIS immediately terminates the business process and writes a text message to the Event Log indicating the reason for failure. In general, this happens automatically, when unsuccessful values are returned from <call> activities. However, BPL business process code can initiate a immediate, but graceful exit by setting the *status* value using <assign> or <code>. See the description at the end of this topic.
syncresponses	*syncresponses* is a collection of response objects, keyed by the names of the <call> activities being synchronized. Only completed calls are represented. You can retrieve a response from *syncresponses* only after a <sync> and before the end of the current <sequence>. Do so using the syntax `syncresponses.GetAt("MyName")` where the relevant call was defined as `<call name="MyName">`

Variable	Purpose
synctimedout	The *synctimedout* value is an integer. *synctimedout* indicates the outcome of a <sync> activity after several calls. You can test the value of *synctimedout* after the <sync> and before the end of the <sequence> that contains the calls and <sync>. *synctimedout* has one of three values: • If 0, no call timed out. All the calls had time to complete. This is also the value if the <sync> activity had no *timeout* set. • If 1, at least one call timed out. This means not all <call> activities completed before the timeout. • If 2, at least one call was interrupted before it could complete. Generally you will test *synctimedout* for status and then retrieve the responses from completed calls out of the *syncresponses* collection.

CAUTION: Like all other execution context variable names, *status* is a reserved word in BPL. Do not use it with <assign> except as described above.

The BPL <assign> element specifies a target and an expression that will be assigned to it. The target may be a property in one of the objects in the business process execution context, or it may be one of the single-valued variables such as *status*. The properties involved in an <assign> element can be data types, objects, or collections of either. Collection properties are declared by setting the *collection* attribute to “array” or “list” in the corresponding <property> element.

As described in the above table, the object called *context* serves as a general-purpose context object for the business process. Properties in the *context* object are defined using the <context> and <property> elements at the beginning of the <process> environment. For example:

XML

```
<process request="Demo.Loan.Msg.Application" response="Demo.Loan.Msg.Approval">
  <context>
    <property name="CreditRating" type="%Integer"/>
    <property name="PrimeRate" type="%Numeric"/>
  </context>
  ...
</process>
```

The above BPL excerpt defines two *context* properties for this business process—context.CreditRating and context.PrimeRate—but does not assign values to them. An <assign> element anywhere below this <context> element and within the <process> environment can assign a value to any of these properties as needed. For example:

XML

```
<process request="Demo.Loan.Msg.Application" response="Demo.Loan.Msg.Approval">
  <context>
    <property name="CreditRating" type="%Integer"/>
    <property name="PrimeRate" type="%Numeric"/>
  </context>
  <sequence>
    <call name="PrimeRate" target="Demo.Loan.WebOperations" async="0">
      <request type="Demo.Loan.Msg.PrimeRateRequest">
      </request>
      <response type="Demo.Loan.Msg.PrimeRateResponse">
        <assign property="context.PrimeRate" value="callresponse.PrimeRate"/>
      </response>
    </call>
    ...
  </sequence>
  ...
</process>
```

The above BPL excerpt continues the first one. Note that the <call> element in this example is synchronous, and has both a <request> and a <response> element.

The <response> in this case contains an <assign> operation that references two properties on objects inside the execution context: context.PrimeRate (from the general-purpose context object) and callresponse.PrimeRate (from the response object associated with the current <call> element, in this case Demo.Loan.Msg.PrimeRateResponse as you can see above). The <assign> operation receives the value of the PrimeRate property returned from the <call> and places it in the general-purpose context object.

Inside the <sequence> element shown above, and continuing from the <call> element just discussed, the example continues as follows:

XML

```
<call name="CreditRating" target="Demo.Loan.WebOperations" async="0">
  <request type="Demo.Loan.Msg.CreditRatingRequest">
    <assign property="callrequest.SSN" value='request.SSN'/>
  </request>
  <response type="Demo.Loan.Msg.CreditRatingResponse">
    <assign property="context.CreditRating" value="callresponse.CreditRating"/>
  </response>
</call>
```

The above statements assign the SSN property from the primary request (request.SSN) to the SSN property in the request being made by the current <call> element (callrequest.SSN). After this assignment is made, the <call> element issues the request. It is a synchronous call of type Demo.Loan.WebOperations. When a response returns, the <call> element gets the value of the CreditRating property returned from the <call> (callresponse.CreditRating) and places it in a property on the general-purpose context object (context.CreditRating).

The following statement assigns the integer value 1 to the IsApproved property in the primary response object for the business process (response.IsApproved). In this example, IsApproved is a Boolean value (true or false) according to InterSystems IRIS conventions. That is, an integer value of 1 means true (the applicant was approved), and 0 means false (the applicant was not approved).

XML

```
<assign name='IsApproved' property="response.IsApproved" value="1">
  <annotation>
    <![CDATA[Copy IsApproved into the response object.]]>
  </annotation>
</assign>
```

The following statement assigns a calculated value—the result of an expression involving two properties in the general-purpose context object—to the InterestRate property in the primary response object for the business process (response.InterestRate):

XML

```
<assign name='InterestRate'
        property="response.InterestRate"
        value="context.PrimeRate+1+(2*(1-(context.CreditRating/100)))">
  <annotation>
    <![CDATA[Copy InterestRate into the response object.]]>
  </annotation>
</assign>
```

Types of <assign> Operation

The syntax for the BPL <assign> element works as follows:

1. The property attribute identifies an object and property that is the target of the assignment operation.

2. The value attribute provides the value for the target property. This may be an expression that is evaluated at runtime to provide a value for the assignment. Expressions within an <assign> element must use the language specified by the <process> element for the business process.
3. There are several types of BPL <assign> operation, as specified by the optional action attribute. The allowable values for the action attribute are:

Value	Description
`append`	Add the target element to the end of the list.
`set`	(Default) Set the target element to a new value.
`insert`	Insert a new value into the collection.
`remove`	Remove the target element from the collection.
`clear`	Clear the contents of the target collection.

Aside from the default value `set`, most of these variations are intended to handle assignments involving collection properties. The various assignment types are summarized in the following table.

Property Type	action Attribute Value	key Attribute Required	Result
Non-collection	`set`	No	Property is set to new value
Array	`clear`	No	Array is cleared
Array	`remove`	Yes	Element at *key* is removed
Array	`set`	Yes	Element at *key* is set to new value
List	`append`	No	Element is added to the end of the list
List	`clear`	No	List is cleared
List	`insert`	Yes	Element is inserted at position determined by *key*
List	`remove`	Yes	Element at *key* is removed
List	`set`	Yes	Element at *key* is replaced

Details about each type of BPL <assign> operation follow.

The append Operation

The `append` operation adds the target element to the end of a list property.

The set Operation

The `set` operation sets the value of the specified property to the value of the value attribute. Note that the value attribute contains an expression and can itself refer to an object or property of an object within the execution context:

XML

```
<assign name='CopyResult' property='context.SSN' value='callresponse.SSN' />
```

If the target property is an array collection, then the value of the key attribute specifies an item in the array, otherwise the key attribute is ignored.

If the target property is a collection and the value attribute specifies a collection of the same type, then the collection contents are copied into the target collection:

XML

```
<assign name='CopyResults' property='context.List' value='callresponse.List' />
```

The default action for the assign element is the set operation; if action is not specified, then the assign specifies a set operation.

The clear Operation

This operation applies to collection properties only. The `clear` operation clears the contents of the specified collection property. The value and key attributes are ignored, but since the BPL schema for the <assign> element requires it, a value attribute must be present in the statement.

For example, the following will clear the contents of the collection property List:

XML

```
<assign name='ClearResults' property='context.List' action='clear' value='' />
```

The insert Operation

This applies to list collection properties only. The `insert` operation inserts a value into the specified collection property. If the key attribute is present the new value is inserted after the position (an integer) specified by key otherwise the new item is inserted at the end.

For example, the following will insert a value into the array collection property Array using the key `primary`:

XML

```
<assign name='Ins' property='context.Array'
        action='insert'
        key='primary'
        value='request.Primary' />
```

The remove Operation

This applies to collection properties only. The `remove` operation removes an item from the specified collection property. The value attribute is ignored, but since the BPL schema for the <assign> element requires it, a value attribute must be present in the statement.

If the target property is an array collection, then the value of the key attribute specifies an item in the array, otherwise the key attribute is ignored.

For example, the following will remove the element with key `abc` from the array property Array:

XML

```
<assign name='Remove' property='context.Array' action='remove'
        key='abc' value='' />
```

Using <assign> to Set the status Variable

status is a business process execution context variable of type %Status that indicates success or failure.

Note: Error handling for a BPL business process happens automatically, without your ever needing to test or set the *status* value in the BPL source code. The *status* value is documented here in case you need to trigger a BPL business process to exit under certain special conditions.

When a BPL business process starts up, *status* is automatically assigned a value indicating success. To test that *status* has a success value, you can use the macro `$$$ISOK(status)` in ObjectScript. If the test returns a True value, *status* has a success value.

As the BPL business process runs, if at any time *status* acquires a failure value, InterSystems IRIS immediately terminates the business process and writes the corresponding text message to the Event Log. This happens regardless of how *status* acquired the failure value. Thus, the best way to cause a BPL business process to exit immediately, but gracefully is to set *status* to a failure value.

You can use an <assign> element to set *status* to a failure value. The usual convention for doing this is to use an <if> element to test the result of some prior activity, and then within the <true> or <false> element, use <assign> to set *status* to a failure value when failure conditions exist.

status is available to a BPL business process anywhere inside the <process>. You can refer to *status* with the same syntax as for any variable of the %Status type, that is: `status`

See Also

- <call>
- <context>

BPL <branch>

Conditionally causes an immediate change in the flow of execution, within a BPL business process.

Syntax

```
<branch condition="myVar='1'" label="JumpToMe" />
```

Attributes and Elements

condition attribute

Required. An expression that, if true, causes the flow of control to jump to the identified <label>.

Specify an expression that evaluates to the integer value 1 (if true) or 0 (if false).

LanguageOverride attribute

Optional. Specifies the scripting language in which any expressions (within in this element) are written.

Can be `"python"`, `"objectscript"`, or `"basic"` (not documented). Default is the language specified in the containing <process> element.

label attribute

Required. The name of the <label> to jump to. Specify a string of up to 255 characters.

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Description

The <branch> element causes an immediate change in the flow of execution if the value of its *condition* expression is true. Control passes to the <label> element whose name is specified as the value of the *label* attribute in the <branch>.

In the following BPL example, if the *condition* expression is true, the flow of control shifts directly from the <branch> with the *label* value TraceSkipped to the <label> with the *name* value TraceSkipped, while the intervening <trace> element is ignored:

```
<branch condition="myVar='1'" label="TraceSkipped" />
<trace value="Ignore me when myVar is 1..." />
<label name="TraceSkipped" />
```

If the <branch> *condition* expression is false, control simply passes to the next BPL statement following the <branch>, in this case <trace>.

A destination <label> must be in the same scope as the <branch> that references it. Thus:

- Each <sequence> element within a <flow> has its own <label> scope. The BPL execution engine prevents any attempt to <branch> to a <label> outside the current <sequence> container.
- There are similar restrictions on any other BPL container element that controls the flow of execution at runtime. Each container has its own <label> scope.

In addition to these restrictions, each <label> *name* value must be unique across the entire BPL business process, not just within the current scope.

CAUTION: As is true in all programming languages, the BPL branch mechanism must be used with care. The BPL editor does not prevent basic programming mistakes such as infinite loops or invalid branch cases.

BPL <break>

Breaks out of a loop and exits the loop activity, within a BPL business process.

Syntax

```
<break/>
```

Attributes and Elements

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Description

BPL syntax permits any element that can contain a sequence of activities—<case>, <default>, <foreach>, <false>, <sequence>, <true>, <until>, or <while>—to contain a <break> element if desired.

The <break> element allows the flow of control to exit a loop immediately without completing any more of the operations inside the containing loop. For example:

XML

```
<while condition="0">

  //...do various things...

  <if condition="somecondition">
    <true>
      <break/>
    </true>
  </if>

  //...do various other things...

</while>
```

In the above example, it is the <true> element that contains the <break> element. However, the loop affected by this <break> is actually the containing <while> loop.

The example works as follows: If on some pass through this loop, the <if> element finds “somecondition” to be true (that is, equal to the integer value 1) then the flow of control passes to the <true> element inside the <if>. Upon encountering the <break> element, execution immediately exits the containing <while> loop and proceeds to the next statement following the </while>.

Loop activities that you might want to modify by using a <break> element include <foreach>, <until>, and <while>.

Note: BPL business process code can initiate a immediate, but graceful exit by setting the business process execution context variable *status* to a failure value using an <assign> or <code> statement.

See Also

- <continue>

BPL <call>

Sends a request to a business operation or to another business process, as a step in a BPL business process.

Syntax

```
<call name="Call" target="MyApp.MyOperation" async="1">
   <request type="MyApp.Request">
     ...
   </request>
   <response type="MyApp.Response">
     ...
   </response>
</call>
```

Attributes and Elements

name attribute

Required. The name of the <call> element; provide a literal string, or by using the @ indirection operator to refer to the value of an execution context variable variable. If you wish to use a <sync> element to retrieve responses from asynchronous calls, refer to them using this *name*.

Specify a string of up to 255 characters.

target attribute

Required. The configured name of the business operation or business process to which the request is being sent. Provide this value as a literal string, or by using the ObjectScript @ indirection operator to refer to the value of an execution context variable.

async attribute

Required. Specifies the type of request to make. If 1 (true), the request is asynchronous. If 0 (false), the request is synchronous. Specify 1 (true) or 0 (false).

timeout attribute

Optional. Sets a timeout on a synchronous call. The *timeout* value is used only when the *async* attribute of the <call> is set to 0 (false). Specifies the time, in seconds, to wait for the response, as an expression that evaluates to an XML xsd:dateTime value.

For example `2023:10:19T10:10`

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

<request> element

Required. Specifies the type (class name) of the request to send.

<response> element

Optional. Specifies the type (class name) of the response to return. If omitted, no response is returned from this <call>.

Description

The <call> element sends a request (synchronously or asynchronously) to a business operation or business process. The <call> element has a required attribute, *async*, that determines how the request is made:

- If *async* is 0 (or False), the request is made synchronously; the business process waits until it receives a response before continuing execution.

  Important: A <call> element with `async='False'` and a <response> block defined suspends execution of its business process thread until the called operation completes.

  A <call> element with `async='False'` but with *no* <response> block defined behaves as if `async='True'`. If you want to send a synchronous request but do not require a response, create a non-functioning <response> block so that the <call> waits for the target host to finish before continuing execution.

- If *async* is 1 (or True), the request is made asynchronously; the business process continues to execute after making the request. The business process can later receive the responses from several asynchronous calls by providing a <sync> element that specifies a list of the <call> elements for which it is waiting. For details, see <sync>.

The <call> element has the child elements <request> and <response> which identify the class of request and response objects to use in making the call. Either element can contain one or more <assign> elements. In the <request> element, <assign> elements are used to fill in the properties of the request object used for the call. The <response> element uses <assign> elements when it needs to move the properties of the resulting response object to a new location, such as the *context* or *response* variables in the business process execution context.

Note: There is detailed information about the business process execution context in documentation of the <assign> element. Also see Business Process Execution Context.

In the case of an asynchronous request, the <assign> elements within the body of the <response> element are executed when the corresponding request is received. There is no guarantee when this will occur, so a business process will typically use the <sync> element to wait for an asynchronous response. Note that if a response is not received within the *timeout* period specified by the <sync> element, then the assignments defined by the corresponding <response> block will not be executed, and the response itself will be marked with a status of Discarded.

If the call is synchronous, an optional timeout can be specified using the *timeout* attribute on the <call> element itself. This attribute cannot be used for asynchronous calls. If the <call> element has *async* set to 1 (true) then the only way to set a timeout period is to use the *timeout* attribute on the <sync> element that is being used to collect the asynchronous response(s).

The following example sends an synchronous Ens.StringRequest request to the Get Weather Report business operation:

```
<call name='Get Weather Report' target='Get Weather Report' async='0' >
   <request type='Ens.StringRequest' >
     <assign property="callrequest.StringValue" value="context.Location" action="set" />
   </request>
   <response type='Demo.Service.Msg.WeatherOperationResponse' >
      <assign property="context.OperationReport" value="callresponse" action="set" />
   </response>
</call>
```

The following example uses the <call> element to send an asynchronous MyApp.SalaryRequest request to the MyApp.PayrollApp business operation:

XML

```
<call name="FindSalary" target="MyApp.PayrollApp" async="1">
  <request type="MyApp.SalaryRequest">
    <assign property="callrequest.Name" value="request.Name" />
    <assign property="callrequest.SSN" value="request.SSN" />
  </request>
  <response type="MyApp.SalaryResponse">
    <assign property="context.Salary" value="callresponse.Salary" />
  </response>
</call>
```

Whenever a <call> element is executed, the BPL engine inserts the *name* of the <call> element into the message header so that it is visible in later Message Browser and Visual Trace displays.

Use of the <assign> Element

The above example includes <assign> elements that manipulate properties in the variables in the business process execution context such as *context*, *request*, *callrequest*, and *callresponse*. While many details concerning these variables are found in the documentation for the <assign> element, the following table describes the execution context variables as they relate to the <call> activity:

The <call> element can refer to the following variables and their properties. Do not use variables not listed here.

Variable	Purpose
callrequest	A <call> element contains a <request> element that identifies the type of message that will be sent to the target. If this message type has input parameters, the <request> element must provide <assign> elements that assign values to properties in the *callrequest* object. These properties must match the input parameters for the message type. After the <request> completes, the *callrequest* object goes out of scope.
callresponse	If the request message type has a corresponding response message type, the <call> element contains a <response> element. When the response arrives, control passes to the <response> element. The output parameters from the response message become properties of the *callresponse* object. Since *callresponse* only has meaning inside the <response> element, to preserve these values the <response> element must provide <assign> elements that assign *callresponse* values to properties of other, more permanent objects in the business process execution context, usually *context* or *response*.
context	Throughout the business process, the *context* object serves as a general-purpose container for any business process data that needs to be persistent.
request	Throughout the business process, the *request* object contains the original properties that were sent to the business process as parameters of the request that instantiated it.
response	The *response* object retains its scope throughout the business process. It contains the properties that are expected to be returned to the caller as output parameters of this business process. Whatever is inside the *response* object, when a business process completes or exits, will be interpreted as the return values of the business process.
status	*status* is a %Status value that indicates success or failure. When a BPL business process starts up, *status* is automatically assigned a value indicating success. As the BPL business process runs, if at any time *status* acquires a failure value, the business process immediately exits and writes the corresponding text message to the Event Log. *status* automatically receives the returned %Status value returned from any <call> activity, without any special statements in the BPL code. Thus, if any <call> activity fails, the BPL business process immediately exits and writes an Event Log entry.

Variable	Purpose
syncresponses	*syncresponses* is a collection of response objects, keyed by the names of the <call> activities being synchronized. Only completed calls are represented. You can retrieve a response from *syncresponses* only after a <sync> and before the end of the current <sequence>. Do so using the syntax `syncresponses.GetAt("MyName")` where the relevant call was defined as `<call name="MyName">`
synctimedout	The *synctimedout* value is an integer. *synctimedout* indicates the outcome of a <sync> activity after several calls. You can test the value of *synctimedout* after the <sync> and before the end of the <sequence> that contains the calls and <sync>. *synctimedout* has one of three values: • If 0, no call timed out. All the calls had time to complete. This is also the value if the <sync> activity had no *timeout* set. • If 1, at least one call timed out. This means not all <call> activities completed before the timeout. • If 2, at least one call was interrupted before it could complete. Generally you test *synctimedout* for status and then retrieve the responses from completed calls out of the *syncresponses* collection.

CAUTION: Like all other execution context variable names, *status* is a reserved word in BPL; do not use it except as described in this table.

Indirection in the name or target Attributes (Accessing Execution Context Variables)

The values of the *name* or *target* attributes are strings. The *name* identifies the call and may be referenced in a later <sync> element. The *target* is the configured name of the business operation or business process to which the request is being sent. Either of these strings can be a literal value:

```
<call name="Call" target="MyApp.MyOperation" async="1">
```

Or the @ indirection operator can be used to access the value of an execution context variable that contains the appropriate string. This example accesses the value of the `nextCallName` and `nextBusinessHost` properties of the `context` object.

```
<call name="@context.nextCallName" target="@context.nextBusinessHost" async="1">
```

Using Multiple Asynchronous <calls> in a Loop, Followed by a <sync>

This section describes how to use multiple asynchronous <calls> in a loop, followed by a <sync>.

When a BPL makes a <call> it makes note of the name of the call; in the <sync>, you must specify that same name to designate which pending request to wait for. In some scenarios, you have multiple asynchronous calls in a loop, as in this example:

```
<sequence>
  <while condition='...'>
      <call name="A" async="1" />
   </while>
  ...
  <sync calls="A" type="all" timeout="3600"/>
</sequence>
```

Because the BPL tracks which call to wait for by the call name, the sync completes as soon as the *first* response comes in. If you want the sync to wait until all such calls are completed, it is necessary to generate a set of unique call names and then use that list of names. Here is a way to do so:

1. Create a context variable containing a string which changes for each call by adding a numeric iterator (`i` in the example below). Before the call, initialize this variable as in the following example:

   ```
   set context.callname = "A" _ context.i
   ```

2. Set the `Name` of the <call> equal to this variable.
3. Create a string containing all the <call> names, comma-separated, i.e.: `"A1,A2,A3,A4,A5"`. Save that in a separate variable, `context.allCallNames`, in the example below.
4. Set the `calls` attribute of the <sync> equal to the variable containing the list of calls.

```
<sequence>
  <while condition='...'>
      .... code here to set up callname and allCallNames ...
      <call name="@context.callname" async="1" />
   </while>
  ...
  <sync calls="@context.allCallNames" type="all" timeout="3600"/>
</sequence>
```

See Also

- <assign>
- <code>
- <reply>
- <sequence>
- <sync>

BPL <case>

Performs a set of activities when a condition is matched within a <switch> element, as a step in a BPL business process.

Syntax

```
<switch>
   <case>
     ...
   </case>
    ...
   <default>
     ...
   </default>
</switch>
```

Attributes and Elements

condition attribute

Required. If this expression evaluates to true, the contents of this <case> element are executed. If false, this <case> is ignored.

Specify an expression that evaluates to the integer value 1 (if true) or 0 (if false).

LanguageOverride attribute

Optional. Specifies the scripting language in which any expressions (within in this element) are written.

Can be `"python"`, `"objectscript"`, or `"basic"` (not documented). Default is the language specified in the containing <process> element.

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Other elements

Optional. <case> may contain zero or more of the following elements in any combination: <alert>, <assign>, <branch>, <break>, <call>, <code>, <continue>, <delay>, <empty>, <flow>, <foreach>, <if>, <label>, <milestone>, <reply>, <rule>, <scope>, <sequence>, <sql>, <switch>, <sync>, <throw>, <trace>, <transform>, <until>, <while>, <xpath>, or <xslt>.

Description

A <case> element is used within <switch>.

A <switch> element contains a sequence of one or more <case> elements and an optional <default> element.

When a <switch> element is executed, it evaluates each <case> condition in turn. These conditions are logical expressions in the scripting language of the containing <process> element. If any expression evaluates to the integer value 1 (true), then the contents of the corresponding <case> element are executed; otherwise, the expression for the next <case> element is evaluated.

If no <case> condition is true, the contents of the <default> element are executed.

As soon as one of <case> elements is executed, execution control leaves the surrounding <switch> statement. If no <case> condition matches, control leaves the <switch> after the <default> activity executes.

Activities within a <case> element can be any BPL activity, including <assign> elements as in the example below:

XML

```
<switch name='Approved?'>
  <case name='No PrimeRate' condition='context.PrimeRate=""'>
    <assign name='Not Approved' property="response.IsApproved" value="0"/>
  </case>
  <case name='No Credit' condition='context.CreditRating=""'>
    <assign name='Not Approved' property="response.IsApproved" value="0"/>
  </case>
  <default name='Approved' >
    <assign name='Approved' property="response.IsApproved" value="1"/>
    <assign name='InterestRate'
            property="response.InterestRate"
            value="context.PrimeRate+10+(99*(1-(context.CreditRating/100)))">
      <annotation>
        <![CDATA[Copy InterestRate into response object.]]>
      </annotation>
    </assign>
  </default>
</switch>
```

See Also

- <switch>
- <default>

BPL <catch>

Catches a fault produced by a <throw> element, as a step in a BPL business process.

Syntax

```
<scope>
   <throw fault='"MyFault"'/>
         ...
   <faulthandlers>
      <catch fault='"MyFault"'>
            ...
     </catch>
   </faulthandlers>
</scope>
```

Attributes and Elements

fault attribute

Required. The name of the fault. It can be a literal text string (up to 255 characters) or an expression to be evaluated.

LanguageOverride attribute

Optional. Specifies the scripting language in which any expressions (within in this element) are written.

Can be `"python"`, `"objectscript"`, or `"basic"` (not documented). Default is the language specified in the containing <process> element.

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Other elements

Optional. <catch> may contain zero or more of the following elements in any combination: <alert>, <assign>, <branch>, <break>, <call>, <code>, <compensate>, <continue>, <delay>, <empty>, <foreach>, <if>, <label>, <milestone>, <reply>, <rule>, <scope>, <sequence>, <sql>, <switch>, <sync>, <throw>, <trace>, <transform>, <until>, <while>, <xpath>, or <xslt>.

Description

When a <throw> statement executes, control immediately shifts to the <faulthandlers> block inside the same <scope>, skipping all intervening statements after the <throw>. Inside the <faulthandlers> block, the program attempts to find a <catch> block whose *value* attribute matches the *fault* string expression in the <throw> statement. This comparison is case-sensitive. When you specify a fault string it *needs* the extra set of quotes to contain it, as shown below:

XML

```
<catch fault='"thrown"'/>
```

If there is a <catch> block that matches the fault, the program executes the code within this <catch> block and then exits the <scope>. The program resumes execution at the next statement following the closing </scope> element.

If a fault is thrown, and the corresponding <faulthandlers> block contains *no* <catch> block that matches the fault string, control shifts from the <throw> statement to the <catchall> block inside <faulthandlers>. After executing the contents of

the <catchall> block, the program exits the <scope>. The program resumes execution at the next statement following the closing </scope> element. It is good programming practice to ensure that there is always a <catchall> block inside every <faulthandlers> block, to ensure that the program catches any unanticipated errors.

For details, see Handling Errors in BPL.

Note: If a <catchall> is provided, it must be the last statement in the <faulthandlers> block. All <catch> blocks must appear before <catchall>.

See Also

- <catchall>
- <compensate>
- <compensationhandlers>
- <faulthandlers>
- <scope>
- <throw>

BPL <catchall>

Catches a fault or system error that does not match any <catch>, as a step in a BPL business process.

Syntax

```
<scope>
   <throw fault='"MyFault"'/>
          ...
     <faulthandlers>
         <catch fault='"MyFault"'>
          ...
         </catch>
        <catch fault='"OtherFault"'>
           ...
        </catch>
        <catchall>
           ...
        </catchall>
     </faulthandlers>
</scope>
```

Attributes and Elements

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Other elements

Optional. <catchall> may contain zero or more of the following elements in any combination: <alert>, <assign>, <branch>, <break>, <call>, <code>, <compensate>, <continue>, <delay>, <empty>, <foreach>, <if>, <label>, <milestone>, <reply>, <rule>, <scope>, <sequence>, <sql>, <switch>, <sync>, <throw>, <trace>, <transform>, <until>, <while>, <xpath>, or <xslt>.

Description

A <catchall> element is used within a <faulthandlers> element.

When a <throw> statement executes, control immediately shifts to the <faulthandlers> block inside the same <scope>, skipping all intervening statements after the <throw>. Inside the <faulthandlers> block, the program attempts to find a <catch> block whose *value* attribute matches the *fault* string expression in the <throw> statement. If it finds one, the program executes the code within this <catch> block and then exits the <scope>. The program resumes execution at the next statement following the closing </scope> element.

If a fault is thrown, and the corresponding <faulthandlers> block contains *no* <catch> block that matches the fault string, control shifts from the <throw> statement to the <catchall> block inside <faulthandlers>. After executing the contents of the <catchall> block, the program exits the <scope>. The program resumes execution at the next statement following the closing </scope> element. It is good programming practice to ensure that there is always a <catchall> block inside every <faulthandlers> block, to ensure that the program catches any unanticipated errors.

For details, see Handling Errors in BPL.

Important: When you use this error handling system with <call> statements that communicate with other business hosts, make sure that the target business hosts return an error status in the case of an error. If the target component returns success even in the case of an error, the BPL process will not trigger <catchall> logic.

Note: If a <catchall> is provided, it must be the last statement in the <faulthandlers> block. All <catch> blocks must appear before <catchall>.

See Also

<catch>, <compensate>, <compensationhandlers>, <faulthandlers>, <scope>, and <throw>.

BPL <code>

Executes one or more lines of custom code, as a step in a BPL business process.

Syntax

```
<code name='CodeWrittenInBasic'>
   <![CDATA['invoke custom method "MyApp.MyClass".Method(context.Value)   ]]>
</code>
```

Attributes and Elements

LanguageOverride attribute

Optional. Specifies the scripting language in which the code in this element is written.

Can be `"python"`, `"objectscript"`, or `"basic"` (not documented). Default is the language specified in the containing <process> element.

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Description

The BPL <code> element executes one or more lines of user-written code within a BPL business process. You can use the <code> element to perform special tasks that are difficult to express using the BPL elements. Any properties referenced by the <code> element must be properties within the business process execution context.

The scripting language for a BPL <code> element is specified by the language attribute of the containing <process> element. This should be `objectscript`. For further information, see:

- Using ObjectScript
- ObjectScript Reference

Typically a developer wraps the contents of a <code> element within a `CDATA` block so that it is not necessary to escape special XML characters such as the apostrophe (') or the ampersand (&). For example:

XML

```
<code name="MyCode" language="objectscript">
  <![CDATA[ callrequest.Name = request.FirstName & " " & request.LastName]]>
</code>
```

To ensure you can properly suspend and restore execution of a business process, follow these guidelines when using the <code> element:

- The execution time should be short; custom code should not tie up the general execution of the business process.
- Do not allocate any system resources (such as taking out locks or opening devices) without releasing them within the same <code> element.
- If a <code> element starts a transaction, make sure that the same <code> element ends the transactions in *all possible scenarios*; otherwise, the transaction can be left open indefinitely. This could prevent other processing or can cause significant downtime.

- Do not rely on variables that are not part of the business process execution context. InterSystems IRIS automatically restores the contents of the execution context whenever a business process is suspended and later resumed; any other variables will be cleaned up.

Also, InterSystems strongly recommends that instead of including multiple lines of code within <code>, you invoke a class method or a routine that contains the needed code. This approach makes it far easier to test and debug your processing.

Available Variables

The <code> element can refer to the following execution context variables and their properties. Do not use variables not listed here.

Variable	Purpose
context	The *context* object is a general-purpose data container for the business process. *context* has no automatic definition. To define the properties of this object, use the <context> element. That done, you may refer to these properties anywhere inside the <process> element using dot syntax, as in: `context.Balance`
request	The *request* object contains any properties of the original request message object that caused this business process to be instantiated. You may refer to *request* properties anywhere inside the <process> element using dot syntax, as in: `request.UserID`
response	The *response* object contains any properties that are required to build the final response message object to be returned by the business process. You may refer to *response* properties anywhere inside the <process> element using dot syntax, as in: `response.IsApproved`. Use the <assign> element to assign values to these properties.
status	*status* is a value of type %Status that indicates success or failure. When a BPL business process starts up, *status* is automatically assigned a value indicating success. As the BPL business process runs, if at any time *status* acquires a failure value, InterSystems IRIS immediately terminates the business process and writes the corresponding text message to the Event Log. In general, this happens automatically, when unsuccessful values are returned from <call> activities. However, BPL business process code can initiate a immediate, but graceful exit by setting the *status* value using <assign> or <code>. See the description at the end of this topic.
process	The *process* object represents the current instance of the BPL business process object (an instance of the BPL class). This object has one property for each property defined in that class. You can invoke methods of the *process* object; for example: `process.SendRequestSync()`

CAUTION: Like all other execution context variable names, *status* is a reserved word in BPL. Do not use it in <code> blocks except to cause the <code> block to exit.

Using <code> to Set the status Variable

status is a business process execution context variable of type %Status that indicates success or failure.

Note: Error handling for a BPL business process happens automatically, without your ever needing to test or set the *status* value in the BPL source code. The *status* value is documented here in case you need to trigger a BPL business process to exit under certain special conditions.

When a BPL business process starts up, *status* is automatically assigned a value indicating success. To test that *status* has a success value, you can use the macro `$$$ISOK(status)` in ObjectScript. If the test returns a True value, *status* has a success value.

As the BPL business process runs, if at any time *status* acquires a failure value, InterSystems IRIS immediately terminates the business process and writes the corresponding text message to the Event Log. This happens regardless of how *status* acquired the failure value. Thus, the best way to cause a BPL business process to exit immediately, but gracefully is to set *status* to a failure value.

Statements within a <code> activity can set *status* to a failure value. The BPL business process does not perceive the change in the value of *status* until the <code> activity has fully completed. Therefore, if you want a failure *status* to cause an immediate exit from a <code> activity, you must place a quit command in the <code> activity immediately after setting a failure value for *status*.

status is available to a BPL business process anywhere inside the <process>. You can refer to *status* with the same syntax as for any variable of the %Status type, that is: `status`

See Also

- <call>
- <sql>

BPL compensate>

Invokes a <compensationhandler> from <catch> or <catchall>, as a step in a BPL business process.

Syntax

```
<scope>
   <throw fault='"BuyersRegret"'/>
   <compensationhandlers>
     <compensationhandler name="RestoreBalance">
       <assign property='context.MyBalance' value='context.MyBalance+1'/>
     </compensationhandler>
   </compensationhandlers>
   <faulthandlers>
     <catch fault='"BuyersRegret"'>
       <compensate target="RestoreBalance"/>
     </catch>
   </faulthandlers>
</scope>
```

Attributes and Elements

target attribute

Required. The name of a <compensationhandler> that provides a sequence of activities to undo previous actions.

Specify a string of up to 255 characters.

<annotation> element

See Common Attributes and Elements.

Description

The <compensate> element invokes a <compensationhandler> block by specifying its name as a target:

XML

```
<compensate target="general"/>
```

<compensate> may only appear within <catch> or <catchall>. Its *target* value must match the *name* of a <compensation-handler> within the same BPL business process.

For details, see Handling Errors in BPL.

BPL <compensationhandlers> and <compensationhandler>

Provides compensation handlers, each of which performs a sequence of activities to undo a previous action, as a step in a BPL business process.

Syntax

```
<scope>
   <throw fault='"BuyersRegret"'/>
   <compensationhandlers>
     <compensationhandler name="RestoreBalance">
       <assign property='context.MyBalance' value='context.MyBalance+1'/>
     </compensationhandler>
   </compensationhandlers>
   <faulthandlers>
     <catch fault='"BuyersRegret"'>
       <compensate target="RestoreBalance"/>
     </catch>
   </faulthandlers>
 </scope>
```

Elements

<compensationhandler>

Zero or more <compensationhandler> elements may appear inside the <compensationhandlers> container. Each <compensationhandler> element contains a specific sequence of BPL activities that undo a previous action.

In turn, a <compensationhandler> has all the common attributes and elements

Description

In business process management, it is often necessary to reverse some segment of logic. This convention is known as "compensation." The ruling principle is that if the business process does something, it must be able to undo it. That is, if a failure occurs, the business process must be able to compensate by undoing the action that failed. You need to be able to unroll all of the actions from that failure point back to the beginning, as if the problem action never occurred. BPL enables this with a mechanism called a compensation handler.

BPL <compensationhandler> blocks are somewhat like subroutines, but they do not provide a generalized subroutine mechanism. You can "call" them, but *only* from <faulthandler> blocks, and *only* within the same <scope> as the <compensationhandler> block. The <compensate> element invokes a <compensationhandler> block by specifying its name as a target. Extra quotes are *not* needed for this syntax:

XML

```
<compensate target="general"/>
```

Compensation handlers are only useful if you *can* undo the actions already performed. For example, if you transfer money into the wrong account, you can transfer it back again, but there are some actions that cannot be neatly undone. You must plan compensation handlers accordingly, and also organize them according to how far you want to roll things back.

For details, see Handling Errors in BPL.

Note: It is not possible to reverse the order of <compensationhandlers> and <faulthandlers>. If both blocks are provided, <compensationhandlers> must appear first and <faulthandlers> second.

See Also

- <catch>

- <catchall>
- <compensate>
- <faulthandlers>
- <scope>
- <throw>

BPL <context>

Defines one or more properties in the business process execution context, for use in a BPL business process.

Syntax

```
<context>
    <property name="P1" type="%String" />
    <property name="P2" type="%String" />
    ...
</context>
```

Elements

<property> element

Optional. Zero or more <property> elements may appear. Each defines one property of the business process execution context.

Description

The life cycle of a business process requires it to have certain state information saved to disk and restored from disk, whenever the business process suspends or resumes execution. A BPL business process supports the business process life cycle with a group of variables known as the *execution context*.

The execution context variables include the objects called *context*, *request*, *response*, *callrequest*, *callresponse* and *process*; the integer value *synctimedout*; the collection *syncresponses*; and the %Status value *status*. Each variable has a specific purpose, as described in documentation for the <assign>, <call>, <code>, and <sync> elements.

Most of the execution context variables are automatically defined for the business process. The exception to this rule is the general-purpose container object called *context*, which a BPL developer must define. Any value that you want to be persistent and available everywhere within the business process should be declared as a property of the *context* object. You can do this by providing <context> and <property> elements at the beginning of the BPL document, as follows. The resulting BPL code is the same whether you use the Business Process Designer or type the code directly into the BPL document:

- When using the Business Process Designer, you can add properties of various types to the *context* object from the **Context** tab to the right of the BPL diagram. Add whatever properties you need by clicking the plus-sign next to **Context properties**. You can also edit or delete a property using the icons next to its name. The appropriate <context> and <property> elements appear in the generated BPL for the business process.
- You can add <context> and <property> elements together at the beginning of the <process> element, as shown in the following example.

XML

```
<process request="Demo.Loan.Msg.Application" response="Demo.Loan.Msg.Approval">
  <context>
    <property name="BankName" type="%String"
              initialexpression="BankOfMomAndDad" />
    <property name="IsApproved" type="%Boolean"/>
    <property name="InterestRate" type="%Numeric"/>
    <property name="TheResults"
              type="Demo.Loan.Msg.Approval"
              collection="list"/>
    <property name="Iterator" type="%String"/>
    <property name="ThisResult" type="Demo.Loan.Msg.Approval"/>
  </context>
  ...
</process>
```

Each <property> element defines the name and data type for a property. For a list of available data type classes, see Parameters. You may assign an initial value in the <property> element by providing an *initialexpression* attribute. Alternatively, you may assign values during business process execution, using the <assign> element.

BPL <continue>

Jumps to the next iteration within a loop, without exiting the loop, within a BPL business process.

Syntax

```
<continue/>
```

Attributes and Elements

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Description

BPL syntax permits any element that can contain a sequence of activities—<case>, <default>, <foreach>, <false>, <sequence>, <true>, <until>, or <while>—to contain a <continue> element if desired.

The <continue> element allows the flow of control to jump to the next iteration of a loop, without completing the remaining operations inside the current iteration. For example:

XML

```
<foreach property="P1" key="K1">

  //...do various things...

  <if condition="somecondition">
    <true>
      <continue/>
    </true>
  </if>

  //...do various other things...

</foreach>
```

In the above example, it is the <true> element that contains the <continue> element. However, the loop affected by this <continue> is actually the containing <foreach> loop.

The example works as follows: If on some pass through this loop, the <if> element finds “somecondition” to be true (that is, equal to the integer value 1) then the flow of control passes to the <true> element inside the <if>. Upon encountering the <continue> element, execution halts the current pass through the <foreach> loop, proceeds to the next item in the collection (if there is a next item), and begins processing that next item from the beginning of the loop.

Loop activities that you might want to modify by using a <continue> element include <foreach>, <until>, and <while>. The effect of <continue> for each type of loop element is to halt the current pass through the loop, jump to the condition test for the loop, and allow that test and the type of loop to determine what to do next: continue looping, or exit the loop, as normal for that type of loop. For example:

Containing Loop	Behavior of <continue>
<foreach>	Test for the next item in the collection. If an item is found, begin processing it from the top of the loop. However, if there are no more items in the collection that match the test condition, exit the loop.
<until>	Jump to the condition test at the bottom of the loop. If the condition is true, exit the loop; if false, execute the statements in the loop.
<while>	Jump to the condition test at the top of the loop. If the condition is true, exit the loop; if false, execute the statements in the loop.

See Also

- <break>

BPL <default>

Performs a set of activities when no matching condition can be found within a <switch> element, as a step in a BPL business process.

Syntax

```
<switch>
   <case>
     ...
   </case>
   ...
   <default>
     ...
   </default>
</switch>
```

Attributes and Elements

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Other elements

Optional. <default> may contain zero or more of the following elements in any combination: <alert>, <assign>, <branch>, <break>, <call>, <code>, <continue>, <delay>, <empty>, <flow>, <foreach>, <if>, <label>, <milestone>, <reply>, <rule>, <scope>, <sequence>, <sql>, <switch>, <sync>, <throw>, <trace>, <transform>, <until>, <while>, <xpath>, or <xslt>.

Description

A <default> element is an optional part of a <switch> element. A <switch> element contains a sequence of one or more <case> elements and an optional <default> element.

When present, the <default> element must be the last element in the <switch>. Correspondingly, in the Business Process Designer, the <default> element must be the right-most option in the <switch> part of the diagram.

When a <switch> element is executed, it evaluates each <case> condition in turn. These conditions are logical expressions in the scripting language of the containing <process> element. If any expression evaluates to the integer value 1 (true), then the contents of the corresponding <case> element are executed; otherwise the expression for the next <case> element is evaluated.

If no <case> condition is true, the contents of the <default> element are executed.

Activities within a <default> element can be any BPL activity listed above, including <assign> elements as in the example below:

XML

```
<switch name='Approved?'>
  <case name='No PrimeRate' condition='context.PrimeRate=""'>
    <assign name='Not Approved' property="response.IsApproved" value="0"/>
  </case>
  <case name='No Credit' condition='context.CreditRating=""'>
    <assign name='Not Approved' property="response.IsApproved" value="0"/>
  </case>
  <default name='Approved' >
    <assign name='Approved' property="response.IsApproved" value="1"/>
    <assign name='InterestRate'
            property="response.InterestRate"
            value="context.PrimeRate+10+(99*(1-(context.CreditRating/100)))">
      <annotation>
        <![CDATA[Copy InterestRate into response object.]]>
      </annotation>
    </assign>
  </default>
</switch>
```

BPL <delay>

Delays execution of a business process for a specified duration or until a future time, as a step in a BPL business process.

Syntax

```
<delay duration='"PT60S"'/>
```

Or:

```
<delay until='"2020-10-19T10:10"'/>
```

Attributes and Elements

duration attribute

Optional. Specifies the duration of the delay as an expression that evaluates to an XML `duration` value.

For example: `PT60S` for 60 seconds or `P1Y2M3DT10H30M` for 1 year, 2 months, 3 days, 10 hours, and 30 minutes. The <delay> element ignores fractional seconds. If duration has a value less than one second, it is treated as 0 seconds.

For details on XML duration values, see appropriate entry in the Primitive Datatypes section of the W3C Recommendation XML Schema Part 2: Datatypes Second Edition, which you can view at the following:

- https://www.w3.org/TR/xmlschema-2/#duration
- https://www.w3.org/TR/xmlschema-2/#dateTime

until attribute

Optional. Specifies a future time at which the delay will expire, as an expression that evaluates to an XML `dateTime` value.*

For example `2023:10:19T10:10`

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Description

The <delay> element suspends execution of a business process (or the current thread within a <flow>) for either a specified duration or until a specific time. For example:

XML

```
<sequence>
  <annotation>
    <![CDATA[ Write the time now, and sixty seconds later.]]>
  </annotation>
  <trace value='"The time is: "_$ZDATETIME($H,3)' />
  <delay duration='"PT60S"' />
  <trace value='"The time is: "_$ZDATETIME($H,3)' />
</sequence>
```

The <delay> element causes the execution of a business process to pause for either a specific duration (specified by the *duration* attribute) or until a specific future time (specified by the *until* attribute). You must provide either the *duration* attribute or the until attribute, or no delay will take place.

During the delay period, execution of the current business process thread is suspended and the state of the business process is saved to the database.

The format for values of *duration* and *until* is discussed at length in World Wide Web Consortium documents about XML data types. For details, see the "Primitive Datatypes" section of the W3C Recommendation XML Schema Part 2: Datatypes Second Edition, which you can view at https://www.w3.org/TR/xmlschema-2/#built-in-primitive-datatypes. Some *duration* examples are:

- PT60S or PT1M for one minute
- PT219S or PT3M39S for 3 minutes, 39 seconds

Whenever a <delay> element is executed, the BPL engine inserts the *name* of the <delay> element into the message header so that it is visible in later Message Browser and Visual Trace displays.

BPL <empty>

Performs no action, as a placeholder step in a BPL business process.

Syntax

```
<empty />
```

Attributes and Elements

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Description

The <empty> element performs no operation. Its purpose is to serve as a placeholder within a BPL definition or as a place to hold additional annotation without affecting the execution of the business process. For example.

XML

```
<empty>
  <annotation>This is an empty element.
  </annotation>
</empty>
```

BPL <false>

Performs a set of activities when the condition for an <if> element is false, as a step in a BPL business process.

Syntax

```
<if condition="0">
   <true>
     ...
   </true>
   <false>
     ...
   </false>
</if>
```

Attributes and Elements

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Other elements

Optional. <false> may contain zero or more of the following elements in any combination: <alert>, <assign>, <branch>, <break>, <call>, <code>, <continue>, <delay>, <empty>, <flow>, <foreach>, <if>, <label>, <milestone>, <reply>, <rule>, <scope>, <sequence>, <sql>, <switch>, <sync>, <throw>, <trace>, <transform>, <until>, <while>, <xpath>, or <xslt>.

Description

A <false> element is used within an <if> to contain elements that need to be executed if the condition is false.

See Also

- <if>
- <true>

BPL <faulthandlers>

Provides zero or more <catch> and one <catchall> element to catch faults and system errors, as a step in a BPL business process.

Syntax

```
<scope>
   <throw fault='"MyFault"'/>
         ...
     <faulthandlers>
        <catch fault='"MyFault"'>
           ...
        </catch>
        <catch fault='"OtherFault"'>
           ...
        </catch>
        <catchall>
           ...
        </catchall>
   </faulthandlers>
</scope>
```

Attributes and Elements

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<catch> element

There may be zero or more <catch> elements inside <faulthandlers>. Each catches a specific, named fault produced by a <throw> element.

<catchall> element

Catch a fault or system error that does not match any <catch>. If there are no <catch> elements in <faulthandlers>, there must be a <catchall>. Otherwise, <catchall> is optional.

Description

To enable error handling, BPL defines an element called <scope>. A scope is a wrapper for a set of activities. This scope may contain one or more activities, one or more fault handlers, and zero or more compensation handlers. The <faulthandlers> element is intended to catch any errors that activities within the <scope> produce. The <catch> and <catchall> elements within <faulthandlers> may provide <compensate> statements that invoke <compensationhandler> elements to compensate for those errors.

When a <scope> provides no <faulthandlers> block, InterSystems IRIS automatically outputs the system error to the Event Log. When a <scope> *does* contain a <faulthandlers> block, the BPL business process must output <trace> messages to the Event Log for system error messages to appear there. System error messages appear in the ObjectScript shell, in either case.

For details, see Handling Errors in BPL.

Note: It is not possible to reverse the order of <compensationhandlers> and <faulthandlers>. If both blocks are provided, <compensationhandlers> must appear first and <faulthandlers> second.

See Also

- <catch>

- <catchall>
- <compensate>
- <compensationhandlers>
- <scope>
- <throw>

BPL <flow>

Performs a set of activities in a non-determinate order, within a BPL business process.

Syntax

```
<flow>
   <sequence name="thread1">
     ...
   </sequence>
   <sequence name="thread2">
     ...
   </sequence>
   ...
</flow>
```

Attributes and Elements

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

<sequence> element

Optional. Zero or more <sequence> elements contain whatever activities are needed for the <flow>. If no <sequence> elements are provided, no action is taken by the <flow>.

Description

The <flow> element specifies that each of the elements it contains are executed in a non-determinate order. A <flow> element contains one or more <sequence> elements, each of which is referred to as a *thread*.

When you are using the Business Process Designer and you add a <flow> element to the business process, a <sequence> element is automatically inserted inside the <flow>, as you can see by examining the generated BPL code.

If you need to temporarily disable one of the <sequence> elements within a <flow>, you can edit the generated BPL code by setting the *disabled* attribute of the corresponding <sequence> element.

The following abbreviated example shows the usage of the <flow> element. In this hand-coded BPL example, the developer has decided to use two parallel sequences inside the flow. Each is executed in a separate thread: `thread1` and `thread2`.

XML

```
<process>
  <flow>
    <sequence name="thread1">
      <call name="A" />
      <call name="B" />
    </sequence>
    <sequence name="thread2">
      <call name="C" />
      <call name="A" />
    </sequence>
  </flow>
  <call name="E" />
</process>
```

In this example, the <flow> element defines two threads, specified by <sequence> elements `thread1` and `thread2`. The order in which the two threads are executed is indeterminate (though, of course, the <call> elements *within* the <sequence> elements are executed in sequential order).

If possible, the execution of threads is interlaced. For example, if the execution of one thread is suspended (say it is waiting for a response from a asynchronous call), then execution of one of the other threads proceeds (if possible).

Note that, strictly speaking, the threads within a <flow> element do not execute at the same time: this is because only one thread is given access to the business process execution context at a time, to preserve proper concurrency and data consistency.

Note: For more information about the business process execution context, see <assign> , and see Developing BPL Processes.

The <flow> element waits for all of its threads to complete before it allows execution to continue. After both threads in the previous example are completed, execution continues and <call> element E is executed.

A thread within a <flow> element may contain additional, nested <flow> elements.

For information about using <sync> with <flow>, see documentation of the <sync> element.

BPL <foreach>

Defines a sequence of activities to be executed iteratively, within a BPL business process.

Syntax

```
<foreach property="P1" key="K1">
   ...
</foreach>
```

Attributes and Elements

property attribute

Required. The collection property (list or array) to iterate over. It must be the name of a valid object and property in the execution context.

key attribute

Required. The index used to iterate through the collection. It must be a name of a valid object and property in the execution context. It is assigned a value for each element in the collection.

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Other elements

Optional. <foreach> may contain zero or more of the following elements in any combination: <alert>, <assign>, <branch>, <break>, <call>, <code>, <continue>, <delay>, <empty>, <flow>, <foreach>, <if>, <label>, <milestone>, <reply>, <rule>, <scope>, <sequence>, <sql>, <switch>, <sync>, <throw>, <trace>, <transform>, <until>, <while>, <xpath>, or <xslt>.

Description

The <foreach> element defines a sequence of activities that are executed iteratively, once for every element within a specified collection property. For example:

XML

```
<foreach property="callrequest.Location" key="context.K1">
  <assign property="total"
          value="context.total+context.prices.GetAt(context.K1)"/>
</foreach>
```

The <foreach> element can refer to the following variables and their properties. Do not use variables not listed here.

Variable	Purpose
context	The *context* object is a general-purpose data container for the business process. *context* has no automatic definition. To define properties of this object, use the <context> element. That done, you may refer to these properties anywhere inside the <process> element using dot syntax, as in: `context.Balance`

Variable	Purpose
request	The *request* object contains any properties of the original request message object that caused this business process to be instantiated. You may refer to *request* properties anywhere inside the <process> element using dot syntax, as in: `request.UserID`
response	The *response* object contains any properties that are required to build the final response message object to be returned by the business process. You may refer to *response* properties anywhere inside the <process> element using dot syntax, as in: `response.IsApproved`. Use the <assign> element to assign values to these properties.

Note: There is more information about the business process execution context in documentation of the <assign> element.

You can fine-tune loop execution by including <break> and <continue> elements within a <foreach> element. See the descriptions of these elements for details.

BPL <if>

Evaluates a condition and performs one action if true, another if false, as a step in a BPL business process.

Syntax

```
<if condition="1">
   <true>
     ...
   </true>
   <false>
     ...
   </false>
</if>
```

Attributes and Elements

condition attribute

Required. An expression that, if true, causes the contents of the <true> element to execute. If false, the contents of the <false> element are executed.

Specify an expression that evaluates to 1 (if true) or 0 (if false).

LanguageOverride attribute

Optional. Specifies the scripting language in which any expressions (within in this element) are written.

Can be `"python"`, `"objectscript"`, or `"basic"` (not documented). Default is the language specified in the containing <process> element.

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

<true> element

Optional. If the condition is true, activities inside the <true> element are executed.

<false> element

Optional. If the condition is false, activities inside the <false> element are executed.

Description

The <if> element evaluates an expression and, depending on its value, executes one of two sets of activities (one if the expression evaluates to a true value, the other if it evaluates to a false value).

The <if> element may contain a <true> element and a <false> element which define the actions to execute if the expression evaluates to true or false, respectively.

If both <true> and <false> elements are provided, they may appear within the <if> element in any order.

If the condition is true and there is no <true> element, or if the condition is false and there is no <false> element, no activity results from the <if> element.

The following example shows an <if> element used to coordinate the results of a combination of <call> and <sync> elements used together.

XML

```
<sequence name="thread1">
  <call name="A" />
  <call name="B"  />
  <sync calls="A,B" type="all" timeout="10" />
  // Did the synchronization time out before it finished?
  <if condition='synctimedout="1"'>
    <true>
      <trace value="thread1 timeout: Call A or B did not return." />
    </true>
    // If not, then the calls came back, so assign the results.
      <false>
        <assign property="context.TheResultsFromEast"
                value='syncresponses.GetAt("A")'
                action="append"/>
        <assign property="context.TheResultsFromWest"
                value='syncresponses.GetAt("B")'
                action="append"/>
      </false>
    </if>
  </sequence>
```

The <if> activity in this example has a condition that tests the execution context variable *synctimedout* against the integer value 1. *synctimedout* can have the value 0, 1, or 2 as described in the documentation for <call>. If the two values are equal, this <if> condition receives the integer value 1 and statements inside the <true> element are executed. Otherwise, statements inside the <false> element are executed.

Note: There is more information about the business process execution context in documentation of the <assign> element.

See Also

- <true>
- <false>

BPL <label>

Provides a destination for a conditional branch operation, within a BPL business process.

Syntax

```
<label name="JumpToMe" />
```

Attributes and Elements

name attribute

Required. The name of this label. This name must be unique across the entire BPL business process. Specify a string of up to 255 characters.

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Description

The <label> element provides the destination for a conditional <branch> element.

For details, see the documentation for <branch>.

BPL <milestone>

Stores a message to acknowledge an achievement within a BPL business process.

Syntax

```
<milestone value='"The applicant has been notified of the interest rate."' />
```

Attributes and Elements

value attribute

Required. This is the text for the milestone message. It can be a literal text string or an expression to be evaluated.

LanguageOverride attribute

Optional. Specifies the scripting language in which any expressions (within in this element) are written.

Can be `"python"`, `"objectscript"`, or `"basic"` (not documented). Default is the language specified in the containing <process> element.

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Description

A <milestone> activity writes a message to the InterSystems IRIS database. <milestone> works very much like <trace>, but unlike <trace> messages, <milestone> messages exist only while the associated business process is running. After the business process exits, all messages generated by <milestone> activities are removed.

Often a programmer uses <trace> messages for diagnostic purposes, whereas <milestone> messages can be helpful to track the progress of a correctly operating, long-running business process.

You can retrieve <milestone> messages by examining values in the ^Ens.Milestone global. The global is defined only if your production has issued <milestone> messages. To obtain the value of ^Ens.Milestone:

- Programmatically, use the information in Using Multidimensional Storage (Globals).
- From the Management Portal, navigate to the **System Explorer** > **Globals** page, ensure that the **Namespaces** option is selected, and click the name of the namespace where your production runs. The **View Globals** option is selected by default.

BPL <parameters> and <parameter>

Specifies the parameters for another BPL element as a set of name-value pairs, for use in a BPL business process.

Syntax

```
<parameters>
    <parameter name='MAXLEN' value='1024' />
    <parameter name='MINLEN' value='1' />
</parameters>
```

Elements

<parameter> element

Zero or more <parameter> elements may appear inside the <parameters> container. Each <parameter> element defines one parameter.

Each <parameter> element has two attributes, *name* and *value*, as described below.

Description

The optional <parameters> element is valid only within <property> or <xslt>. <parameters> defines the parameters for its containing BPL element as a set of name-value pairs:

- Within <context>, <parameters> contains the data type parameters for a <property> that you are defining in the business process execution context. There is a detailed explanation of the business process execution context in documentation of the <assign> element.
- Within <xslt>, <parameters> contains any XSLT name-value pairs that you wish to pass to the stylesheet that controls the XSLT transformation.

<parameters> does not support any BPL attributes. It is simply a container for zero or more <parameter> element, one for each parameter. You may provide as many <parameter> elements as you wish, but all must appear within the same <parameters> block. For example:

XML

```
<context>
  <property name='Test' type='%Integer' initialexpression='342' >
    <parameters>
      <parameter name='MAXVAL' value='1000' />
    </parameters>
  </property>
  <property name='Another' type='%String' initialexpression='Yo' >
    <parameters>
      <parameter name='MAXLEN' value='2' />
      <parameter name='MINLEN' value='1' />
    </parameters>
  </property>
</context>
```

<parameter> Attributes

name attribute

Required. The name of this parameter:

- Within <property>, *name* identifies a data type parameter for the property. For valid names, see Parameters.
- Within <xslt>, *name* must be the name of a valid XSLT parameter.

value attribute

Optional. The value to assign to the parameter.

See Also

- <context>
- <xslt>

BPL <process>

Defines a BPL business process.

Syntax

```
<process request="MyApp.Request" response="MyApp.Response">
    <context>
     ...
    </context>
    <sequence>
     ...
    </sequence>
</process>
```

Attributes and Elements

request attribute

Required. The name of the request class, specifying the type of the initial request to this business process.

response attribute

Optional. The name of the response class, specifying the type of the response returned by this business process, if any.

component attribute

Optional. Setting this value to 1 (true) designates this <process> as a reusable component.

Specify a Boolean value: 1 (true) or 0 (false). The default is false.

contextsuperclass attribute

Optional. Lets you specify the superclass for your business process context. This is useful if you have many different business processes that share the same execution context variables. The idea is that you subclass Ens.BP.Context yourself, adding your own properties, then use that class for the *contextsuperclass*. If not specified, Ens.BP.Context is the default. Specify a class name.

height attribute

Optional. Refers to the graphical representation of the business process in the Business Process Designer. Specify a positive integer.

includes attribute

Optional. A comma-delimited list of ObjectScript include file names, so that you can use macros in your <code> segments.

language attribute

Optional. Specifies the default language in which any expressions or <code> elements are written.

Can be `"python"`, `"objectscript"`, or `"basic"` (not documented). Default is `"objectscript"`.

layout attribute

Optional. The name of the layout style used in BPL diagrams for this business process. The value `automatic` indicates that the Business Process Designer and BPL Viewer will choose layouts for the diagram elements. The value `manual` overrides the tools to use the exact layout that you specify. Specify a string, either `manual` or `automatic`. If not specified, the default layout is `automatic`.

version attribute

Optional. An integer that expresses a version number. Higher values indicate later versions. If an expression, the *version* attribute value must use ObjectScript. Specify a positive integer. May be a literal integer, or an expression that evaluates to an integer.

width attribute

Optional. Refers to the graphical representation of the business process in the Business Process Designer. Specify a positive integer.

<context> element

Optional. Defines general-purpose properties in the business process execution context. For information about the business process execution context, see <assign>, and see Developing BPL Processes.

<pyFromImport> element

Optional. An optional list of Python `from / import` statements, one per line. Use this so that Python code within this business process can refer to these modules.

<sequence> element

Optional. Zero or more <sequence> elements may appear. Each defines actions that the business process can perform.

Description

The <process> element is the outermost element for a BPL document. All the other BPL elements are contained within a <process> element.

A business process consists of an execution context (defined by the <context> element) and a sequence of activities (defined by the <sequence> element).

The request attribute defines the type (class name) for the business process's initial request. The response attribute defines the type (class name) for the eventual response from the business process. The request attribute is required, but the response attribute is optional, since the business process might not return a response.

Execution Context

The life cycle of a business process requires it to have certain state information saved to disk and restored from disk, whenever the business process suspends or resumes execution. A BPL business process supports the business process life cycle with a group of variables known as the *execution context*.

The execution context variables include the objects called *context*, *request*, *response*, *callrequest*, *callresponse* and *process*; the integer value *synctimedout*; the collection *syncresponses*; and the %Status value *status*. Each variable has a specific purpose, as described in documentation for the <assign>, <call>, <code>, and <sync> elements.

Example

The following sample business process provides a <sync> element to synchronize several <call> elements. Further activities within the <process> element are replaced by ellipses (...) near the end of the example:

XML

```
<process request="Demo.Loan.Msg.Application">
<context>
  <property name="BankName" type="%String"/>
  <property name="IsApproved" type="%Boolean"/>
  <property name="InterestRate" type="%Numeric"/>
  <property name="TheResults" type="Demo.Loan.Msg.Approval" collection="list"/>
  <property name="Iterator" type="%String"/>
  <property name="ThisResult" type="Demo.Loan.Msg.Approval"/>
</context>
<sequence>
  <trace value='"received application for "_request.Name'/>
  <call name="BankUS" target="Demo.Loan.BankUS" async="1">
    <annotation>
      <![CDATA[Send an asynchronous request to Bank US.]]>
    </annotation>
    <request type="Demo.Loan.Msg.Application">
      <assign property="callrequest" value="request"/>
    </request>
    <response type="Demo.Loan.Msg.Approval">
      <assign property="context.TheResults"
              value="callresponse"
              action="append"/>
    </response>
  </call>

  <call name="BankSoprano" target="Demo.Loan.BankSoprano" async="1">
    <annotation>
      <![CDATA[Send an asynchronous request to Bank Soprano.]]>
    </annotation>
    <request type="Demo.Loan.Msg.Application">
      <assign property="callrequest" value="request"/>
    </request>
    <response type="Demo.Loan.Msg.Approval">
      <assign property="context.TheResults"
              value="callresponse"
              action="append"/>
    </response>
  </call>

  <call name="BankManana" target="Demo.Loan.BankManana" async="1">
    <annotation>
      <![CDATA[Send an asynchronous request to Bank Manana.]]>
    </annotation>
    <request type="Demo.Loan.Msg.Application">
      <assign property="callrequest" value="request"/>
    </request>
    <response type="Demo.Loan.Msg.Approval">
      <assign property="context.TheResults"
              value="callresponse"
              action="append"/>
    </response>
  </call>

  <sync name='Wait for Banks'
        calls="BankUS,BankSoprano,BankManana"
        type="all"
        timeout="5">
    <annotation>
      <![CDATA[Wait for responses. Wait up to 5 seconds.]]>
    </annotation>
  </sync>
  <trace value='"sync complete"'/>
  ...
</sequence>
</process>
```

Replies

The *primary response* from a business process is the response it returns to the request that originally invoked the specific business process instance. Normally, the business process returns its primary response automatically, as soon as it is done executing. However, the <reply> element can be used to return the primary response sooner. This can be useful if the response needed by the original caller is ready to be returned, but there is additional work for the business process to perform as a result of the original call.

Language

The <process> element defines the scripting language used by a business process by providing a value for the language attribute: The value should be `"objectscript"`. Any expressions found in the business process, as well as lines of code within <code> elements, must use the specified language.

Versioning

Developers can update the *version* number for a BPL business process to indicate that its new functionality is incompatible with previous versions. A higher number indicates later versions. There is no automatic versioning of BPL business processes. A developer manually updates the value of the *version* attribute within the BPL <process> element to highlight the fact that the new code contains changes that are incompatible with previous versions of the same business process. Examples include adding or deleting properties within the business process <context>, or changing the flow of activities within the business process <sequence>.

Prior versions of the same BPL business process that have instances already executing continue to execute their original activities, with their original context. New versions use their own context and their own activities. InterSystems IRIS achieves this by generating new context and thread classes for each version. The version appears as a subpackage in the generated class hierarchy. For example, if you have a class MyBPL, version 3 generates MyBPL.V3.Context and MyBPL.V3.Thread1.

Layout

By default, when a user opens a BPL diagram in the Business Process Designer, the tool display the diagram using automatic layout arrangements. These automatic choices may or may not be appropriate for a particular drawing. If you suspect that this may be an issue for your diagram, you can disable automatic layout to ensure that your diagram always displays with exactly the layout you want.

The most direct way to control the layout of your diagram is to clear the **Auto arrange** check box on the **Preferences** tab.

You can also click the General tab and choose either Automatic or Manual for the Layout. The `manual` selection preserves the exact position of each element each time you save the diagram, so that when the diagram is displayed in the Business Process Designer, it does not take on any layout characteristics except any that you specify.

Problems in scrolling through a business process diagram in the Business Process Designer can be fixed by adjusting the height or width attributes of the <process> element. You can do this using the General tab as for the layout attribute.

BPL <property>

Defines a property within the <context> element for a business process, as a step in a BPL business process.

Syntax

```
<property name='Test' type='%Integer' initialexpression='342' >
   <parameters>
      <parameter name='MAXVAL' value='1000' />
   </parameters>
</property>
```

Attributes and Elements

name attribute

Required. The name of this property. It must be a valid property name.

type attribute

Optional. The name of the class that specifies the type of this property. It can be a data type class (%String) or a serial or persistent class.

initialexpression attribute

Optional. This ObjectScript expression is evaluated to provide a default value for the property. Specify an expression that provides a valid value for the property. See the discussion below.

instantiate attribute

Optional. Acts as a `create` flag for the property. If not specified, the default is 0 (do not create). Specify 1 (create) or 0 (do not create)

collection attribute

Optional. If present, specifies that this property is a collection of a certain type. Specify a literal string, either `list`, `array`, `binarystream`, or `characterstream`

<parameters>

An optional <parameters> element may appear. Inside the <parameters> container, zero or more <parameter> elements may appear. Each <parameter> element defines one data type parameter for the property by providing a parameter *name* and *value*. For valid names and values, see Parameters.

Description

The <property> element defines a property within the business process execution context.

The life cycle of a business process requires it to have certain state information saved to disk and restored from disk, whenever the business process suspends or resumes execution. A BPL business process supports the business process life cycle with a group of variables known as the *execution context*.

The execution context variables include the objects called *context*, *request*, *response*, *callrequest*, *callresponse* and *process*; the integer value *synctimedout*; the collection *syncresponses*; and the %Status value *status*. Each variable has a specific purpose, as described in documentation for the <assign>, <call>, <code>, and <sync> elements.

Most of the execution context variables are automatically defined for the business process. The exception to this rule is the general-purpose container object called *context*, which a BPL developer must define. Any value that you want to be persistent and available everywhere within the business process should be declared as a property of the *context* object. You can do

this by providing <context> and <property> elements at the beginning of the BPL document. Each <property> element defines one property of the *context* object.

A <property> element must provide a name.

For non-collection properties, the initialexpression and instantiate attributes dictate how the object will be initialized. If the instantiate attribute has the integer value 1 (true), then a call to "new" the object will be generated. If an initialexpression attribute is specified as well, then the result of this expression will be assigned to the object.

The instantiate attribute should be used to initialize properties that can be instantiated, whereas the initialexpression attribute should be used to initialize data type classes such as %String. For string values, be sure to provide the string quotes wrapped inside another set of quotes. That is: `initialexpression='"hello"'` to set an initial string value of `"hello"`.

If the collection attribute is set (`list`, `array`, `binarystream`, or `characterstream`) the property is automatically instantiated as a collection of that type.

The following example shows a set of <property> elements within the <context> element at the beginning of a business process:

XML

```
<process request="Demo.Loan.Msg.Application" response="Demo.Loan.Msg.Approval">
  <context>
    <property name="BankName" type="%String"
              initialexpression="BankOfMomAndDad" />
    <property name="IsApproved" type="%Boolean"/>
    <property name="InterestRate" type="%Numeric"/>
    <property name="TheResults"
              type="Demo.Loan.Msg.Approval"
              collection="list"/>
    <property name="Iterator" type="%String"/>
    <property name="ThisResult" type="Demo.Loan.Msg.Approval"/>
  </context>
  ...
</process>
```

Each <property> element defines the name and data type for a property. For a list of available data type classes, see Parameters. <property> may assign an initial value by providing an *initialexpression* attribute. Alternatively, you may assign values during business process execution, using the <assign> element.

See Also

- <parameters>

<pyFromImport>

Specifies optional Python `from / import` statements.

Syntax

```
<pyFromImport>
from math import cos
</pyFromImport>
```

An optional list of Python `from / import` statements, one per line. Use this so that Python code within this business process can refer to these modules.

BPL <reply>

Sends a response from a business process before its execution is complete, as a step in a BPL business process.

Syntax

```
<reply/>
```

Attributes and Elements

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Description

The “primary response” from a business process is the response it returns to the request that originally invoked the specific business process instance. Normally, the business process will return its primary response automatically, as soon as it is done executing. However, the <reply> element can be used to return the primary response sooner. This can be useful if the response needed by the original caller is ready to be returned, but there is additional work for the business process to perform as a result of the original call.

The <reply> element returns the *response* object from the business process execution context, so a business process must use the <assign> element to assign values to properties on the *response* object, prior to making a <reply>.

Note: There is more information about the business process execution context in the documentation for <assign>.

Example

The following example shows the reply action, used to return a response before continuing to execute the rest of the business process:

```
<call name="FindSalary" target="MyApp.PayrollApp" async="1">
    <request type="MyApp.SalaryRequest">
        <assign property="callrequest.Name" value="request.Name" />
        <assign property="callrequest.SSN" value="request.SSN" />
    </request>
    <response type="MyApp.SalaryResponse">
        <assign property="context.Salary" value="callresponse.Salary" />
    </response>
</call>
<assign property="response.Salary" value="context.Salary" />
</reply>
<call name="UpdateSalaryCache" target="MyApp.PayrollApp" async="0">
    <request type="MyApp.SalaryCacheRequest">
        <assign property="callrequest.SSN" value="request.SSN" />
        <assign property="callrequest.Salary" value="context.Salary" />
    </request>
</call>
```

BPL <request>

Prepares a request within a <call> element, as a step in a BPL business process.

Syntax

```
<call name="Call" target="MyApp.MyOperation" async="1">
   <request type="MyApp.Request">
     ...
   </request>
   <response type="MyApp.Response">
     ...
   </response>
</call>
```

Attributes and Elements

type attribute

Required. The name of the request message class.

name attribute

Optional. The name of the <request> element. Specify a string of up to 255 characters.

Other elements

Optional. <request> may contain zero or more of the following elements in any combination: <assign>, <empty>, <milestone>, or <trace>.

Description

A <request> element is a required child element of <call>. Inside the <call> context, the <request> element specifies the type (class name) of the request to send. The <request> element can also contain one or more <assign> elements. Each of these assigns a value to a property on an object in the business process execution context. For example:

XML

```
<call name="FindSalary" target="MyApp.PayrollApp" async="1">
  <request type="MyApp.SalaryRequest">
    <assign property="callrequest.Name" value="request.Name" />
    <assign property="callrequest.SSN" value="request.SSN" />
  </request>
  <response type="MyApp.SalaryResponse">
    <assign property="context.Salary" value="callresponse.Salary" />
  </response>
</call>
```

The intention of any <assign> elements found within a <request> element is usually to assign values to properties on the *callrequest* object. This object is the member of the business process execution context that acts as a container for the properties of the request object used for the call. However, properties on the *context*, *request*, and *response* objects can also be set, appended, or otherwise manipulated in an <assign> element inside a <request>.

For further discussion of the business process execution context, see the documentation for <call> and <assign>.

See Also

- <process>
- <reply>

BPL <response>

Handles a response received within a <call> element, as a step in a BPL business process.

Syntax

```
<call name="Call" target="MyApp.MyOperation" async="1">
   <request type="MyApp.Request">
     ...
   </request>
   <response type="MyApp.Response">
     ...
   </response>
</call>
```

Attributes and Elements

type attribute

Required. The name of the response message class.

name attribute

Optional. The name of the <response> element. Specify a string of up to 255 characters.

Other elements

Optional. <response> may contain zero or more of the following elements in any combination: <assign>, <empty>, <milestone>, or <trace>.

Description

A <response> element is an optional child element of <call>. Inside the <call> context, the <response> element specifies the type (class name) of the response to return from the call. The <response> element can also contain one or more <assign> elements. For example:

XML

```
<call name="FindSalary" target="MyApp.PayrollApp" async="1">
  <request type="MyApp.SalaryRequest">
    <assign property="callrequest.Name" value="request.Name" />
    <assign property="callrequest.SSN" value="request.SSN" />
  </request>
  <response type="MyApp.SalaryResponse">
    <assign property="context.Salary" value="callresponse.Salary" />
  </response>
</call>
```

When a call returns a response to the calling business process, any output parameters from the message type named in the <response> element become properties of the *callresponse* object in the business process execution context. Since *callresponse* only has meaning inside the <response> element, to preserve these values the <response> element must provide <assign> elements that assign *callresponse* values to properties of other, more permanent objects in the business process execution context, usually *context* or *response*.

For further discussion, see the documentation for <call> and <assign>.

While a <request> element is required inside every <call>, a <response> is not. If the <response> element is omitted from a <call> element, no response is returned from the <call>, even if the <request> type is designed to return a response. When the <request> is asynchronous, the <assign> elements within the body of the <response> element are executed only after the call response is received. There is no guarantee when this will occur, so a business process will typically use the <sync> element to wait for an asynchronous response.

If a response is not received within the timeout period specified by the <sync> element, then the assignments defined by the corresponding <response> block will not be executed. The response itself will be marked with a status of Discarded.

See Also

- <process>
- <reply>

BPL <rule>

Calls a production business rule class, as a step in a BPL business process.

Syntax

```
<rule name="ApproveLoan"
        rule="LoanApproval"
        resultLocation="context.Answer"
        reasonLocation="context.Reason">
</rule>
```

Attributes and Elements

name attribute

Required. The name of the <rule> element.

rule attribute

Required. The name of the business rule to be executed. This must be a valid rule within the namespace; see Identifying the Rule, below. If the rule is not defined or otherwise cannot be found at runtime, the rule will return a default value of "" (an empty string).

ruleContext attribute

Optional. If defined, this is an expression that identifies the object to pass to the rules engine; see Identifying the Context, below. For example:

```
context.MyObject
```

By default the rule passes the business process execution context to the rules engine.

resultLocation attribute

Optional. The location in which to store the return value of the rule. Typically this is a property within the business process execution context; that is, context.MyValue.

Specify the name of a valid property and object, usually within the business process execution context.

reasonLocation attribute

Optional. The location in which to store the reason returned by the rule. The rule reason is a string indicating why a business rule reached its decision. For example, “Rule 1” or “Default”. If the business rule is empty (for example, it is a rule set that contains no rules) then the reason given for the decision is `Rule Missing.`

disabled, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Description

The <rule> element invokes a business rule from a business process. When a <rule> executes, it invokes its associated business rule (named by the *rule* attribute) and gets its response immediately (in the same manner as a <code> or <assign> activity).

Identifying the Rule

When you use the <rule> element in BPL, the value of the *rule* attribute can be either of the following:

- A simple **Rule Name**:

  `MyRule`

- A full **Package Name** plus **Rule Name** combination:

  `MyClassPackage.Organization.Levels.MyRule`

If any <rule> element identifies a simple **Rule Name**, InterSystems IRIS automatically prepends a **Package Name** that is equal to the full package and class name of the BPL business process that contains that <rule> element. That is:

BPLFullPackageAndClassName`.MyRule`

This combination must identify a valid rule within the namespace, or the return value of the <rule> will be a null string.

Identifying the Context

By default, the *ruleContext* passed to the rule is the business process execution context. If you specify a different object as a context, there are some restrictions on this object: It must have a property called %Process of type Ens.BusinessProcess; this is used to pass the business process calling context to the rules engine. You do not need to set the value of this property, but it must be present. Also, the object must match what is expected by the rule itself. No checking is done to ensure this; it is up to the developer to set this up correctly.

A Simple Example

The following is a BPL excerpt showing the use of the <rule> activity with a <switch> element to process the results from the rule:

```
<sequence>
  <rule name="ExecuteRule"
        rule="MyRule"
        resultLocation="context.MyResult" />
  <switch>
    <case condition="context.MyResult=1">
      <!-- ...Rule is true... -->
    </case>
    <default>
      <!-- ... Rule is false... -->
    </default>
  </switch>
</sequence>
```

The <rule> activity in this example returns a Boolean value (true or false) according to InterSystems IRIS conventions. That is, an integer value of 1 means true; 0 means false. All rules return a single value, as in this example, but the type need not be Boolean. The single value returned from a rule may be *any literal value* such as an integer number, decimal number, or text string.

Return Values

The result and reason for the result are stored in the variables identified by the *resultLocation* and *reasonLocation* attributes, respectively. Usually, these attributes give the names of properties in the *context* variable. This is the general-purpose, persistent variable that you define at the beginning of the BPL business process using <context> and <property> elements.

See Also

- Developing Business Rules

BPL <sequence>

Performs activities in sequential order, as a step in a BPL business process.

Syntax

```
<sequence>
   ...
</sequence>
```

Attributes and Elements

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Other elements

Optional. <sequence> may contain zero or more of the following elements in any combination: <alert>, <assign>, <branch>, <break>, <call>, <code>, <continue>, <delay>, <empty>, <flow>, <foreach>, <if>, <label>, <milestone>, <reply>, <rule>, <scope>, <sequence>, <sql>, <switch>, <sync>, <throw>, <trace>, <transform>, <until>, <while>, <xpath>, or <xslt>.

Description

A <sequence> element is used within a <process> or a <flow> to contain elements that need to be executed in sequential order.

<sequence> and <process>

Every BPL document must have at least one <sequence> element within its <process> element that specifies the main sequence of activities for the business process. For example:

XML

```
<process>
  <sequence>
    <call name="A" />
    <call name="B" />
  </sequence>
</process>
```

When you use the Business Process Designer, as you add activities between the <start> and <end> elements of a new, top-level BPL diagram, everything you add is contained within a single top-level <sequence> that the BPL code generator places inside the <process> element in the generated code. Such a <sequence> is shown in the preceding example. The <call> element `A` is executed first, followed by the <call> element `B`.

When you use the Business Process Designer, if you need to temporarily disable the top-level <sequence> within a <process>, you can disable it in your IDE in the generated BPL code by adding the *disabled* attribute to the corresponding <sequence> element.

Nested <sequence> Elements

A <sequence> can contain other sequences. Nested <sequence> elements do not start additional execution threads; for that you need the <flow> element. However, you can use superfluous nested <sequence> elements as a means to group items within a BPL document. For example:

XML

```
<process>
  <sequence>
    <sequence>
      <call name="A" />
      <call name="B" />
    </sequence>
  </sequence>
</process>
```

Nested <sequence> elements have no effect on the code generated for the business process. The BPL diagram, however, displays such nested sequences as a single <sequence> icon. You can drill down into the <sequence> icon to view the elements contained within.

<sequence> and <flow>

When you are using the Business Process Designer and you add a <flow> element to the business process, a <sequence> element is automatically inserted inside the <flow>, as you can see by examining the generated BPL code. You may add additional <sequence> elements to the flow; in fact, each branch of the <flow> *must* be enclosed within its own <sequence> element.

If you need to temporarily disable one of the <sequence> elements within a <flow>, you can do it in your IDE in the generated BPL code by adding the *disabled* attribute and setting it to true in the corresponding <sequence> element.

BPL <scope>

Defines the error handling mechanisms for a sequence of activities, as a step in a BPL business process.

Syntax

```
<scope>
   <throw fault='"MyFault"'/>
          ...
   <faulthandlers>
     <catch fault='"MyFault"'>
           ...
     </catch>
   </faulthandlers>
</scope>
```

Attributes and Elements

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Other elements

Optional. <scope> may contain zero or more of the following elements in any combination: <alert>, <assign>, <branch>, <break>, <call>, <code>, <continue>, <delay>, <empty>, <flow>, <foreach>, <if>, <label>, <milestone>, <reply>, <rule>, <scope>, <sequence>, <sql>, <switch>, <sync>, <throw>, <trace>, <transform>, <until>, <while>, <xpath>, or <xslt>.

Description

To enable error handling, BPL defines an element called <scope>. A scope is a wrapper for a set of activities. This scope may contain one or more activities, one or more fault handlers, and zero or more compensation handlers. The fault handlers and are intended to catch any errors that activities within the <scope> produce. The fault handlers may invoke compensation handlers to compensate for those errors.

The following example provides a <scope> with a <faulthandlers> block that includes a <catchall>:

Class Member

```
XData BPL
{
<process language='objectscript'
         request='Test.Scope.Request'
         response='Test.Scope.Response' >
  <sequence>
    <trace value='"before scope"'/>
    <scope>
      <trace value='"before assign"'/>
      <assign property="SomeProperty" value="1/0"/>
      <trace value='"after assign"'/>
      <faulthandlers>
        <catchall>
          <trace value='"in catchall faulthandler"'/>
          <trace value=
            '"%LastError "_
            $System.Status.GetErrorCodes(..%Context.%LastError)_
            " : "_
            $System.Status.GetOneStatusText(..%Context.%LastError)'
            />
        </catchall>
      </faulthandlers>
    </scope>
```

```
    <trace value='"after scope"'/>
  </sequence>
</process>
}
```

When a <scope> provides no <faulthandlers> block, InterSystems IRIS automatically outputs the system error to the Event Log. When a <scope> *does* contain a <faulthandlers> block, the BPL business process must output <trace> messages to the Event Log for system error messages to appear there. System error messages do appear in the ObjectScript shell, in either case.

It is possible to nest <scope> elements. An error or fault that occurs within the inner scope may be caught within the inner scope, or the inner scope may ignore the error and allow it to be caught by the <faulthandlers> block in the outer scope.

For details, see Handling Errors in BPL.

See Also

- <catch>
- <catchall>
- <compensate>
- <compensationhandlers>
- <faulthandlers>
- <throw>

BPL <sql>

Executes an embedded SQL SELECT statement, as a step in a BPL business process.

Syntax

```
<sql name="LookUp">
   <![CDATA[
     SELECT SSN INTO :context.SSN
     FROM MyApp.PatientTable
     WHERE PatID = :request.PatID   ]]>
</sql>
```

Attributes and Elements

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Description

The <sql> element executes an arbitrary embedded SQL SELECT statement from within the execution of a business process.

The <sql> element is especially powerful for performing lookup operations using tables. For example, suppose the primary request coming into a business process provides a PatId property that indicates a Patient Identity number, and you need to find the matching Social Security number (SSN) before the business process can perform work. If you have available a PatientTable table relating PatId with SSN, you can perform the lookup using the following <sql> element:

XML

```
<process>
  <sql name="LookUp"><![CDATA[
    SELECT SSN INTO :context.SSN
    FROM MyApp.PatientTable
    WHERE PatID = :request.PatID
    ]]>
  </sql>
</process>
```

Where the execution context variable *context* has an SSN property that is suitable to receive the result of the SQL query. The execution context variable *request* automatically contains the PatId property, as it always contains the properties received in the primary request object.

Note: For more information about the business process execution context, see <assign>, and see Developing BPL Processes.

If you maintain a local copy of the PatientTable within the InterSystems IRIS database, the above example is especially efficient, as it can be executed without using any expensive network operations or additional middleware.

To use the <sql> element effectively, keep the following tips in mind:

- Always use the fully qualified name of the table, including both the SQL schema name and table name, as in:

  `MyApp.PatientTable`

  Where `MyApp` is the SQL schema name and `PatientTable` is the table name.

- The contents of the <sql> element must contain a valid embedded SQL SELECT statement.

It is convenient to place the SQL query within a CDATA block so that you do not have to worry about escaping special XML characters.

- If the SQL returns a SQLCODE error, this action will also return an error which can be handled using BPL error handling.
- Any tables listed in the SQL query's FROM clause must either be stored within the local InterSystems IRIS database or linked to an external relational database using the SQL Gateway.
- Within the INTO and WHERE clauses of the SQL query, you can refer to a property of one of the variables in the business process execution context by placing a colon (`:`) in front of the variable name. For example:

  XML

  ```
  <sql name="LookUp"><![CDATA[
    SELECT Name INTO :response.Name
    FROM MainFrame.EmployeeRecord
    WHERE SSN = :request.SSN AND City = :request.Home.City
  ]]>
  </sql>
  ```

- Only the first row returned by the query will be used. Make sure that your WHERE clause correctly specifies the desired row.

BPL <switch>

Evaluates a set of conditions to determine which of several actions to perform, as a step in a BPL business process.

Syntax

```
<switch>
   <case>
     ...
   </case>
   ...
   <default>
     ...
   </default>
</switch>
```

Attributes and Elements

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

<case> element

Required (at least one). Each <case> element defines a condition that may or may not be true.

<default> element

Optional. Specifies the action to take if no <case> condition is satisfied. If present, must appear last in the <switch> element.

Description

The <switch> element contains a sequence of one or more <case> elements and an optional <default> element.

When a <switch> element is executed, it evaluates each <case> condition in turn. These conditions are logical expressions in the scripting language of the containing <process> element. If any expression evaluates to the integer value 1 (true), then the contents of the corresponding <case> element are executed; otherwise the expression for the next <case> element is evaluated.

If no <case> condition is true, the contents of the <default> element are executed.

As soon as one of <case> elements is executed, execution control leaves the surrounding <switch> statement. If no <case> condition matches, control leaves the <switch> after the <default> activity executes.

If no <case> is true and there is no <default>, no activity results from the <switch> statement.

Activities within a <case> element can be any BPL activity, including <assign> elements as in the example below:

XML

```
<switch name='Approved?'>
  <case name='No PrimeRate' condition='context.PrimeRate=""'>
    <assign name='Not Approved' property="response.IsApproved" value="0"/>
  </case>
  <case name='No Credit' condition='context.CreditRating=""'>
    <assign name='Not Approved' property="response.IsApproved" value="0"/>
  </case>
  <default name='Approved' >
    <assign name='Approved' property="response.IsApproved" value="1"/>
    <assign name='InterestRate'
            property="response.InterestRate"
            value="context.PrimeRate+10+(99*(1-(context.CreditRating/100)))">
      <annotation>
        <![CDATA[Copy InterestRate into response object.]]>
      </annotation>
    </assign>
  </default>
</switch>
```

BPL <sync>

Waits for a response from one or more asynchronous requests, as a step in a BPL business process.

Syntax

```
<sequence>
   <call name="A" async="1" />
   <call name="B" async="1" />
   ...
   <sync calls="A,B" type="all" timeout="3600"/>
</sequence>
```

Attributes and Elements

calls attribute

Required. A list of the names of one or more asynchronous <call> elements that <sync> will wait for. Specify a comma-separated list of <call> element names. This value can be provided as a literal string, or by using the ObjectScript @ indirection operator to refer to the value of an execution context variable. See details below.

allowresync attribute

Optional. If true, the <sync> element can "poll" repeatedly to detect completion of an asynchronous call. That is, you can <sync> repeatedly on the same call. This feature is useful when a call may take an indefinite time to complete. The default *allowresync* value is false. Specify 1 (true) or 0 (false).

timeout attribute

Optional. Specifies the time, in seconds, to wait for the responses, as an expression that evaluates to an XML xsd:dateTime value. For example: `2023:10:19T10:10`.

type attribute

Optional. Specify either `"all"` (the default) or `"any"`

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Description

A typical business process makes one or more requests to external systems. These requests are usually made asynchronously to compensate for the fact that the external system may be slow to respond or occasionally unavailable. The <sync> element provides an easy way to wait for a response from one or more asynchronous calls. It is used in conjunction with the <call> element.

The behavior of the <sync> element is specified via the calls, timeout, and type attributes. The type attribute either has the value `all`, which specifies that the <sync> should wait for a response from all calls, or `any`, which specifies that it will only wait for the first response it receives (in this case, the remaining responses are discarded in the same manner as an expired timeout).

The following BPL fragment makes a two asynchronous requests, A and B, and then uses the <sync> element to wait for their responses (for up to one hour):

XML

```
<sequence>
  <call name="A" async="1" />
  <call name="B" async="1" />
  <sync calls="A,B" type="all" timeout="3600" />
</sequence>
```

Any responses received after the timeout period are marked with a status of Discarded, and are not processed by the business process. If no value is provided for a timeout, the <sync> element continues to wait until all responses are received, regardless of the amount of time that passes. Meanwhile, however, the business process is saved to disk and the job in which it was running is freed up to host other business processes while the <sync> element is waiting.

The following sample BPL <process> issues two calls, then waits for 5 seconds.

XML

```
<process request="Demo.Loan.Msg.Application">
  <context>
    <property name="BankName" type="%String"/>
    <property name="IsApproved" type="%Boolean"/>
    <property name="InterestRate" type="%Numeric"/>
    <property name="Results" type="Demo.Loan.Msg.Approval" collection="list"/>
    <property name="Iterator" type="%String"/>
    <property name="ThisResult" type="Demo.Loan.Msg.Approval"/>
  </context>
  <sequence>
    <trace value='"received application for "_request.Name'/>
    <call name="BankUS" target="Demo.Loan.BankUS" async="1">
    <annotation>
      <![CDATA[Send an asynchronous request to Bank US.]]>
    </annotation>
    <request type="Demo.Loan.Msg.Application">
      <assign property="callrequest" value="request"/>
    </request>
    <response type="Demo.Loan.Msg.Approval">
      <assign property="context.Results"
              value="callresponse"
              action="append"/>
    </response>
  </call>

  <call name="BankSoprano" target="Demo.Loan.BankSoprano" async="1">
    <annotation>
      <![CDATA[Send an asynchronous request to Bank Soprano.]]>
    </annotation>
    <request type="Demo.Loan.Msg.Application">
      <assign property="callrequest" value="request"/>
    </request>
    <response type="Demo.Loan.Msg.Approval">
      <assign property="context.Results"
              value="callresponse"
              action="append"/>
    </response>
  </call>

  <call name="BankManana" target="Demo.Loan.BankManana" async="1">
    <annotation>
      <![CDATA[Send an asynchronous request to Bank Manana.]]>
    </annotation>
    <request type="Demo.Loan.Msg.Application">
      <assign property="callrequest" value="request"/>
    </request>
    <response type="Demo.Loan.Msg.Approval">
      <assign property="context.Results"
              value="callresponse"
              action="append"/>
    </response>
  </call>

  <sync name='Wait for Banks'
        calls="BankUS,BankSoprano,BankManana"
        type="all"
        timeout="5">
    <annotation>
      <![CDATA[Wait for responses from the banks. Wait up to 5 seconds.]]>
    </annotation>
  </sync>
```

```
  <trace value='"sync complete"'/>
  </sequence>
</process>
```

Whenever a <sync> element is executed, the BPL engine inserts the *name* of the <sync> element into the message header so that it is visible in later Message Browser and Visual Trace displays.

Unique Names for <call> Elements

If you attempt to define a <call> element with the same *name* as another <call> element, the BPL editor displays an error message and requires that you provide a unique name.

Indirection in the calls Attribute

The value of the *calls* attribute is a string. This string must provide a comma-separated list of <call> element names. The string can be a literal value:

```
calls="BankUS,BankSoprano,BankManana"
```

Or the @ indirection operator can be used to access the value of an execution context variable that contains the appropriate string:

```
calls="@context.myListOfCalls"
```

syncresponses

There is an additional mechanism for dealing with responses received as a result of the <call> element.

Whenever the <sync> element is used, it fills a collection with the various responses it receives. This collection is a variable in the business process execution context called *syncresponses*. The execution context also provides a integer variable called *synctimedout*. The two variables *synctimedout* and *syncresponses* work together as follows:

Object	Description
syncresponses	*syncresponses* is a collection of response objects, keyed by the names of the <call> activities being synchronized. Only completed calls are represented. You can retrieve a response from *syncresponses* only after a <sync> and before the end of the current <sequence>. Do so using the syntax `syncresponses.GetAt("MyName")` where the relevant call was defined as `<call name="MyName">`
synctimedout	The *synctimedout* value is an integer. *synctimedout* indicates the outcome of a <sync> activity after several calls. You can test the value of *synctimedout* after the <sync> and before the end of the <sequence> that contains the calls and <sync>. *synctimedout* has one of three values: • If 0, no call timed out. All the calls had time to complete. This is also the value if the <sync> activity had no *timeout* set. • If 1, at least one call timed out. This means not all <call> activities completed before the timeout. • If 2, at least one call was interrupted before it could complete. Generally you will test *synctimedout* for status and then retrieve the responses from completed calls out of the *syncresponses* collection.

As soon as a <sync> activity executes, the *syncresponses* collection is cleared in preparation for new responses. As the calls return, their responses go into the *syncresponses* collection. When the <sync> activity completes, *syncresponses* may contain some or all of the responses that you were waiting for.

For example, suppose you <sync> on Call1 and Call2 with this syntax:

```
<sync type="all" timeout="60">
```

Suppose that Call1 returns within 60 seconds, but Call2 does not. At this point, *syncresponses* contains the response to Call1, but not Call2. You can test the value of *synctimedout* to determine whether or not to expect the appropriate values to be present in *syncresponses*.

Following a <sync> activity, you can access whatever responses have returned by using the name of the <call> activity as a key. For a call defined as:

```
<call name="nameOfCall">
```

You would access the response using this syntax:

```
syncresponses.GetAt("nameOfCall")
```

Suppose the following sequence executes:

XML

```
<sequence>
  <call name="A" async="1" />
  <call name="B" async="1" />
  <call name="C" async="1" />
  <sync calls="A,B,C" type="all" />
</sequence>
```

After the <sync> element completes, the *syncresponses* collection will contain references to three response objects, as follows:

- `syncresponses.GetAt("A")` = Response from A (if any)
- `syncresponses.GetAt("B")` = Response from B (if any)
- `syncresponses.GetAt("C")` = Response from C (if any)

If no responses were received, the *syncresponses* collection will be empty.

Note: For more information about the business process execution context, see <assign>, and see Developing BPL Processes.

syncresponses in Multiple Threads

When you use the <sync> element in conjunction with <flow>, be aware that there is a separate *syncresponses* collection for each thread, including the primary thread in which the <process> itself executes. Therefore, in the course of a business process there may be different *syncresponses* collections that go in and out of scope; each has relevance only in its immediate <sequence> and not in any other.

The following example illustrates the use of *synctimedout* and *syncresponses* in three threads, the primary business process thread and two additional threads created by a <flow>:

Class Member

```
XData BPL
{
<process>
  <context>
    <property name="ResultsFromNorth" type="%String"/>
    <property name="ResultsFromSouth" type="%String"/>
    <property name="ResultsFromEast" type="%String"/>
    <property name="ResultsFromWest" type="%String"/>
  </context>
  <sequence>
  // In this context, syncresponses refers to the primary process thread
    <flow>
    // This flow runs two sequences (two threads) in parallel

      <sequence name="thread1">
```

```
      // In this context, syncresponses refers to results in thread1
        <call name="A" />
        <call name="B"  />
        <sync calls="A,B" type="all" timeout="10" />
        // Did the synchronization time out before it finished?
        <if condition='synctimedout="1"'>
          <true>
            <trace value='"thread1 timeout: Call A or B did not return."' />
          </true>
          // If not, then the calls came back, so assign the results.
          <false>
            <assign property="context.ResultsFromEast"
                    value='syncresponses.GetAt("A")'
                    action="append"/>
            <assign property="context.ResultsFromWest"
                    value='syncresponses.GetAt("B")'
                    action="append"/>
          </false>
        </if>
        </sequence>

        <sequence name="thread2">
        // In this context, syncresponses refers to results in thread2
          <call name="C" />
          <call name="A" />
          <sync calls="C,A" type="all"/>
          // Assign the results
          <assign property="context.ResultsFromNorth"
                  value='syncresponses.GetAt("C")'
                  action="append"/>
          <assign property="context.ResultsFromSouth"
                  value='syncresponses.GetAt("A")'
                  action="append"/>
        </sequence>
      </flow>

    // In this context, syncresponses refers to the primary process thread
    <call name="E" />
  </sequence>
</process>
}
```

The <if> activity in this example has a condition that tests *synctimedout* against the integer value 1. *synctimedout* can have the value 0, 1, or 2 as described in the documentation for <call>. If the two values are equal, this <if> condition receives the integer value 1 and statements inside the <true> element are executed. Otherwise, statements inside the <false> element are executed.

allowresync

The BPL business process can make the <call> and then <sync> on this call multiple times, with or without a timeout. The <sync> *allowresync* attribute controls this behavior. If you set *allowresync* to 1 (true) this enables a subsequent <sync> on the same <call>. You can do this repeatedly until the call completes. A value of 0 (false) for <sync> *allowresync* disallows a subsequent <sync> on the same call. The default *allowresync* value is 0.

Suppose you have an asynchronous <call> A, a long-running activity whose response can be indefinitely delayed. Suppose you <sync> on A with a *timeout* of 5. This <sync> returns immediately if A is complete, or returns in 5 seconds if A is not complete but the timeout expires. Now, suppose you know that A can take an indefinite amount of time, but generally returns without problems. That is, suppose A usually completes within 5 seconds, but sometimes takes over an hour, and that the delay is acceptable when it occurs. In this case, you will want to check A frequently for completion, in case it does complete in the usual time, but also allow subsequent <sync> activities on the same <call>, in case it takes longer to complete.

The following would be typical usage:

XML

```
<sequence>
  <call name="A" async="1" />
  <sync call="A" timeout="5" allowresync="1" />
  <while condition='synctimedout=1'>
    <alert value="Waiting for call A to complete."/>
    <sync call="A" timeout="5" allowresync="1" />
  </while>
</sequence>
```

If a timeout is not specified in the <sync>, then it is important to check the *synctimedout* variable before each <sync>. Otherwise the <sync> could be waiting for a call that has already completed.

Consecutive <sync> Timeout

Suppose you have multiple consecutive <sync> elements that refer to the same <call> element, and each <sync> has a *timeout* value. Once the first <sync> has been satisfied, either because the <call> has returned or because the <sync> *timeout* value has expired, the second <sync> element does not wait but instead completes immediately.

XML

```
<sequence>
  <call name="A" async="1" />
  <sync name="Sync1" calls="A" type="all" timeout="60" />
  <sync name="Sync2" calls="A" type="all" timeout="300" />
</sequence>
```

BPL <throw>

Throws a specific, named fault, as a step in a BPL business process.

Syntax

```
<scope>
   <throw fault='"MyFault"'/>
         ...
   <faulthandlers>
     <catch fault='"MyFault"'>
           ...
     </catch>
   </faulthandlers>
</scope>
```

Attributes and Elements

fault attribute

Required. The name of the fault. It can be a literal text string (up to 255 characters) or an expression to be evaluated.

LanguageOverride attribute

Optional. Specifies the scripting language in which any expressions (within in this element) are written.

Can be `"python"`, `"objectscript"`, or `"basic"` (not documented). Default is the language specified in the containing <process> element.

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Description

When a <throw> statement executes, control immediately shifts to the <faulthandlers> block inside the same <scope>, skipping all intervening statements after the <throw>. Inside the <faulthandlers> block, the program attempts to find a <catch> block whose *value* attribute matches the *fault* string expression in the <throw> statement. This comparison is case-sensitive. When you specify a fault string it *needs* the extra set of quotes to contain it, as shown below:

XML

```
<throw fault='"thrown"'/>
```

If there is a <catch> block that matches the fault, the program executes the code within this <catch> block and then exits the <scope>. The program resumes execution at the next statement following the closing </scope> element.

If a fault is thrown, and the corresponding <faulthandlers> block contains *no* <catch> block that matches the fault string, control shifts from the <throw> statement to the <catchall> block inside <faulthandlers>. After executing the contents of the <catchall> block, the program exits the <scope>. The program resumes execution at the next statement following the closing </scope> element. It is good programming practice to ensure that there is always a <catchall> block inside every <faulthandlers> block, to ensure that the program catches any unanticipated errors.

For details, see Handling Errors in BPL.

See Also

- <catch>
- <catchall>
- <compensate>
- <compensationhandlers>
- <faulthandlers>
- <scope>

BPL <trace>

Writes a message to the foreground ObjectScript shell, as a step in a BPL business process.

Syntax

```
<trace value='"The time is: "_$ZDATETIME($H,3)' />
```

Attributes and Elements

value attribute

Required. This is the text for the trace message. It can be a literal text string (up to 255 characters) or an expression to be evaluated.

In an ObjectScript expression, you can also use virtual property syntax using the {} convention.

LanguageOverride attribute

Optional. Specifies the scripting language in which any expressions (within in this element) are written.

Can be `"python"`, `"objectscript"`, or `"basic"` (not documented). Default is the language specified in the containing <process> element.

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Description

The <trace> element writes a message to the ObjectScript shell. <trace> messages appear only if the BPL business process that generates them has been configured to **Run in Foreground** mode.

Trace messages may be written to the Event Log as well as to the console. A system administrator controls this behavior by configuring a production from the **Interoperability** > **Configure** > **Production** page in the Management Portal. If a BPL business process has the **Log Trace Events** option checked, it writes trace messages to the Event Log as well as displaying them at the console. If a trace message is logged, its Event Log entry type is Trace.

The BPL <trace> element generates trace message with User priority; the result is the same as calling the *$$$TRACE* utility from ObjectScript.

Note: For details, see Adding Trace Elements.

BPL <transform>

Transforms one object into another using a data transformation, as a step in a BPL business process.

Syntax

```
<transform class="MyApp.SAPtoJDE" target="context.xform" source="request" />
```

Attributes and Elements

class attribute

Required. The name of the data transformation class that will perform the data transformation. This value can be provided as a literal string, or by using the ObjectScript @ indirection operator to refer to the value of an execution context variable. See details below. Specify the name of a data transformation class.

target attribute

Required. The target (output object) for this data transformation. This is one of the objects in the execution context, or a property of one of these objects. Specify the name of a valid property and object in the execution context.

source attribute

Required. The source (input object) for this data transformation. This is one of the objects visible in the current execution context or a property of one of these objects. Specify the name of a valid property and object in the execution context.

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Description

The <transform> element lets you invoke a data transformation class from within a business process.

A data transformation class (a subclass of Ens.DataTransform) defines a method that takes an instance of an input object and transforms it into an instance of an output object. The transform element invokes this method to automatically transform an object of one type into another using the data transformation class specified by the class attribute.

The source attribute specifies the input object for the transformation. This is an object (or one of its object-valued properties) visible within the business process execution context and should be of the input type expected by the specified data transformation class.

The target attribute specifies the destination of the output object. This is also an object (or one of its object-valued properties) visible within the business process execution context and should be of the output type expected by the specified data transformation class.

Variables in the Execution Context

The <transform> element can refer to the following variables and their properties. Do not use variables not listed here.

Variable	Purpose
context	The *context* object is a general-purpose data container for the business process. *context* has no automatic definition. To define properties of this object, use the <context> element. That done, you may refer to these properties anywhere inside the <process> element using dot syntax, as in: `context.Balance`
request	The *request* object contains any properties of the original request message object that caused this business process to be instantiated. You may refer to *request* properties anywhere inside the <process> element using dot syntax, as in: `request.UserID`
response	The *response* object contains any properties that are required to build the final response message object to be returned by the business process. You may refer to *response* properties anywhere inside the <process> element using dot syntax, as in: `response.IsApproved`. Use the <assign> element to assign values to these properties.

Note: There is more information about the business process execution context in documentation of the <assign> element.

Value of the class Attribute

While the <transform> element lets you invoke a data transformation class from within a business process, the data transformation class itself must already be defined using the Data Transformation Language (DTL), a subset of BPL. For more information about DTL, see Data Transformation Language Reference.

Indirection in the class Attribute

The value of the *class* attribute is a string that identifies the package and class name of a DTL data transformation. The string can be a literal value:

```
<transform class="MyApp.SAPtoJDE" target="context.xform" source="request" />
```

Or the @ indirection operator can be used to access the value of an execution context variable that contains the appropriate string:

```
<call class="@context.nextTransform" target="context.xform" source="request"/>
```

BPL <true>

Performs a set of activities when the condition for an <if> element is true, as a step in a BPL business process.

Syntax

```
<if condition="1">
   <true>
     ...
   </true>
   <false>
     ...
   </false>
</if>
```

Attributes and Elements

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Other elements

Optional. <true> may contain zero or more of the following elements in any combination: <alert>, <assign>, <branch>, <break>, <call>, <code>, <continue>, <delay>, <empty>, <flow>, <foreach>, <if>, <label>, <milestone>, <reply>, <rule>, <scope>, <sequence>, <sql>, <switch>, <sync>, <throw>, <trace>, <transform>, <until>, <while>, <xpath>, or <xslt>.

Description

A <true> element is used within an <if> to contain elements that need to be executed if the condition is true.

See Also

- <if>
- <false>

BPL <until>

Performs activities repeatedly *until* a condition is true, as a step in a BPL business process.

Syntax

```
<until condition='context.IsApproved="1"'>
   ...
</until>
```

Attributes and Elements

condition attribute

Required. This expression is evaluated at the end of each pass through the activities in the <until> element. Once true, it stops execution of the <until> element.

LanguageOverride attribute

Optional. Specifies the scripting language in which any expressions (within in this element) are written.

Can be `"python"`, `"objectscript"`, or `"basic"` (not documented). Default is the language specified in the containing <process> element.

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Other elements

Optional. <until> may contain zero or more of the following elements in any combination: <alert>, <assign>, <branch>, <break>, <call>, <code>, <continue>, <delay>, <empty>, <flow>, <foreach>, <if>, <label>, <milestone>, <reply>, <rule>, <scope>, <sequence>, <sql>, <switch>, <sync>, <throw>, <trace>, <transform>, <until>, <while>, <xpath>, or <xslt>.

Description

The <until> element defines a sequence of activities that are repeatedly executed until a logical expression evaluates to the integer value 1 (true). The expression is re-evaluated *after* each loop through the sequence.

To fine-tune loop execution, include <break> and <continue> elements within an <until> element. See the descriptions of these elements for details.

BPL <while>

Performs activities repeatedly *as long as* a condition is true, as a step in a BPL business process.

Syntax

```
<while condition='context.IsApproved="1"'>
   ...
</while>
```

Attributes and Elements

condition attribute

Required. This expression is evaluated before each pass through the activities in the <while> element. Once false, it stops execution of the <while> element.

Specify an expression that evaluates the integer value 1 (if true) or 0 (if false).

LanguageOverride attribute

Optional. Specifies the scripting language in which any expressions (within in this element) are written.

Can be `"python"`, `"objectscript"`, or `"basic"` (not documented). Default is the language specified in the containing <process> element.

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Other elements

Optional. <while> may contain zero or more of the following elements in any combination: <alert>, <assign>, <branch>, <break>, <call>, <code>, <continue>, <delay>, <empty>, <flow>, <foreach>, <if>, <label>, <milestone>, <reply>, <rule>, <scope>, <sequence>, <sql>, <switch>, <sync>, <throw>, <trace>, <transform>, <until>, <while>, <xpath>, or <xslt>.

Description

The <while> element defines a sequence of activities that are repeatedly executed as long as a logical expression evaluates to the integer value 1 (true). The expression is re-evaluated *before* each loop through the sequence.

You can fine-tune loop execution by including <break> and <continue> elements within a <while> element. For example:

XML

```
<while condition="0">

  //...do various things...

  <if condition="somecondition">
    <true>
      <break/>
    </true>
  </if>

  //...do various other things...

</while>
```

BPL <xpath>

Evaluates an XPath expression on a target XML document, as a step in a BPL business process.

Syntax

```
<xpath name="xpath"
        source="request.MetaDataXML"
        property="context.Result" context="/staff/doc"
        expression="name[@last='Marston']"/>
```

Note that the editor uses double quotes around the values of the attributes. Thus if an attribute value needs to include quotes, those must be single quotes as shown here.

Attributes and Elements

source attribute

Required. An expression that yields a stream containing the XML on which the XPath expressions are to be performed. Typically the *source* attribute will name a context or request property.

property attribute

Required. The property (typically a context property) in which to place the result of the evaluation.

context attribute

Required. The document context.

expression attribute

Required. The XPath expression.

prefixmappings attribute

Optional. Specifies prefix mappings for the document. This is a comma-delimited list of prefix-to-namespace mappings. See details below. Specify a string of up to 255 characters.

schemaspec attribute

Optional. The schema specification. Specify a string of up to 255 characters.

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

Description

The <xpath> element allows a business process to evaluate XPath expressions on a target XML document.

When the <xpath> element executes, the *source* stream is processed into an XPath document and then the XPath expressions are evaluated in sequence. The BPL runtime engine automatically manages the lifetime of the documents and caches them to allowing processing to be as efficient as possible

Each *prefixmappings* entry is defined as a prefix, a space, and then the URI to which that prefix maps. This is especially useful if the document defines a default namespace with the `xmlns="http://somenamespaceuri"` syntax, but does not supply an explicit prefix mapping. The following *prefixmappings* string would map the `myprefix` prefix to the `http://somenamespaceuri` URI. Note the space character in the string:

```
prefixmappings="myprefix http://somenamespaceuri"
```

The BPL <xpath> element is intended to support XPath expressions which yield a scalar value, that is a single piece of text, number, date etc. It is *not* intended to deal with expressions that yield an XPath DOM. This means that if the expression does yield a DOM, the target property will *not* be updated. DOM programming is beyond the scope of BPL. If your business needs such processing, then the XPath should be performed in a code block or a call to a utility class.

See Also

- <xslt>

BPL <xslt>

Executes an embedded XSLT transformation, as a step in a BPL business process.

Syntax

```
<xslt name='simon'
       xslurl="https://www.intersystems.com/transform.xsl"
       source="context.a" target="context.b">
   <parameters>
     <parameter name="surname" value="sez"/>
   </parameters>
</xslt>
```

Attributes and Elements

xslurl attribute

Required. URI of the XSLT definition that controls the transformation. The URI may begin with one of the following strings: “file:” “http:” “url:” or “xdata:” Specify a string of up to 255 characters.

source attribute

Required. Name of the source (stream) object. Specify a string of up to 255 characters.

target attribute

Required. Name of the target (stream) object Specify a string of up to 255 characters.

name, *disabled*, *xpos*, *ypos*, *xend*, *yend* attributes

See Common Attributes and Elements.

<annotation> element

See Common Attributes and Elements.

<parameters> element

An optional <parameters> element may appear. Inside the <parameters> container, zero or more <parameter> elements may appear. Each <parameter> element defines an XSLT name-value pair to pass to the stylesheet that controls the XSLT transformation.

xsltversion attribute

Specifies whether the XSLT transformation uses XSLT 1.0 or 2.0. Specify either `"1.0"` or `"2.0"`

Description

The <xslt> element allows you to apply an XSLT transformation during a business process. The <xslt> element transforms an input stream to an output stream via an arbitrary XSLT definition. The XSLT definition may be in an external file, or it may be defined in a class in the same namespace as the BPL business process.

The *source* and *target* stream objects must be declared as properties of the *context* object for the business process. The *context* object is a general-purpose data container for the business process. You may define *context* properties by providing <context> and <property> elements at the beginning of the <process> element. That done, you may refer to these properties anywhere inside the <process> element using dot syntax, as in: `context.MyInputStream` or `context.MyOutputStream`

The *xslurl* string is a URI that identifies the location of the XSLT definition. The *xslurl* value may begin with one of the following strings:

```
file:
http:
url:
xdata:
```

Where `file:`, `http:`, and `url:` have the standard meanings. An `xdata:` string takes this form:

`xdata://`*PackageName*`.`*ClassName*`:`*XDataName*

Where:

- *PackageName*`.`*ClassName* identifies a class in the same namespace as the BPL business process.
- *XDataName* is the name of an XData block within that class that contains the XSLT definition for this <xslt> statement. This convention allows XSLT definitions to be stored inside InterSystems IRIS classes, as an efficient alternative to storing them outside InterSystems IRIS in the local file system or on the Web.

If the XSLT requires parameters, include them in a <parameters> block within the <xslt> element.

See Also

- <parameters>
- <xpath>